Conteúdo

1 DP 1
1.1 Digit DP . . . 1
1.2 LCS . . . 1
1.3 LIS . . . 1
1.4 SOS DP . . . 1
1.5 TSP . . . 1

2 Data Structures 2
2.1 BIT . . . 2
2.2 BIT2D . . . 2
2.3 BIT2DSparse . . . 2
2.4 MinQueue . . . 2
2.5 PrefixSum2D . . . 2
2.6 SegTree . . . 2
2.7 SegTreeIterativa . . . 3
2.8 SegTreeLazy . . . 3
2.9 SegTreeLazyIterativa . . . 4
2.10 SegTreePersistente . . . 4
2.11 SegTreePersistente2x . . . 4
2.12 SegTreeSparse . . . 5
2.13 SegTreeSparseLazy . . . 5
2.14 SparseTable . . . 5
2.15 SqrtDecomposition . . . 6
2.16 orderedSet . . . 6
2.17 sack . . . 6

3 Geometry 6
3.1 3D . . . 6
3.2 Circles . . . 7
3.3 ConvexHull . . . 7
3.4 KDTree . . . 7
3.5 Line . . . 8
3.6 LineContainer . . . 8
3.7 Minkowski . . . 8
3.8 Point . . . 8
3.9 Polygons . . . 9
3.10 Segment . . . 9
3.11 quadtree . . . 9

4 Grafos 10
4.1 2SAT . . . 10
4.2 Bellman-Ford . . . 10
4.3 BlockCutTree . . . 11
4.4 CentroidDecomposition . . . 11
4.5 CentroidMinPath . . . 11
4.6 DSU Persistente . . . 12
4.7 DSU Rollback . . . 12
4.8 DSU . . . 12
4.9 Dijkstra . . . 12
4.10 Dinic . . . 12
4.11 DominatorTree . . . 14
4.12 Euler Path . . . 15
4.13 Euler Tour . . . 15
4.14 Floyd-Warshall . . . 16
4.15 HLD . . . 16
4.16 Kahn's . . . 16
4.17 Kruskal . . . 17
4.18 LCA . . . 17
4.19 MinCostMaxFlow . . . 17
4.20 SCC - Kosaraju . . . 17
4.21 Tarjan . . . 18
4.22 bridges-cutpoints . . . 18
4.23 gridBFS . . . 19

5 Math 19
5.1 CRT . . . 19
5.2 Catalan . . . 19
5.3 Combinatorics . . . 19
5.4 FFT . . . 20
5.5 FFT2in1 . . . 20
5.6 FFTmod . . . 21
5.7 FWHT . . . 21
5.8 GaussElimin . . . 21
5.9 Matrix . . . 22
5.10 NTT . . . 22
5.11 Sieve . . . 23
5.12 StirlingNumbers . . . 23
5.13 fexp . . . 23
5.14 frac . . . 23
5.15 getprimes . . . 23
5.16 mint . . . 24
5.17 random . . . 24
5.18 utils . . . 24

6 Others 24
6.1 BuscaBinariaParalela . . . 24
6.2 BuscaTernaria . . . 25
6.3 Date . . . 25
6.4 Dice . . . 25
6.5 Hungarian . . . 26
6.6 MO . . . 26
6.7 MOTree . . . 26
6.8 stressTest . . . 27

7 String 27
7.1 KMP . . . 27
7.2 Manacher . . . 27
7.3 SuffixArray . . . 27
7.4 Z-Function . . . 27
7.5 ahoCorasick . . . 28
7.6 hash . . . 28
7.7 hash2 . . . 28
7.8 hash2A . . . 29
7.9 trie . . . 29

8 Theorems 29
8.1 Combinatorics . . . 29
8.2 Games . . . 30
8.3 Geometria . . . 30
8.4 Point in Polygon e Truques Geométricos . . . 31
8.5 Grafos . . . 31
8.6 Propriedades Matemáticas . . . 32

9 Number Theory 33
9.1 DP . . . 34
9.1.1 General . . . 34

1 DP

1.1 Digit DP

```
Digit DP - Sum of Digits - O(D^2 * B^2)  (B = Base = 10)

Solve(K)  ->  Retorna a soma dos digitos de todo numero
     X tal que: 0 <= X <= K
dp[D][S][f]  ->  D: Quantidade de digitos; S: Soma dos
     digitos; f: Flag que indica o limite.
int limite[D] -> Guarda os digitos de K.
```

```cpp
ll dp[2][19][170];

int limite[19];

ll digitDP(int idx, int sum, bool flag){
  if(idx < 0) return sum;
  if(~dp[flag][idx][sum]) return dp[flag][idx][sum];

  dp[flag][idx][sum] = 0;
  int lm = flag ? limite[idx] : 9;

  for(int i=0; i<=lm; i++)
    dp[flag][idx][sum] += digitDP(idx-1, sum+i, (flag && i
        == lm));

  return dp[flag][idx][sum];
}

ll solve(ll k){
  memset(dp, -1, sizeof dp);

  int sz=0;
  while(k){
    limite[sz++] = k % 10LL;
    k /= 10LL;
  }

  return digitDP(sz-1, 0, true);
}
```

1.2 LCS

```
-> LCS - Longest Common Subsequence O(N^2)
* If recursive: memset(memo, -1, sizeof memo); LCS(0, 0)
    ;
* RecoverLCS O(N) Recover just one of all the possible
    LCS
```

```cpp
const int MAXN = 5*1e3 + 5;
int memo[MAXN][MAXN];

string s, t;

inline int LCS(int i, int j){
  if(i == s.size() || j == t.size()) return 0;
  if(memo[i][j] != -1) return memo[i][j];

  if(s[i] == t[j]) return memo[i][j] = 1 + LCS(i+1, j+1);

  return memo[i][j] = max(LCS(i+1, j), LCS(i, j+1));
}

int LCS_It(){
  for(int i=s.size()-1; i>=0; i--)
    for(int j=t.size()-1; j>=0; j--)
      if(s[i] == t[j])
        memo[i][j] = 1 + memo[i+1][j+1];
      else
        memo[i][j] = max( memo[i+1][j], memo[i][j+1] );

  return memo[0][0];
}

string RecoverLCS(int i, int j){
  if(i == s.size() || j == t.size()) return "";

  if(s[i] == t[j]) return s[i] + RecoverLCS(i+1, j+1);

  if(memo[i+1][j] > memo[i][j+1]) return RecoverLCS(i+1, j);
```

```
  return RecoverLCS(i, j+1);
}
```

1.3 LIS

-> **LIS - Longest Increasing Subsequence** - O(N Log N)
* For ICREASING sequence, use lower_bound()
* For NON DECREASING sequence, use upper_bound()

```
int LIS(vector<int>& nums){
  vector<int> lis;

  for(auto x : nums){
    auto it = lower_bound(lis.begin(), lis.end(), x);
    if(it == lis.end()) lis.push_back(x);
    else *it = x;
  }
  return (int) lis.size();
}
```

1.4 SOS DP

-> **SOS DP - Sum over Subsets**

Dado que cada mask possui um valor inicial (iVal), computa
para cada mask a soma dos valores de todas as suas submasks.

N -> Numero Maximo de Bits
iVal[mask] -> initial Value / Valor Inicial da Mask
dp[mask]   -> Soma de todos os SubSets

Iterar por todas as submasks: for(int sub=mask; sub>0; sub=(sub-1)&mask)

```
const int N = 20;
ll dp[1<<N], iVal[1<<N];

void sosDP(){ // O(N * 2^N)
    for(int i=0; i<(1<<N); i++)
        dp[i] = iVal[i];

  for(int i=0; i<N; i++)
    for(int mask=0; mask<(1<<N); mask++)
      if(mask&(1<<i))
        dp[mask] += dp[mask^(1<<i)];
}

void sosDPsub(){ // O(3^N) //suboptimal
  for (int mask = 0, i; mask < (1<<N); mask++)
    for(i = mask, dp[mask] = iVal[0]; i>0; i=(i-1) & mask)
        //iterate over all submasks
      dp[mask] += iVal[i];
}
```

1.5 TSP

```
const int MOD = 1e9 + 7;

int n;
int costs[10][10];
int dp[1<<10][10];

void tsp_iter() {
    const int inf = 2e9;
    for(int mask = 0; mask < (1<<n); mask++) {
        for(int i = 0; i < n; i++) {
            dp[mask][i] = inf;
        }
    }
    dp[1][0] = 0;
    for(int mask = 1; mask < (1<<n); mask++) {
        for(int i = 0; i < n; i++) {
            if(mask & (1<<i)) { // visitando
                for(int j = 0; j < n; j++) {
                    if(mask & (1<<j)) { // ja visitado
                        continue;
                    }
                    // atualiza o custo minimo para o estado
                    dp[mask ^ (1<<j)][j] = min(dp[mask ^
                        (1<<j)][j], dp[mask][i] + costs[i][
                        j]);
                }
            }
        }
    }
    int best = inf;
    for(int i = 1; i < n; i++) {
        best = min(best, dp[(1<<n) - 1][i] + costs[i][0]);
    }
    cout << best << "\n";
}
```

```
int n;
int matches[21][21];
ll dp[1<<21];

void tsp_iter() {
    if (n == 1) {
        cout << matches[0][0] << "\n";
        return;
    }
    dp[0] = 1;
    for(int mask = 0; mask < (1<<n); mask++) {
        int i = __builtin_popcount(mask);
        if(i >= n) {
            continue;
        }
        for(int j = 0; j < n; j++) {
            if(mask & (1<<j)) {
                continue;
            }
            if(matches[i][j] == 0) {
                continue;
            }
            dp[mask ^ (1<<j)] += dp[mask] % MOD;
        }
    }
    cout << dp[(1<<n)-1] % MOD << "\n";
}
```

2 Data Structures

2.1 BIT

```
struct BIT {
  vector<int> bit;
  int N;

  BIT(){}
  BIT(int n) : N(n+1), bit(n+1){}

  void update(int pos, int val){
    for(; pos < N; pos += pos&(-pos))
      bit[pos] += val;
  }

  int query(int pos){
    int sum = 0;
    for(; pos > 0; pos -= pos&(-pos))
      sum += bit[pos];
    return sum;
  }
};
```

2.2 BIT2D

Complexity: O(Log^2 N)

```
const int MAXN = 1e3 + 5;

struct BIT2D {
  int bit[MAXN][MAXN];

  void update(int X, int Y, int val){
    for(int x = X; x < MAXN; x += x&(-x))
      for(int y = Y; y < MAXN; y += y&(-y))
        bit[x][y] += val;
  }

  int query(int X, int Y){
    int sum = 0;
    for(int x = X; x > 0; x -= x&(-x))
      for(int y = Y; y > 0; y -= y&(-y))
        sum += bit[x][y];
    return sum;
  }

  void updateArea(int xi, int yi, int xf, int yf, int val);
      //Same of BIT2DSparse
  int queryArea(int xi, int yi, int xf, int yf); //Same of
      BIT2DSparse
};
```

2.3 BIT2DSparse

Sparse Binary Indexed Tree 2D

Recebe o conjunto de pontos que serao usados para fazer os updates e as queries e cria uma BIT 2D esparsa que independe do "tamanho do grid".

Build: O(N Log N) (N -> Quantidade de Pontos)

Query/Update: O(Log N)
IMPORTANTE! **Offline!**

BIT2D(pts); // pts -> vecotor<pii> com todos os pontos em que serao feitas queries ou updates

```
#define pii pair<ll, ll>
#define upper(v, x) (upper_bound(begin(v), end(v), x) - begin(v))

struct BIT2D {
  vector<ll> ord;
  vector<vector<ll>> bit, coord;

  BIT2D(vector<pii> pts){
    sort(begin(pts), end(pts));

    for(auto [x, y] : pts)
      if(ord.empty() || x != ord.back())
        ord.push_back(x);

    bit.resize(ord.size() + 1);
    coord.resize(ord.size() + 1);

    sort(begin(pts), end(pts), [&](pii &a, pii &b){ return a.second < b.second; });

    for(auto [x, y] : pts)
      for(int i=upper(ord, x); i < bit.size(); i += i&-i)
        if(coord[i].empty() || coord[i].back() != y)
          coord[i].push_back(y);

    for(int i=0; i<bit.size(); i++) bit[i].assign(coord[i].size()+1, 0);
  }

  void update(ll X, ll Y, ll v){
    for(int i = upper(ord, X); i<bit.size(); i += i&-i)
      for(int j = upper(coord[i], Y); j < bit[i].size(); j += j&-j)
        bit[i][j] += v;
  }

  ll query(ll X, ll Y){
    ll sum = 0;
    for(int i = upper(ord, X); i > 0; i -= i&-i)
      for(int j = upper(coord[i], Y); j > 0; j -= j&-j)
        sum += bit[i][j];
    return sum;
  }

  ll queryArea(ll xi, ll yi, ll xf, ll yf){
    return query(xf, yf) - query(xf, yi-1) - query(xi-1, yf) + query(xi-1, yi-1);
  }

  void updateArea(ll xi, ll yi, ll xf, ll yf, ll val){ // OPTIONAL
    update(xi,   yi,    val);  // DOESN'T UPDATE AN AREA!!!
    update(xf+1, yi,   -val);  // It is like: bit1d.update(l -1, -v), bit1d.update(r, +v)
    update(xi,   yf+1, -val);  // so you can do like bit1d.query(i) to see the value "at" i
    update(xf+1, yf+1,  val);  // in this case, call bit2d.query(X, Y)
  }
};
```

2.4 MinQueue

```
struct MinQueue : deque<pair<int, int>> {
    int min(){ return front().first; }
    void push(int x){
        int cnt = 1;
        while(!empty() && x <= back().first)
            cnt += back().second, pop_back();
        push_back({x, cnt});
    }
    void pop(){ if(!--front().second) pop_front(); }
};
```

2.5 PrefixSum2D

```
const int MAXN = 1e3 + 5;
int ps [MAXN][MAXN];

void calcPS2d(){
  for (int i = 1; i < MAXN; i++) ps[0][i] += ps[0][i - 1];
       //inicializo a 1a linha
  for (int i = 1; i < MAXN; i++) ps[i][0] += ps[i - 1][0];
       //inicializo a 1a coluna

  for (int i = 1; i < MAXN; i++)
    for (int j = 1; j < MAXN; j++)
      ps[i][j] += ps[i - 1][j] + ps[i][j - 1] - ps[i - 1][j - 1];
}
int queryPS2d(int xi, int yi, int xf, int yf){ return ps[xf][yf] - ps[xf][yi-1] - ps[xi-1][yf] + ps[xi-1][yi-1]; }
```

2.6 SegTree

```
#pragma once

/* Segment Tree (Point Update, Range Query)
 * Complexidade: O(log N) para update e query.
 * Memoria: O(4*N)
 * Requisitos:
 * - NODE deve ter: static merge(L, R), apply(V), e
 * construtor identidade.
 */

/* --- Exemplo de NODE (Soma com Update de Substituicao) ---
struct Node {
    ll val = 0;
    Node(ll v = 0) : val(v) {}

    static Node merge(const Node& l, const Node& r) {
        return Node(l.val + r.val);
    }

    // Para Point Set: val = v;
    // Para Point Add: val += v;
    void apply(ll v) {
        val = v;
    }
};
*/

template <typename NODE>
struct SegTree {
  int N;
  vector<NODE> seg;

  explicit SegTree(int n) : N(n), seg(4 * n) {}

  template <typename T>
  SegTree(const vector<T> &v) : SegTree((int)v.size()) {
    build(1, 0, N - 1, v);
  }

  template <typename T>
  void build(int no, int l, int r, const vector<T> &v) {
    if (l == r) {
      seg[no] = NODE(v[l]);
      return;
    }
    int m = (l + r) >> 1;
    build(no << 1, l, m, v);
    build((no << 1) | 1, m + 1, r, v);
    seg[no] = NODE::merge(seg[no << 1], seg[(no << 1) | 1]);
  }

  template <typename V>
  void update(int no, int l, int r, int idx, V val) {
    if (l == r) {
      seg[no].apply(val);
      return;
    }
    int m = (l + r) >> 1;
    if (idx <= m) update(no << 1, l, m, idx, val);
    else update((no << 1) | 1, m + 1, r, idx, val);
    seg[no] = NODE::merge(seg[no << 1], seg[(no << 1) | 1]);
  }

  NODE query(int no, int l, int r, int a, int b) {
    if (b < l || r < a) return NODE();
    if (a <= l && r <= b) return seg[no];
    int m = (l + r) >> 1;
    return NODE::merge(query(no << 1, l, m, a, b),
                       query((no << 1) | 1, m + 1, r, a, b));
  }

  template <typename V>
  void update(int idx, V val) {
    update(1, 0, N - 1, idx, val);
  }
  NODE query(int l, int r) { return query(1, 0, N - 1, l, r); }
};
```

2.7 SegTreeIterativa

```
template<typename T> struct SegTree {
  int n;
  vector<T> seg;
  T join(T&l, T&r){ return l + r; }

  SegTree(int n) : n(n), seg(2*n) {}
  SegTree(){}
  void init(vector<T>&base){
    n = base.size();
    seg.resize(2*n);
    for(int i=0; i<n; i++) seg[i+n] = base[i];
    for(int i=n-1; i>0; i--) seg[i] = join(seg[i*2], seg[i*2+1]);
  }

  T query(int l, int r){ //[L, R] // [0, n-1]
    T lp = 0, rp = 0; //NEUTRO
    for(l+=n, r+=n+1; l<r; l/=2, r/=2){
```

```
      if(l&1) lp = join(lp, seg[l++]);
      if(r&1) rp = join(seg[--r], rp);
    }
    return join(lp, rp);
  }

  void update(int i, T v){ // Set Value seg[i+=n] = v //
       change to += v to sum
    for(seg[i+=n] = v; i/=2;) seg[i] = join(seg[i*2], seg[i
        *2+1]);
  }
};
```

2.8 SegTreeLazy

```
-> Segment Tree - Lazy Propagation com:
  - Query em Range
  - Update em Range
  - Closed interval & 0-indexed: [L, R] & [0, N-1]
Build:  O(N)
Query:  O(log N) | seg.query(l, r);
Update: O(log N) | seg.update(l, r, v);
Unlazy: O(1)
Update Join, NEUTRO, Update and Unlazy if needed
```

```
// CREDITOS: https://github.com/src-lua/lgf-cpLib/blob/main/
     lib/data-structures/segment-tree/lazy-segment-tree.hpp

/* Lazy Segment Tree (Range Update, Range Query)
 * Realiza atualizacoes e consultas em range.
 * Complexidade: O(log N) para update e query.
 * Memoria: O(4*N)
 * Requisitos:
 * - NODE deve ter: static merge(L, R), apply(TAG, L, R) e
 *   construtor identidade.
 * - TAG deve ter: compose(TAG) e construtor identidade.
 */

/* --- Exemplo de NODE e TAG (Soma e Multiplicacao com
     Modulo) ---
struct Tag {
    ll mul = 1, add = 0;
    void inline compose(const Tag& t) {
        add = (add * t.mul + t.add) % MOD;
        mul = (mul * t.mul) % MOD;
    }
};

struct Node {
    ll val = 0;
    Node(ll v = 0) : val(v) {}
    static inline Node merge(const Node& l, const Node& r) {
        return Node((l.val + r.val) % MOD);
    }
    void inline apply(const Tag& t, int l, int r) {
        val = (val * t.mul + t.add * (r - l + 1)) % MOD;
    }
};
*/

template<typename NODE, typename TAG>
struct LazySegmentTree {
    int N;
    vector<NODE> seg;
    vector<TAG> lazy;

    LazySegmentTree(int n) : N(n), seg(4 * n), lazy(4 * n)
        {}

    template<typename T>
    LazySegmentTree(const vector<T>& v)
        : N(v.size()), seg(4 * v.size()), lazy(4 * v.size())
            {
        build(1, 0, N - 1, v);
    }

    template<typename T>
    void build(int no, int l, int r, const vector<T>& v) {
        if (l == r) {
            seg[no] = NODE(v[l]);
            return;
        }
        int m = (l + r) >> 1;
        build(no << 1, l, m, v);
        build((no << 1) | 1, m + 1, r, v);
        seg[no] = NODE::merge(seg[no << 1], seg[(no << 1) |
            1]);
    }

    void push(int no, int l, int r) {
        int m = (l + r) >> 1;
        int e = no << 1, d = e | 1;

        seg[e].apply(lazy[no], l, m);
        lazy[e].compose(lazy[no]);

        seg[d].apply(lazy[no], m + 1, r);
        lazy[d].compose(lazy[no]);

        lazy[no] = TAG();
    }

    void update(int no, int l, int r, int a, int b, const
         TAG& v) {
        if (b < l || r < a) return;
        if (a <= l && r <= b) {
            seg[no].apply(v, l, r);
            lazy[no].compose(v);
            return;
        }
        push(no, l, r);
        int m = (l + r) >> 1;
        update(no << 1, l, m, a, b, v);
        update((no << 1) | 1, m + 1, r, a, b, v);
        seg[no] = NODE::merge(seg[no << 1], seg[(no << 1) |
            1]);
    }

    NODE query(int no, int l, int r, int a, int b) {
        if (b < l || r < a) return NODE();
        if (a <= l && r <= b) return seg[no];
        push(no, l, r);
        int m = (l + r) >> 1;
        return NODE::merge(query(no << 1, l, m, a, b),
                           query((no << 1) | 1, m + 1, r, a,
                                b));
    }

    void update(int l, int r, const TAG& v) { update(1, 0, N
         - 1, l, r, v); }
    NODE query(int l, int r) { return query(1, 0, N - 1, l,
        r); }
};
```

2.9 SegTreeLazyIterativa

```
template<typename T> struct SegTree {
  int n, h;
  T NEUTRO = 0;
  vector<T> seg, lzy;
  vector<int> sz;
  T join(T&l, T&r){ return l + r; }

  void init(int _n){
    n = _n;
    h = 32 - __builtin_clz(n);
    seg.resize(2*n);
    lzy.assign(n, NEUTRO);
    sz.resize(2*n, 1);
    for(int i=n-1; i; i--) sz[i] = sz[i*2] + sz[i*2+1];
    // for(int i=0; i<n; i++) seg[i+n] = base[i];
    // for(int i=n-1; i; i--) seg[i] = join(seg[i*2], seg[i
        *2+1]);
  }

  void apply(int p, T v){
    seg[p] += v * sz[p]; // cumulative?
    if(p < n) lzy[p] += v;
  }
  void push(int p){
    for(int s=h, i=p>>s; s; s--, i=p>>s)
      if(lzy[i] != 0) {
        apply(i*2,   lzy[i]);
        apply(i*2+1, lzy[i]);
        lzy[i] = NEUTRO; //NEUTRO
      }
  }
  void build(int p) {
    for(p/=2; p; p/= 2){
      seg[p] = join(seg[p*2], seg[p*2+1]);
      if(lzy[p] != 0) seg[p] += lzy[p] * sz[p];
    }
  }

  T query(int l, int r){ //[L, R] & [0, n-1]
    l+=n, r+=n+1;
    push(l); push(r-1);

    T lp = NEUTRO, rp = NEUTRO; //NEUTRO
    for(; l<r; l/=2, r/=2){
      if(l&1) lp = join(lp, seg[l++]);
      if(r&1) rp = join(seg[--r], rp);
    }
    return ans;
  }

  void update(int l, int r, T v){
    l+=n, r+=n+1;
    push(l); push(r-1);

    int l0 = l, r0 = r;
    for(; l<r; l/=2, r/=2){
      if(l&1) apply(l++, v);
      if(r&1) apply(--r, v);
    }
    build(l0); build(r0-1);
  }
};
```

2.10 SegTreePersistente

```
-> Segment Tree Persistente
Build(1, N) -> Cria uma Seg Tree completa de tamanho N;
     RETORNA um *Ponteiro pra Raiz
Update(Root, 1, N, pos, v)  -> Soma +V na posicao POS;
     RETORNA um *Ponteiro pra Raiz da nova versao;
Query(Root, 1, N, a, b)   -> RETORNA o valor calculado
     no range [a, b];
Kth(RootL, RootR, 1, N, K)  -> Faz uma Busca Binaria na
     Seg; Mais detalhes abaixo;
[ Root -> No Raiz da Versao da Seg na qual se quer
     realizar a operacao ]
Para guardar as Raizes, use: vector<Node*> roots

Build:  O(N) !!! Sempre chame o Build
Query:  O(log N)
Update: O(log N)
Kth:  O(Log N)

Comportamento do K-th(SegL, SegR, 1, N, K):
  -> Retorna indice da primeira posicao i cuja soma de
       prefixos [1, i] e >= k
  na Seg resultante da subtracao dos valores da (Seg R)
       - (Seg L).
  -> Pode ser utilizada para consultar o K-esimo menor
       valor no intervalo [L, R] de um array.
  A Seg deve ser utilizada como um array de frequencias.
        Comece com a Seg zerada (Build).
  Para cada valor V do Array chame um update(roots.back
       (), 1, N, V, 1) e guarde o ponteiro da seg.
  Consultar o K-esimo menor valor de [L, R]: chame kth(
       roots[L-1], roots[R], 1, N, K);
```

```
struct Node {
  int val = 0;
  Node *L = NULL, *R = NULL;
  Node(int v = 0) : val(v), L(NULL), R(NULL) {}
};

Node* build(int l, int r){
  if(l == r) return new Node();

  int m = (l+r)/2;

  Node *node = new Node();

  node->L = build(l,   m);
  node->R = build(m+1, r);
  node->val = node->L->val + node->R->val;

  return node;
}

Node* update(Node *node, int l, int r, int pos, int v){
  if( pos < l || r < pos ) return node;
  if(l == r) return new Node(node->val + v);

  int m = (l+r)/2;

  Node *nw = new Node();

  nw->L = update(node->L, l,   m, pos, v);
  nw->R = update(node->R, m+1, r, pos, v);
  nw->val = nw->L->val + nw->R->val;

  return nw;
}

int query(Node *node, int l, int r, int a, int b){
  if(b <  l || r <  a || node == NULL) return 0;

  if(a <= l && r <= b) return node->val;

  int m = (l+r)/2;

  return query(node->L, l, m, a, b) + query(node->R, m+1, r,
       a, b);
}

int kth(Node *Left, Node *Right, int l, int r, int k){
  if(l == r) return l;

  int sum = Right->L->val - Left->L->val;
  int m = (l+r)/2;

  if(sum >= k) return kth(Left->L, Right->L, l, m, k);
  return kth(Left->R, Right->R, m+1, r, k - sum);
}
```

2.11 SegTreePersistente2x

```
-> Segment Tree
     Persistente: (2x mais rapido que com ponteiro)
Build(1, N) -> Cria uma Seg Tree completa de tamanho N;
     RETORNA o NodeId da Raiz
Update(Root, pos, v) -> Soma +V em POS; RETORNA o NodeId
      da nova Raiz;
Query(Root, a, b) -> RETORNA o valor do range [a, b];
Kth(RootL, RootR, K) -> Faz uma Busca Binaria na Seg de
     diferenca entre as duas versoes.
[ Root -> No Raiz da Versao da Seg na qual se quer
     realizar a operacao ]

Build:  O(N)  !!! Sempre chame o Build
Query:  O(log N)
Update: O(log N)
Kth:  O(Log N)

Comportamento do K-th(SegL, SegR, 1, N, K):
  -> Retorna indice da primeira posicao i cuja soma de
       prefixos [1, i] e >= k na Seg resultante da
       subtracao dos valores da (Seg R) - (Seg L).
  -> Pode ser utilizada para consultar o K-esimo menor
       valor no intervalo [L, R] de um array.
  A Seg deve ser utilizada como um array de frequencias.
        Comece com a Seg zerada (Build).
  Para cada valor V do Array chame um update(roots.back
       (), 1, N, V, 1) e guarde o ponteiro da seg.
  Consultar o K-esimo menor valor de [L, R]: chame kth(
       roots[L-1], roots[R]);
```

```
const int MAXN = 1e5 + 5;
const int MAXLOG = 31 - __builtin_clz(MAXN) + 1;
typedef int NodeId;
typedef int STp;

const STp NEUTRO = 0;
int IDN, LSEG, RSEG;
extern struct Node NODES[];

struct Node {
  STp val;
  NodeId L, R;
  Node(STp v = NEUTRO) : val(v), L(-1), R(-1) {}
  Node& l(){ return NODES[L]; }
  Node& r(){ return NODES[R]; }
```

```
};

Node NODES[4*MAXN + MAXLOG*MAXN]; //!!!CUIDADO COM O TAMANHO
      (aumente se necessario)
pair<Node&, NodeId> newNode(STp v = NEUTRO){ return {NODES[
     IDN] = Node(v), IDN++}; }

STp join(STp lv, STp rv){ return lv + rv; }

NodeId build(int l, int r, bool root=true){
  if(root) LSEG = l, RSEG = r;
  if(l == r) return newNode().second;

  int m = (l+r)/2;
  auto [node, id] = newNode();

  node.L = build(l,   m, false);
  node.R = build(m+1, r, false);
  node.val = join(node.l().val, node.r().val);

  return id;
}

NodeId update(NodeId node, int l, int r, int pos, int v){
  if( pos < l || r < pos ) return node;
  if(l == r) return newNode(NODES[node].val + v).second;

  int m = (l+r)/2;
  auto [nw, id] =newNode();

  nw.L = update(NODES[node].L, l,   m, pos, v);
  nw.R = update(NODES[node].R, m+1, r, pos, v);

  nw.val = join(nw.l().val, nw.r().val);

  return id;
}
NodeId update(NodeId node, int pos, STp v){ return update(
     node, LSEG, RSEG, pos, v); }

int query(Node& node, int l, int r, int a, int b){
  if(b <  l || r <  a) return NEUTRO;
  if(a <= l && r <= b) return node.val;

  int m = (l+r)/2;

  return join(query(node.l(), l, m, a, b), query(node.r(), m
       +1, r, a, b));
}
int query(NodeId node, int a, int b){ return query(NODES[
     node], LSEG, RSEG, a, b); }

int kth(Node& Left, Node& Right, int l, int r, int k){
  if(l == r) return l;

  int sum =Right.l().val - Left.l().val;
  int m = (l+r)/2;

  if(sum >= k) return kth(Left.l(), Right.l(), l, m, k);
  return kth(Left.r(), Right.r(), m+1, r, k - sum);
}
int kth(NodeId Left, NodeId Right, int k){ return kth(NODES[
     Left], NODES[Right], LSEG, RSEG, k); }
```

2.12 SegTreeSparse

```
template<typename T> struct SparseSeg {
  struct Node {
    T val = 0; Node *L = NULL, *R = NULL;
    Node(T v = 0) : val(v), L(NULL), R(NULL) {}
  };
  int n; Node* root;
  SparseSeg(int n) : n(n){ root = new Node(NEUTRO); }
  T join(T lv, T rv){ return max(lv, rv); }
  const T NEUTRO = 0;

  T query(Node*& no, int l, int r, int a, int b){
    if(b <  l || r <  a || no == NULL) return NEUTRO;
    if(a <= l && r <= b) return no->val;
    int m=(l+r)/2;
    return join(query(no->L, l, m, a, b), query(no->R, m+1,
         r, a, b));
  }
  void update(Node*& no, int l, int r, int pos, T v){
    if(!no) no = new Node(NEUTRO);
    if(pos < l || r < pos) return;
    if(l == r) return (void)(no->val = v);
    int m=(l+r)/2;

    update(no->L,  l,  m, pos, v);
    update(no->R, m+1, r, pos, v);

    no->val = join(no->L->val, no->R->val);
  }

  void update(int pos, T v){ update(root, 0, n, pos, v); }
  T query(int a, int b){ return query(root, 0, n, a, b); }
};
```

2.13 SegTreeSparseLazy

```
template<typename T> struct SparseSeg {
  struct Node {
    T val = 0, lazy=0; Node *L = NULL, *R = NULL;
    Node(T v = 0) : val(v), L(NULL), R(NULL) {}
  };
  int n; Node* root;
  SparseSeg(int n) : n(n){ root = new Node(NEUTRO); }
  T join(T lv, T rv){ return (lv + rv); }
  const T NEUTRO = 0;

  T query(Node*& no, int l, int r, int a, int b){
    if(!no) no = new Node(NEUTRO); unlazy(no, l, r);
    if(b <  l || r <  a) return NEUTRO;
    if(a <= l && r <= b) return no->val;
    int m=(l+r)/2;
    return join(query(no->L, l, m, a, b), query(no->R, m+1,
         r, a, b));
  }
  void update(Node*& no, int l, int r, int a, int b, T v){
    if(!no) no = new Node(NEUTRO); unlazy(no, l, r);
    if(b <  l || r <  a) return;
    if(a <= l && r <= b){ no->lazy += v; return unlazy(no, l
         , r); }
    int m=(l+r)/2;

    update(no->L,  l,  m, a, b, v);
    update(no->R, m+1, r, a, b, v);

    no->val = join(no->L->val, no->R->val);
  }
  void unlazy(Node* no, int l, int r){ //delete this if not
       lazy
    if(no->lazy == 0) return;
    if(l != r){
      if(!no->L) no->L = new Node(no->val);
      if(!no->R) no->R = new Node(no->val);
      no->L->lazy += no->lazy;
      no->R->lazy += no->lazy;
    }
    no->val += no->lazy * (r-l+1);
    no->lazy = 0;
  }

  void update(int a, int b, T v){ update(root, 0, n, a, b, v
       ); }
  void update(int pos, T v){ update(root, 0, n, pos, pos, v)
       ; }
  T query(int a, int b){ return query(root, 0, n, a, b); }
};
```

2.14 SparseTable

Sparse Table for Range Minimum Query [L, R] [0, N-1]
build: O(N log N) Query: O(1)
build(v) -> v = Original Array
if you want a static array, do this: for(int i=0; i<N; i ++) table[0][i] = v[i];

```
template<typename T> struct Sparse {
  vector<vector<T>> table;

  void build(vector<T> &v){
    int N = v.size(), MLOG = 32 - __builtin_clz(N);
    table.assign(MLOG, v);

    for(int p=1; p < MLOG; p++)
      for(int i=0; i + (1 << p) <= N; i++)
        table[p][i] = min(table[p-1][i], table[p-1][i+(1<<(p
             -1))]);
  }

  T query(int l, int r){
    int p = 31 - __builtin_clz(r - l + 1);  //floor log
    return min(table[p][l], table[p][ r - (1<<p)+1 ]);
  }
};
```

2.15 SqrtDecomposition

```
struct sqrt_decomp {
  int n, block_size, num_blocks;
  vector<int> a;
  vector<long long> b;

  sqrt_decomp() {}
  sqrt_decomp(vector<int>& arr) : n(arr.size()), block_size(
       sqrt(n) + 1), a(arr) {
    num_blocks = (n + block_size - 1) / block_size;
    b.assign(num_blocks, 0);

    for (int i = 0; i < n; i++) {
      b[i / block_size] += a[i];
    }
  }

  void update(int pos, int val) {
```

```
    b[pos / block_size] += (long long)val - a[pos];
    a[pos] = val;
  }

  long long query(int l, int r) {
    long long sum = 0;
    int b_l = l / block_size, b_r = r / block_size;

    if (b_l == b_r) {
      for (int i = l; i <= r; i++) sum += a[i];
    }
    else {
      for (int i = l; i <= block_size * (b_l + 1) - 1; i++)
          sum += a[i];
      for (int i = block_size * b_r; i <= r; i++) sum += a[i
          ];
      for (int i = b_l + 1; i <= b_r - 1; i++) sum += b[i];
    }
    return sum;
  }
};
```

2.16 orderedSet

```
#include "bits/stdc++.h"

using namespace __gnu_pbds;

template <typename T>
using ordered_set = tree <T, null_type, less <T>,
     rb_tree_tag, tree_order_statistics_node_update>;

// use pair<int,int> -> value,id for multiset
class Policy {
private:
    ordered_set<int> pbds;
public:
    Policy() {
        pbds.clear();
    }
    void add(int value) {
        pbds.insert(value);
    }
    void remov(int value) {
        auto fnd = pbds.find(value);
        if (fnd != pbds.end()) {
            pbds.erase(fnd);
        }
    }
    int kth(int k) {
        return *pbds.find_by_order(k); // zero-based
    }
    int countLess(int value) {
        return pbds.order_of_key(value);
    }
    int countEqual(int value) {
        return countLess(value + 1) - countLess(value);
    }
    int countLessOrEqual(int value) {
        return countLess(value + 1);
    }
    int countGreater(int value) {
        return pbds.size() - countLess(value + 1);
    }
};
```

2.17 sack

```
// Time Complexity: O(N * log(N))

int timer;
int cnt[nax], big[nax], fr[nax], to[nax], who[nax];
vector<int> g[nax];

// Ans queries
int color[nax];
int ans[nax];
int distinct_colors = 0;
//

int pre(int u, int p) {
    int sz = 1, tmp;
    who[timer] = u;
    fr[u] = timer++;
    ii best = {-1, -1};

    for (int v : g[u]) {
        if (v == p) continue;
        tmp = pre(v, u);
        sz += tmp;
        best = max(best, {tmp, v});
    }

    big[u] = best.se;
    to[u] = timer - 1;
    return sz;
}

void add(int u, int x) {
    int c = color[u];

    if (x == 1) {
        if (cnt[c] == 0) distinct_colors++;
        cnt[c]++;
    } else {
        cnt[c]--;
        if (cnt[c] == 0) distinct_colors--;
    }
}

void dfs(int u, int p, bool keep = true) {
    for (int v : g[u]) {
        if (v != p && v != big[u]) dfs(v, u, false);
    }

    if (big[u] != -1) dfs(big[u], u, true);

    for (int v : g[u]) {
        if (v != p && v != big[u]) {
            for (int i = fr[v]; i <= to[v]; ++i) {
                add(who[i], 1);
            }
        }
    }
    add(u, 1);

    //
    ans[u] = distinct_colors;
    //

    if (!keep) {
        for (int i = fr[u]; i <= to[u]; ++i) {
            add(who[i], -1);
        }
    }
}

void solve(int root) {
    timer = 0;
    distinct_colors = 0;

    pre(root, root);
    dfs(root, root);
}
```

3 Geometry

3.1 3D

Pode reusar:
- segmentDist

```
#define ld double

struct PT {
  ld x, y, z;
  PT(ld x, ld y, ld z) : x(x), y(y), z(z) {}
  PT() : x(0), y(0), z(0) {}

  PT operator+(const PT&a)const{return PT(x+a.x, y+a.y, z+a.
      z);}
  PT operator-(const PT&a)const{return PT(x-a.x, y-a.y, z-a.
      z);}
  PT operator%(const PT&a)const{return PT(y*a.z - z*a.y, z*a
      .x - x*a.z, x*a.y - y*a.x);}
  PT operator*(ld c) const{ return PT(x*c, y*c, z*c); }
  PT operator/(ld c) const{ return PT(x/c, y/c, z/c); }
  ld operator*(const PT&a)const{return  (x*a.x + y*a.y + z*a
      .z);}
  bool operator< (const PT&a) const{ return tie(x, y, z) <
      tie(a.x, a.y, a.z); }
  ld len() const { return hypot(x,y,z); } // sqrt(p*p)

  PT rotate(double angle, PT axis) {
        double s = sin(angle), c = cos(angle); PT u = axis /
            axis.len();
        return u*(*this*u)*(1-c) + (*this)*c - *this%u*s;
    };
};
```

3.2 Circles

The **circumcirle of a triangle** is the circle intersecting all three vertices.

```
double ccRadius(PT& A, PT& B, PT& C) {
  return (B-A).len()*(C-B).len()*(A-C).len() / abs(A.cross(B
      , C))/2;
}
PT ccCenter(PT& A, PT& B, PT& C) {
  PT b = C-A, c = B-A;
  return A + rotateCCW90(b*(c*c) - c*(b*b)) / (b%c) / 2;
}
```

Return the points at **two circles intersection**. If none or infinity, returns empty

```
vector<PT> circleCircleInter(PT a, ld r1, PT b, ld r2){
  if (a == b) return {}; //r1==r2? infinity : none
  PT v = b-a;

  ld d2 = v*v, sum = r1+r2, dif = r1-r2;
  ld p = (d2 + r1*r1 - r2*r2) / (d2+d2), h2 = r1*r1 - p*p*d2
       ;

  if(sum*sum < d2 || dif*dif > d2) return {};

  PT mid=a+v*p, per=rotateCCW90(v)*sqrt(fmax(0, h2) / d2);

  set<PT> ans = {mid + per, mid - per};
  return {begin(ans), end(ans)};
}
```

Return the **circle line intersection.** Return a vector of 0,1 or 2 PTs

```
vector<PT> circleLineInter(PT c, ld r, PT a, PT b){
  PT ab = b-a;
  PT p = a + ab * ((c-a)*ab) / (ab*ab);
  ld s = a.cross(b, c);
  ld h2 = r*r - s*s / (ab*ab);
  if(h2 < 0) return {};
  if(h2 == 0) return {p};
  PT h = ab/ab.len() * sqrt(h2);
  return {p - h, p + h};
}
```

Returns the **minimum enclosing circle** for a set of points. Expected O(n)

```
pair<PT, ld> minEnclose(vector<PT> ps) {
  shuffle(begin(ps), end(ps), mt19937(time(0)));
  PT o = ps[0];
  ld r=0, EPS = 1 + 1e-8;

  for(int i=0; i<ps.size(); i++) if(dist(o, ps[i]) > r*EPS){
    o = ps[i], r = 0;

    for(int j=0; j<i; j++) if(dist(o, ps[j]) > r*EPS){
      o = (ps[i] + ps[j]) / 2;
      r = dist(o, ps[i]);

      for(int k=0; k<j; k++) if(dist(o, ps[k]) > r*EPS){
        o = ccCenter(ps[i], ps[j], ps[k]);
        r = (o - ps[i]).len();
      }
    }
  }

  return {o, r};
}
```

3.3 ConvexHull

Given a vector of points, return the convex hull in CCW order.
A convex hull is the smallest convex polygon that contains all the points.

If you want colinear points in border, change the >=0 to >0 in the while's.
WARNING: if collinear and all input PT are collinear, may have duplicated points (the round trip)

```
vector<PT> ConvexHull(vector<PT> pts){
  sort(begin(pts), end(pts));
  pts.resize(unique(begin(pts), end(pts)) - begin(pts));
  if(pts.size() <= 1) return pts;

  int s=0, n=pts.size();
  vector<PT> h(2*n+1);

  for(int i=0; i<n; h[s++] = pts[i++])
    while(s > 1 && h[s-2].cross(pts[i], h[s-1]) >= 0 )
      s--;

  for(int i=n-2, t=s; ~i; h[s++] = pts[i--])
    while(s > t && h[s-2].cross(pts[i], h[s-1]) >= 0 )
      s--;

  h.resize(s-1);
  return h;
} //PT operators needed: {- % == <}
```

Check **if a point is inside convex hull** (CCW, no collinear).If strict == true, then pt on boundary return falseO(log N)

```
bool isInside(const vector<PT>& h, PT p, bool strict = true)
    {
  int a = 1, b = h.size() - 1, r = !strict;
  if(h.size() < 3) return r && onSegment(h[0], h.back(), p);
  if(h[0].cross(h[a], h[b]) > 0) swap(a, b);
  if(h[0].cross(h[a], p) >= r || h[0].cross(h[b], p) <= -r)
      return false;
  while(abs(a-b) > 1){
    int c = (a + b) / 2;
    if(h[0].cross(h[c], p) > 0) b = c;
    else a = c;
  }
  return h[a].cross(h[b], p) < r;
}
```

Check **if a point is inside convex hull** O(log N)

```
bool isInside(const vector<PT> &h, PT p){
  if(h[0].cross(p, h[1]) > 0 || h[0].cross(p, h.back()) < 0)
      return false;
    int n = h.size(), l=1, r = n-1;
    while(l != r){
      int mid = (l+r+1)/2;
      if(h[0].cross(p, h[mid]) < 0) l = mid;
      else r = mid - 1;
    }
    return h[l].cross(h[(l+1)%n], p) >= 0;
}
```

Given a convex hull h and a point p, returns the indice of h **where the dot product is maximized.**This code assumes that there are NO 3 colinear points!

```
int maximizeScalarProduct(const vector<PT> &h, PT v) {
    int ans = 0, n = h.size();
    if(n < 20){
    for(int i=0; i<n; i++)
            if(v*h[ans] < v*h[i])
                ans = i;
    return ans;
    }

  for(int rep=0; rep<2; rep++){
    int l = 2, r = n-1;
    while(l != r){
      int mid = (l+r+1)/2;
      int f = v*h[mid] >= v*h[mid-1];

      if(rep) f |= v*h[mid-1] < v*h[0];
      else    f &= v*h[mid]  >= v*h[0];
      if(f) l = mid;
      else  r = mid - 1;
    }
    if(v*h[ans] < v*h[l]) ans = l;
  }
  if(v*h[ans] < v*h[1] ) ans = 1;
  return ans;
}
```

3.4 KDTree

```
struct pt{
  ld x, y;
  pt(){}
  pt(ld x, ld y): x(x), y(y){}
  pt operator+(pt p){ return pt(x+p.x, y+p.y); }
  pt operator-(pt p){ return pt(x-p.x, y-p.y); }
  ld operator*(pt p){ return x*p.x + y*p.y; }
  ld norm2(){ return *this * *this; }
  bool operator<(pt p)const{ // for sort, convex hull/set/
      map
    return x < p.x - eps || (abs(x - p.x) <= eps && y < p.y
        - eps); }
};
inline bool onx(pt a, pt b){ return a.x < b.x; }
inline bool ony(pt a, pt b){ return a.y < b.y; }
// Given a set of N points, answer queries of nearest point
    in O(log(N))
struct Node {
  pt pp;
  ll x0 = inf, x1 = -inf, y0 = inf, y1 = -inf;
  Node *fir = 0, *sec = 0;
  inline ll distance(pt p){
    ll x = min(max(x0, p.x), x1);
    ll y = min(max(y0, p.y), y1);
    return (pt(x, y) - p).norm2();
  }
  Node(vector<pt>&& vp): pp(vp[0]){
    for(pt& p: vp){
      x0 = min(x0, p.x), x1 = max(x1, p.x);
      y0 = min(y0, p.y), y1 = max(y1, p.y);
    }
    if(sz(vp) > 1){
      sort(all(vp), x1 - x0 >= y1 - y0 ? onx : ony);
      int m = sz(vp) / 2;
      fir = new Node({vp.begin(), vp.begin() + m});
      sec = new Node({vp.begin() + m, vp.end()});
    }
  }
};
struct KDTree {
  Node* root;
  KDTree(const vector<pt>& vp):root(new Node({all(vp)})) {}
  pair<ll, pt> search(pt p, Node *node){
    if(!node->fir){ // To avoid query point as answer:
      // ADD:  if(p == node -> pp) return {INF, pt()};
      return {(p - node->pp).norm2(), node->pp};
    }
    Node *f = node->fir, *s = node->sec;
    ll bf = f->distance(p), bs = s->distance(p);
    if(bf > bs) swap(bf, bs), swap(f, s);
    auto best = search(p,f);
    if(bs < best.fi) best = min(best, search(p, s));
    return best;
  }
  pair<ll, pt> nearest(pt p){ return search(p, root); }
```

```
};
```

3.5 Line

```
//if p is on line s to e
bool onLine(PT s, PT e, PT p){ return p.cross(s, e) == 0;}
```

Returns the **signed dist** from p and the **line** of a and b. Positive value on left side and negative on right as seen from a->b. (a!=b)


```
ld sLineDist(PT& a, PT& b, PT& p){ return a.cross(b, p) / (b
    -a).len(); }
```

Intersection between two lines
Unique -> {+1, pt}
No inter -> { 0, pt}
Infinity -> {-1, pt}May be rounded if inter isn't integer; Watch out for overflow if long long.

```
pair<int, PT> lineInter(PT a, PT b, PT e, PT f){
  auto d = (b-a) % (f-e);
  if(d == 0) return {-(a.cross(b, e) == 0), PT()}; //
      parallel
  auto p = e.cross(b, f), q = e.cross(f, a);
  return {1, (a * p + b * q) / d};
}
```

Projects point p onto line ab. Set refl=true to get reflection of point p across line ab instead.

```
PT lineProj(PT a, PT b, PT p, bool refl=false) {
  PT v = b-a;
  return p - rotateCCW90(v) * (1+refl) * (v%(p-a)) / (v*v);
}
```

Calculate coefficients of the line equation **Ax + By = C.**

```
tuple<ll, ll, ll> ptToLine(PT s, PT t){
    ll a = t.y - s.y;
    ll b = s.x - t.x;
    ll c = a*s.x + b*s.y;
    return {a, b, c};
}
```

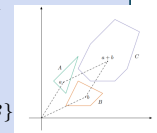
3.6 LineContainer

```
struct Line {
  mutable ll k, m, p;
  bool operator<(const Line& o) const { return k < o.k; }
  bool operator<(ll x) const { return p < x; }
};

struct LineContainer : multiset<Line, less<>> {
  static const ll inf = LLONG_MAX; // Double: inf = 1/.0,
      div(a,b) = a/b
  ll div(ll a, ll b) { return a / b - ((a ^ b) < 0 && a % b)
      ; } //floored division

  bool isect(iterator x, iterator y) {
    if(y == end()) return x->p = inf, 0;
    if(x->k == y->k) x->p = x->m > y->m ? inf : -inf;
    else x->p = div(y->m - x->m, x->k - y->k);
    return x->p >= y->p;
  }

  void add_line(ll k, ll m){ // kx + m  //if minimum k*=-1,
      m*=-1, query*-1
    auto z = insert({k, m, 0}), y = z++, x = y;
    while(isect(y, z)) z = erase(z);
    if(x != begin() && isect(--x, y)) isect(x, y = erase(y))
        ;
    while((y = x) != begin() && (--x)->p >= y->p) isect(x,
        erase(y));
  }

  ll query(ll x) {
    assert(!empty());
    auto l = *lower_bound(x);
    return l.k * x + l.m;
  }
};
```

3.7 Minkowski

Minkowski Sum of convex polygons - O(N)

Returns a convex hull of two polygons minkowski sum.
The minkowski sum of polygons A and B is a polygon such that every vectorinside it is the sum of a vector in A
and a vector in B. $A+B = C = \{a+b \mid a \in A,\ b \in B\}$
min(a.size(), b.size()) >= 2



```
// rotate the polygon such that the (bottom, left)-most
    point is at the first position
void reorder_polygon(vector<PT> &p){
  int j = 0;
  for(int i=1; i<p.size(); i++)
    if(pair(p[i].y, p[i].x) < pair(p[j].y, p[j].x))
      j = i;
  rotate(p.begin(), p.begin() + j, p.end());
}

vector<PT> minkowski(vector<PT> a, vector<PT> b){
  int n = a.size(), m = b.size(), i=0, j=0;
  reorder_polygon(a); reorder_polygon(b);
  a.push_back(a[0]); a.push_back(a[1]);
  b.push_back(b[0]); b.push_back(b[1]);

  vector<PT> c;
  while(i < n || j < m){
    c.push_back(a[i] + b[j]);
    auto p = (a[i+1] - a[i]) % (b[j+1] - b[j]);
    if(p >= 0) i++;
    if(p <= 0) j++;
  }
  return c;
}
```

3.8 Point

Dot product p*q $= p \cdot q$ | inner product | norm | lenght^2
$u \cdot v = x_1x_2 + y_1y_2 = \|u\|\,\|v\| \cos\theta$
$u \cdot v > 0 \Rightarrow$ angle $\theta < 90^\circ$ (acute);
$u \cdot v = 0 \Rightarrow$ angle $\theta = 90^\circ$ (perpendicular);
$u \cdot v < 0 \Rightarrow$ angle $\theta > 90^\circ$ (obtuse);

Cross product p % q $= p \times q$: | Vector product | Determinant
$u \times v = x_1y_2 - y_1x_2 = \|u\|\,\|v\| \sin\theta$.
$u \times v > 0 \Rightarrow v$ is to the *left* of u
$u \times v = 0 \Rightarrow u$ and v are collinear.
$u \times v < 0 \Rightarrow v$ is to the *right* of u
It equals the signed area of the parallelogram spanned by u and v.

+ p.cross(a, b) $= (a-p) \times (b-p)$
- > 0: CCW (left); ↶
- $= 0$: collinear; ⇒
- < 0: CW (right); ↷

```
#define ld double

struct PT {
  ll x, y;
  PT(ll x, ll y) : x(x), y(y) {}
  PT() : x(0), y(0){}

  PT operator+(const PT&a)const{return PT(x+a.x, y+a.y);}
  PT operator-(const PT&a)const{return PT(x-a.x, y-a.y);}
  ll operator*(const PT&a)const{return  (x*a.x + y*a.y);} //
      DOT
  ll operator%(const PT&a)const{return  (x*a.y - y*a.x);} //
      Cross
  PT operator*(ll c) const{ return PT(x*c, y*c); }
  PT operator/(ll c) const{ return PT(x/c, y/c); }
  bool operator==(const PT&a) const{ return x == a.x && y ==
      a.y; }
  bool operator< (const PT&a) const{ return tie(x, y) < tie(
      a.x, a.y); }

  ld len() const { return hypot(x,y); } // sqrt(p*p)
  ll cross(const PT&a, const PT&b) const{ return (a-*this) %
      (b-*this); } // (a-p) % (b-p)
  int quad() { return (x<0)^3*(y<0); } //cartesian plane
      quadrant |0++|1-+|2--|3+-|
  bool ccw(PT q, PT r){ return (q-*this) % (r-q) > 0;}
};

ld dist(PT p, PT q){ return sqrtl((p-q)*(p-q)); }
ld proj(PT p, PT q){ return p*q / q.len(); }
//Projection size from A to B

const ld PI = acos(-1.0L);
ld angle(PT p, PT q){ return atan2(p%q, p*q); } // Angle
    between vectors p and q [-pi, pi] | acos(a*b/a.len()/b.
    len())
ld polarAngle(PT p){  return atan2(p.y, p.x); } // Angle to
    x-axis [-pi, pi]
bool cmp_ang(PT p, PT q){ return p.quad() != q.quad() ? p.
    quad()<q.quad() : q.ccw(PT(0,0), p); }

PT rotateCCW90(PT p){ return PT(-p.y, p.x); } // perp
PT rotateCW90(PT p){ return PT(p.y, -p.x); }
PT rotateCCW(PT p, ld t){
  ld c = cos(t), s = sin(t);
  return PT(p.x*c - p.y*s, p.x*s + p.y*c);
}
```

3.9 Polygons

Returns **twice area of a simple polygon.** area*2 (Shoelace Formula: signed cross product sum)

```cpp
ll Area2x(vector<PT>& p){
  ll area = 0;
  for(int i=2; i < p.size(); i++)
    area += (p[i]-p[0]) % (p[i-1]-p[0]);
  return abs(area);
}
```

Returns if a point is **inside a triangle** (or in the border).

```cpp
bool ptInsideTriangle(PT p, PT a, PT b, PT c){
  if((b-a) % (c-b) < 0) swap(a, b);
  if(onSegment(a,b,p)) return 1;
  if(onSegment(b,c,p)) return 1;
  if(onSegment(c,a,p)) return 1;
  bool x = (b-a) % (p-b) < 0;
  bool y = (c-b) % (p-c) < 0;
  bool z = (a-c) % (p-a) < 0;
  return x == y && y == z;
}
```

Returns 1 if a **point is inside a polygon**, 2 if in border, else 0. Non-convex O(n)

```cpp
int ptInsidePol(vector<PT>& pol, PT p){
  int qt = 0;
  for(int i=0, n=pol.size(); i<n; i++){
    auto s = pol[i], e=pol[(i+1)%n];

    if(onSegment(s, e, p)) return 2;
    if((s.y < p.y) == (e.y < p.y)) continue;

    qt ^= (s.y < p.y) == (s.cross(p, e) > 0);
  }
  return qt != 0;
}
```

Returns the **center of mass for a polygon.** O(n)

```cpp
PT polygonCenter(const vector<PT>& v){
  PT res(0, 0); double A = 0;
  for(int i=0, j=v.size()-1; i<v.size(); j=i++){
    res = res + (v[i]+v[j]) * (v[j]%v[i]);
    A += v[j] % v[i];
  }
  return res / A / 3;
}
```

PolygonCut: Returns the vertices of the polygon cut away at the left of the line s->e.polygonCut(p, PT(0,0), PT(1,0));


```cpp
vector<PT> polygonCut(const vector<PT>& poly, PT s, PT e){
  vector<PT> res;
  for(int i=0; i<poly.size(); i++){
    PT cur = poly[i], prev = i ? poly[i-1] : poly.back();
    auto a = s.cross(e, cur), b = s.cross(e, prev);
    if((a < 0) != (b < 0)) res.push_back(cur + (prev - cur)
        * (a / (a - b)));
    if(a < 0) res.push_back(cur);
  }
  return res;
}
```

getArea Returns the area of the polygon formed by the points in range [L, R]. (Or [0, L] U [R, N) if R<L)Struct is only for lib purpose, copy the code inline.

```cpp
struct PolygonAreaPS{
  int n; vector<ll> ps = {0}; vector<PT> pts;

  PolygonAreaPS(vector<PT> pts) : pts(pts), n(pts.size()){
    for(int i=1; i<n; i++)
      ps.push_back(ps.back() + pts[i-1] % pts[i]);
  }

  ll getArea(int l, int r){
    if(l <= r) return ps[r] - ps[l] + pts[r] % pts[l];
    return getArea(0, n-1) - getArea(r, l);
  }
};
```

Pick's theorem for **lattice points** in a simple polygon. (lattice points = integer points)Area = insidePts + boundPts/2 - 12A - b + 2 = 2i

```cpp
ll cntInsidePts(ll area_db, ll bound){ return (area_db + 2LL
    - bound)/2; }
ll latticePointsInSeg(PT a, PT b){
  ll dx = abs(a.x - b.x);
  ll dy = abs(a.y - b.y);
  return gcd(dx, dy) + 1;
}
```

3.10 Segment

```cpp
//if p is on segment s to e
bool onSegment(PT s, PT e, PT p){
  return p.cross(s, e) == 0 && (s-p) * (e-p) <= 0;
}
```

Returns the shortest **distance** between point p and the **segment** s->e.


```cpp
ld segmentDist(PT& s, PT& e, PT& p){
  if (s==e) return (p-s).len();
  ld d = (e-s)*(e-s);
  ld t = min(d, max<ld>(0, (p-s)*(e-s)));
  return ((p-s)*d - (e-s)*t).len() / d;
}
```

Segment intersection
Unique -> {p}
No inter -> { }
Infinity -> {a, b}, the endpoints of the common segment.
May be rounded if inter isn't integer; Watch out for overflow if long long.

```cpp
int sgn(ll x){ return (x>0) - (x<0); }
vector<PT> segInter(PT a, PT b, PT c, PT d){
  auto oa = c.cross(d, a), ob = c.cross(d, b);
  auto oc = a.cross(b, c), od = a.cross(b, d);
  if(sgn(oa)*sgn(ob) < 0 && sgn(oc)*sgn(od) < 0)
    return {(a*ob - b*oa) / (ob-oa)};
  set<PT> s;
  if(onSegment(c, d, a)) s.insert(a);
  if(onSegment(c, d, b)) s.insert(b);
  if(onSegment(a, b, c)) s.insert(c);
  if(onSegment(a, b, d)) s.insert(d);
  return {begin(s), end(s)};
}
```

3.11 quadtree

```cpp
const int MXN = 1e5+100;

struct Point {
  int x, y;
  bool const operator<(const Point& o) const {
    return make_tuple(x, y) < make_tuple(o.x, o.y);
  }
};

struct AABB {
  int x1, y1, x2, y2;

  AABB() : x1(0), y1(0), x2(MXN), y2(MXN) {}
  AABB(int x1, int y1, int x2, int y2) : x1(x1), y1(y1), x2(
      x2), y2(y2) {}

  bool intersects(const AABB& other) const {
    bool overlap_x = (x1 <= other.x2) && (x2 >= other.x1);
    bool overlap_y = (y1 <= other.y2) && (y2 >= other.y1);
    return overlap_x && overlap_y;
  }

  bool contains(int x, int y) const {
    return (x >= x1 && x <= x2 && y >= y1 && y <= y2);
  }

  array<AABB, 4> subdivide() const {
    int x_center = x1 + (x2 - x1) / 2;
    int y_center = y1 + (y2 - y1) / 2;
    return {
      AABB{x1, y1, x_center, y_center},      // NW
      AABB{x_center, y1, x2, y_center},      // NE
      AABB{x1, y_center, x_center, y2},      // SW
      AABB{x_center, y_center, x2, y2}       // SE
    };
  }
};


struct Circle {
  Point c;
  int r, w;
  AABB box;

  Circle(Point c, int r, int w)
    : c(c), r(r), w(w), box(c.x - r, c.y - r, c.x + r, c.y +
        r) {}

  bool const operator<(const Circle& o) const {
    return make_tuple(c.x, c.y, r) < make_tuple(o.c.x, o.c.y
        , o.r);
  }
};

struct Square {
  Point p1, p2;
  int w;
  AABB box;

  Square(Point p1, Point p2)
    : p1(p1), p2(p2), box(min(p1.x, p2.x), min(p1.y, p2.y),
                max(p1.x, p2.x), max(p1.y, p2.y)) {}

  bool const operator<(const Square& o) const {
    return make_tuple(p1, p2) < make_tuple(o.p1, o.p2);
  }
};

struct Triangle {
  Point p1, p2, p3;
  int w;
  AABB box;

  Triangle(Point p1, Point p2, Point p3)
```

```
    : p1(p1), p2(p2), p3(p3),
      box(min({p1.x, p2.x, p3.x}), min({p1.y, p2.y, p3.y}),
          max({p1.x, p2.x, p3.x}), max({p1.y, p2.y, p3.y}))
          {}

  bool const operator<(const Triangle& o) const {
    return make_tuple(p1, p2, p3) < make_tuple(o.p1, o.p2, o
        .p3);
  }
};

template <typename T>
class Quadtree {
  public:
  static const int MIN_OBJECTS = 8;
  static const int MAX_DEPTH = 16;

  Quadtree(const AABB& root_bounds, const vector<T>&
      all_objects)
    : bounds(root_bounds), depth(0) {

    vector<const T*> object_ptrs;
    object_ptrs.reserve(all_objects.size());
    for (const auto& obj : all_objects) {
      object_ptrs.push_back(&obj);
    }

    build(object_ptrs);
  }

  const vector<const T*>& query(int x, int y) const {
    const Quadtree* node = this;

    while (node->is_branch) {
      int child_index = node->get_child_index(x, y);
      if (child_index == -1) break;
      node = node->children[child_index].get();
    }

    return node->objects;
  }

  private:
  AABB bounds;
  int depth;
  bool is_branch = false;
  int x_center, y_center;

  array<unique_ptr<Quadtree<T>>, 4> children;

  vector<const T*> objects;

  Quadtree(const AABB& box, int d) : bounds(box), depth(d)
      {}

  void build(const vector<const T*>& parent_objects) {
    x_center = bounds.x1 + (bounds.x2 - bounds.x1) / 2;
    y_center = bounds.y1 + (bounds.y2 - bounds.y1) / 2;

    for (const T* obj : parent_objects) {
      if (bounds.intersects(obj->box)) {
        objects.push_back(obj);
      }
    }

    if (objects.size() > MIN_OBJECTS && depth < MAX_DEPTH) {
      is_branch = true;
      auto child_bounds = bounds.subdivide();

      for (int i = 0; i < 4; ++i) {
        children[i] = unique_ptr<Quadtree<T>>(new Quadtree<T
            >(child_bounds[i], depth + 1));
        children[i]->build(objects);
      }

      objects.clear();
    }
  }

  int get_child_index(int x, int y) const {
    if (y <= y_center) return (x <= x_center) ? 0 : 1; // 0
        (NW) ou 1 (NE)
    else return (x <= x_center) ? 2 : 3; // 2 (SW) ou 3 (SE)
  }
};
```

4 Grafos

4.1 2SAT

2 SAT - Two Satisfiability Problem
Retorna uma valoracao verdadeira se possivel ou um vetor vazio se impossivel;
inverso de u = ~u

A	B	OR	AND	NOR	NAND	XOR	XNOR	IMPLY
0	0	0	0	1	1	0	1	1
0	1	1	0	0	1	1	0	1
1	0	1	0	0	1	1	0	0
1	1	1	1	0	0	0	1	1

```
struct TwoSat {
  int N;
  vector<vector<int>> E;

  TwoSat(int N) : N(N), E(2 * N) {}
  inline int eval(int u) const{ return u < 0 ? ((~u)+N)%(2*N
      ) : u; }

  void add_or(int u, int v){
    E[eval(~u)].push_back(eval(v));
    E[eval(~v)].push_back(eval(u));
    }
  void add_nand(int u, int v) {
    E[eval(u)].push_back(eval(~v));
    E[eval(v)].push_back(eval(~u));
  }
  void set_true (int u){ E[eval(~u)].push_back(eval(u)); }
  void set_false(int u){ set_true(~u); }
  void add_imply(int u, int v){ E[eval(u)].push_back(eval(v)
      ); }
  void add_and  (int u, int v){ set_true(u);  set_true(v);
      }
  void add_nor  (int u, int v){ add_and(~u, ~v); }
  void add_xor  (int u, int v){ add_or(u, v); add_nand(u, v)
      ; }
  void add_xnor (int u, int v){ add_xor(u, ~v); }

  vector<bool> solve() {
    vector<bool> ans(N);
    auto scc = tarjan();

    for (int u = 0; u < N; u++)
      if(scc[u] == scc[u+N]) return {}; //false
      else ans[u] = scc[u+N] > scc[u];

    return ans; //true
  }
private:
  vector<int> tarjan() {
    vector<int> low(2*N), pre(2*N, -1), scc(2*N, -1), st;
    int clk = 0, ncomps = 0;

    function<void(int)> dfs = [&](int u){
      pre[u] = low[u] = clk++;
      st.push_back(u);

      for(auto v : E[u])
        if(pre[v] == -1) dfs(v), low[u] = min(low[u], low[v
            ]);
        else
        if(scc[v] == -1) low[u] = min(low[u], pre[v]);

      if(low[u] == pre[u]){
        int v = -1;
        while(v != u) scc[v = st.back()] = ncomps, st.
            pop_back();
        ncomps++;
      }
    };

    for(int u=0; u < 2*N; u++)
      if(pre[u] == -1)
        dfs(u);

    return scc; //tarjan SCCs order is the reverse of
        topoSort, so (u->v if scc[v] <= scc[u])
  }
};
```

4.2 Bellman-Ford

```
const int MAXN = 5e2;
const int INF = 1e18;


struct Edge {
    ll a, b, cost;
};

int n, m, v;
vector<Edge> edges;


void solve()
{
    vector<int> d(n, INF);
    d[v] = 0;
    vector<int> p(n, -1);

    for (;;) {
        bool any = false;
        for (Edge e : edges)
            if (d[e.a] < INF)
                if (d[e.b] > d[e.a] + e.cost) {
                    d[e.b] = d[e.a] + e.cost;
                    p[e.b] = e.a;
                    any = true;
                }
        if (!any)
            break;
```

```
    }

    int t = 0; // curr node
    if (d[t] == INF)
        cout << "No path from " << v << " to " << t << ".";
    else {
        vector<int> path;
        for (int cur = t; cur != -1; cur = p[cur])
            path.push_back(cur);
        reverse(path.begin(), path.end());

        cout << "Path from " << v << " to " << t << ": ";
        for (int u : path)
            cout << u << ' ';
    }
}
```

4.3 BlockCutTree

Block Cut Tree - BiConnected Component

```
BlockCutTree bcc(n);
bcc.addEdge(u, v);
bcc.build();

bcc.tree    -> graph of BlockCutTree (tree.size() <= 2n)
bcc.id[u]   -> componet of u in the tree
bcc.cut[u]  -> 1 if u is a cut vertex; 0 otherwise
bcc.comp[i] -> vertex of comp i (cut are part of
     multiple comp)
```

```
struct BlockCutTree {
  vector<vector<int>> g, tree, comp;
  vector<int> id, cut;
  BlockCutTree(int n) : n(n), g(n), cut(n) {}

  void addEdge(int u, int v){
    g[u].emplace_back(v);
    g[v].emplace_back(u);
  }

  void build(){
    pre = low = id = vector<int>(n, -1);
    for(int u=0; u<n; u++, chd=0) if(pre[u] == -1) //if
         graph is disconected
      tarjan(u, -1), makeComp(-1);                //find cut
          vertex and make components

    for(int u=0; u<n; u++) if(cut[u]) comp.emplace_back(1, u
        ); //create cut components
    for(int i=0; i<comp.size(); i++)
                                //mark id of each node
      for(auto u : comp[i]) id[u] = i;

    tree.resize(comp.size());
    for(int i=0; i<comp.size(); i++)
      for(auto u : comp[i]) if(id[u] != i)
        tree[i].push_back(id[u]),
        tree[id[u]].push_back(i);
  }
private:
  vector<int> pre, low;
  vector<pair<int, int>> st;
  int n, clk = 0, chd=0, ct, a, b;

  void makeComp(int u){
    comp.emplace_back();
    do {
      tie(a, b) = st.back();
      st.pop_back();
      comp.back().push_back(b);
    } while(a != u);
    if(~u) comp.back().push_back(u);
  }

  void tarjan(int u, int p){
    pre[u] = low[u] = clk++;
    st.emplace_back(p, u);

    for(auto v : g[u]) if(v != p){
      if(pre[v] == -1){
        tarjan(v, u);
        low[u]  =  min(low[u], low[v]);
        cut[u] |= ct = (~p && low[v] >= pre[u]) || (p==-1 &&
             ++chd >= 2);
        if(ct) makeComp(u);
      }
      else low[u] = min(low[u], pre[v]);
    }
  }
};
```

4.4 CentroidDecomposition

Centroid Decomposition - O(N*LogN)

```
dfsc()     -> para criar a centroid tree
rem[u]     -> True se U ja foi removido (pra dfsc)
szt[u]     -> Size da subarvore de U (pra dfsc)
parent[u] -> Pai de U na centroid tree *parent[ROOT] =
     -1
dist[u][i]-> Distancia na arvore original de u p i-esimo
      pai na centroid tree *distToAncestor[u][0] = 0

dfsc(u=node, p=parent(subtree), f=parent(centroid tree),
      sz=size of tree)
```

```
const int MAXN = 1e6 + 5;

vector<int> g[MAXN];
deque<int> dist[MAXN];

bool rem[MAXN];
int szt[MAXN], parent[MAXN];

void getDist(int u, int p, int d=0){
  for(auto v : g[u]) if(v != p && !rem[v])
    getDist(v, u, d+1);
  dist[u].emplace_front(d);
}

int getSz(int u, int p){
  szt[u] = 1;
  for(auto v : g[u]) if(v != p && !rem[v])
    szt[u] += getSz(v, u);
  return szt[u];
}

void dfsc(int u=0, int p=-1, int f=-1, int sz=-1){
  if(sz < 0) sz = getSz(u, -1); //starting new tree

  for(auto v : g[u])
    if(v != p && !rem[v] && szt[v]*2 >= sz)
      return dfsc(v, u, f, sz);

  rem[u] = true, parent[u] = f;
  getDist(u, -1, 0); //get subtree dists to centroid

  for(auto v : g[u]) if(!rem[v])
    dfsc(v, u, u, -1);
}
```

4.5 CentroidMinPath

```
const int MAXN = 2e5+5;
vector<int> adj[MAXN];
bool rem[MAXN];
int subtree_size[MAXN], parents[MAXN];
int K; // caminho de tamanho K
ll ans = 0; // contagem de caminhos com tamanho K

int cnt[MAXN];

int get_sub_size(int v, int p = -1) {
    subtree_size[v] = 1;
    for(int u : adj[v]) {
        if(u == p || rem[u]) {
            continue;
        }
        subtree_size[v] += get_sub_size(u, v);
    }
    return subtree_size[v];
}

int get_centroid(int v, int tree_size, int p = -1) {
    for(int u : adj[v]) {
        if(u == p || rem[u]) {
            continue;
        }
        if(subtree_size[u]*2 > tree_size) {
            return get_centroid(u, tree_size, v);
        }
    }
    return v;
}

void get_distances(int v, int p, int dist, vector<int>&
     distances) {
    if(dist > K) {
        return;
    }
    distances.push_back(dist);
    for(int u : adj[v]) {
        if(u == p || rem[u]) {
            continue;
        }
        get_distances(u, v, dist+1, distances);
    }
}

void process_centroid(int centroid) {
    vector<int> used;
    cnt[0] = 1;
    used.push_back(0);
    for(int u : adj[centroid]) {
        if(rem[u]) {
            continue;
        }
        vector<int> curr;
```

```
        get_distances(u, centroid, 1, curr);

        for(int d : curr) {
            if(K - d >= 0) {
                ans += cnt[K - d];
            }
        }
        for(int d : curr) {
            if(cnt[d] == 0) {
                used.push_back(d);
            }
            cnt[d]++;
        }
    }
    for(int d : used) {
        cnt[d] = 0;
    }
}

int decompose(int v, int p = -1) {
    int tree_size = get_sub_size(v);
    int centroid = get_centroid(v, tree_size);
    process_centroid(centroid);
    rem[centroid] = true; parents[centroid] = p;
    for(int u : adj[centroid]) {
        if(!rem[u]) {
            decompose(u);
        }
    }
    return centroid;
}
```

4.6 DSU Persistente

SemiPersistent DSU - O(Log n)
find(u, q) -> Retorna o pai de U no tempo q
* tim -> tempo em que o pai de U foi alterado

```
struct DSUp {
  vector<int> pai, sz, tim;
  int t=1;
  DSUp(int n) : pai(n+1), sz(n+1, 1), tim(n+1) {
    for(int i=0; i<=n; i++) pai[i] = i;
  }

  int find(int u, int q = INT_MAX){
    if( pai[u] == u || q < tim[u] ) return u;
    return find(pai[u], q);
  }

  void join(int u, int v){
    u = find(u), v = find(v);

    if(u == v) return;
    if(sz[v] > sz[u]) swap(u, v);

    pai[v] = u;
    tim[v] = t++;
    sz[u] += sz[v];
  }
};
```

4.7 DSU Rollback

Disjoint Set Union with Rollback - O(Log n)
checkpoint() -> salva o estado atual
rollback() -> restaura no ultimo checkpoint
save another var? +save in join & +line in pop

```
struct DSUr {
  vector<int> pai, sz, savept;
  stack<pair<int&, int>>  st;
  DSUr(int n) : pai(n+1), sz(n+1, 1) {
    for(int i=0; i<=n; i++) pai[i] = i;
  }

  int find(int u){ return pai[u] == u ? u : find(pai[u]); }

  void join(int u, int v){
    u = find(u), v = find(v);

    if(u == v) return;
    if(sz[v] > sz[u]) swap(u, v);

    save(pai[v]); pai[v] = u;
    save(sz[u]);  sz[u] += sz[v];
  }

  void save(int &x){ st.emplace(x, x); }
  void pop(){
    st.top().first = st.top().second; st.pop();
    st.top().first = st.top().second; st.pop();
  }

  void checkpoint(){ savept.push_back(st.size()); }
  void rollback(){
    while(st.size() > savept.back()) pop();
    savept.pop_back();
  }
};
```

4.8 DSU

```
struct DSU {
  vector<int> pai, sz;
  DSU(int n) : pai(n+1), sz(n+1, 1) {
    for(int i=0; i<=n; i++) pai[i] = i;
  }

  int find(int u){ return pai[u] == u ? u : pai[u] = find(
      pai[u]); }

  void join(int u, int v){
    u = find(u), v = find(v);

    if(u == v) return;
    if(sz[v] > sz[u]) swap(u, v);

    pai[v] = u;
    sz[u] += sz[v];
  }
};
```

4.9 Dijkstra

```
#define INF 0x3f3f3f3f3f3f3f3f
#define pii pair<ll,ll>

vector<pii> g[MAXN];

vector<ll> dijkstra(int s, int N){
  vector<ll> dist (N, INF);
  priority_queue<pii, vector<pii>, greater<pii>> pq;
  pq.push({0, s});
  dist[s] = 0;

  while(!pq.empty()){
    auto [d, u] = pq.top();
    pq.pop();
    if(d > dist[u]) continue;

    for(auto [v, c] : g[u])
      if( dist[v] > dist[u] + c ){
        dist[v] = dist[u] + c;
        pq.push({dist[v], v});
      }
  }
  return dist;
}
```

4.10 Dinic

Dinic - Max Flow Min Cut

Algoritmo de Dinitz para encontrar o Fluxo Maximo.
IMPORTANTE! O algoritmo esta 0-indexado

Complexity:
O(V^2 * E) -> caso geral
O(sqrt(V) * E) -> grafos com cap = 1 para toda Edge
// matching bipartido

* Informacoes:
Crie o Dinic: Dinic dinic(n, src, sink);
Adicione as edges: dinic.addEdge(u, v, capacity);
Para calcular o Fluxo Maximo: dinic.maxFlow()
Para saber se um vertice U esta no Corte Minimo: dinic.inCut(u)

* Sobre o Codigo:
vector<Edge> edges; -> Guarda todas as edges do grafo e do grafo residual
vector<vector<int>> adj; -> Guarda em adj[u] os indices de todas as edges saindo de u
vector<int> ptr; -> Pointer para a proxima Edge ainda nao visitada de cada vertice
vector<int> lvl; -> Distancia em vertices a partir do Source. Se igual a N o vertice nao foi visitado.
A BFS retorna se Sink e alcancavel de Source. Se nao e porque foi atingido o Fluxo Maximo
A DFS retorna um possivel aumento do Fluxo

Use Cases of Flow

+ **Minimum cut**: the minimum cut is equal to maximum flow. i.e. to split the graph in two parts, one on the src side and another on sink side. The capacity of each edge is it weight.

+ **Edge-disjoint-paths**: maximum number of edge-disjoint paths equals maximum flow of the graph, assuming that the capacity of each edge is one. (paths can be found greedily)

+ **Node-disjoint-paths**: can be reduced to maximum flow. each node should appear in at most one path, so limit the flow through a node dividing each node in two. One with incoming edges, other with outgoing edges and a new edge from the first to the second with capacity 1.

+ **Maximum matching** (bipartite): maximum matching is equal to maximum flow. Add a src and a sink, edges from the src to every node at one partition and from each node of the other partition to the sink.

+ **Minimum node cover** (bipartite) : minimum set of nodes such each edge has at least one endpoint. The size of minimum node cover is equal to maximum matching (Konig's theorem).

+ **Maximum-independent-set** (bipartite): largest set of nodes such that no two nodes are connected with an edge. Contain the nodes that aren't in "Min node cover" (N - MAXFLOW).

+ **Minimum path cover** (DAG): set of paths such that each node belongs to at least one path.
- Node-disjoint: construc a matching where each node is represented by two nodes, a left and a right at the matching and add the edges (from l to r). Each edge in the matching corresponds to an edge in the path cover. The number of paths in the cover is (N - MAXFLOW).
- General: almost like a minimum node-disjoint. Just add edges to the matching whenever there is an path from U to V in the graph (possibly through several edges).
- Antichain: a set of nodes such that there is no path from any node to another. In a DAG, the size of min general path cover equals the size of maximum antichain (Dilworth's theorem).

+ **Project selection**: Given N projects, each w profit pi, and M machines, each w cost ci.
A project requires a set of machines (can be shared).
Choose a set that maximizes value of the profit(projects) - the cost(machines). Add an edge (cap pi) from Source to project.
An edge (cap ci) from machine to Sink. An edge (cap INF) from a project to each machine it requires.
ans = SUM(pi) - MAXFLOW. If the edge of a machine is saturated, buy it.

+ **Closure**
Problem (directed graph): Each node has a weight w (+ or -). choose a closure with maximum sum.
A closure is a set of nodes such that there is no edge from a node inside the set to a node outside.
Is a general case of project selection. Original edges with cap INF. Add edges from Source to nodes with W > 0; and from nodes with W < 0 to Sink (cap |W|).

```cpp
struct Edge {
  int u, v; ll cap;
  Edge(int u, int v, ll cap) : u(u), v(v), cap(cap) {}
};

struct Dinic {
  int n, src, sink;
  vector<vector<int>> adj;
  vector<Edge> edges;
  vector<int> lvl, ptr; //pointer para a proxima Edge nao saturada de cada vertice

  Dinic(int n, int src, int sink) : n(n), src(src), sink(sink) { adj.resize(n); }

  void addEdge(int u, int v, ll cap){
    adj[u].push_back(edges.size());
    edges.emplace_back(u, v, cap);

    adj[v].push_back(edges.size());
    edges.emplace_back(v, u, 0);
  }

  ll dfs(int u, ll flow = 1e9){
    if(flow == 0) return 0;
    if(u == sink) return flow;

    for(int &i = ptr[u];  i < adj[u].size();  i++){
      int at = adj[u][i];
      int v = edges[at].v;

      if(lvl[u] + 1 != lvl[v]) continue;

      if(ll got = dfs(v, min(flow, edges[at].cap)) ){
        edges[at].cap -= got;
        edges[at^1].cap += got;
        return got;
      }
    }

    return 0;
  }

  bool bfs(){
    lvl = vector<int> (n, n);
    lvl[src] = 0;

    queue<int> q;
    q.push(src);

    while(!q.empty()){
      int u = q.front();
      q.pop();

      for(auto i : adj[u]){
        int v = edges[i].v;

        if(edges[i].cap == 0 || lvl[v] <= lvl[u] + 1 )
          continue;

        lvl[v] = lvl[u] + 1;
        q.push(v);
      }
    }
    return lvl[sink] < n;
  }

  bool inCut(int u){ return lvl[u] < n; }

  ll maxFlow(){
    ll ans = 0;
    while( bfs() ){
      ptr = vector<int> (n+1, 0);
      while(ll got = dfs(src)) ans += got;
    }
    return ans;
  }

  vector<pair<int,int>> MinCut() {
```

```
        vector<pair<int,int>> cut;

    for (int i = 0; i < edges.size(); i += 2) {
            int&u = edges[i].u;
            int&v = edges[i].v;

            if (inCut(u) && !inCut(v)) {
        cut.emplace_back(u, v);
            }
        }

    return cut;

    }

  // all flow paths
  void printPath(int maxflow) {
        ptr = vector<int>(n+1, 0);

        for (int i = 0; i < maxflow; i++) {
            vector<int> currPath;
            int u = src;
            currPath.push_back(u);

            while (u != sink) {
                for (int& j = ptr[u]; j < adj[u].size(); j
                     ++) {
                    int at = adj[u][j];

                    if (at % 2 == 0 && edges[at].cap == 0){
                        u = edges[at].v;
                        currPath.push_back(u);
                        j++;
                        break;
                    }
                }
            }

            cout << currPath.size() << br;
            for(auto&i:currPath) cout << i << ' ';
            cout << br;
        }
    }

  // for bipartite match
  void printPairs(int lft) {
    for (int u = 1; u <= lft; u++) {
      for (int edge_id : adj[u]) {
        if (edge_id % 2 == 0 && edges[edge_id].cap == 0) {
          int v = edges[edge_id].v;
          cout << u << " " << v - lft << br;
        }
      }
    }
  }

};
```

4.11 DominatorTree

Dominator Tree & **Edge Dominator**

Seja S um no de um grafo direcionado, um vertice U domina V sse todo caminho de S para V passa por U.
A definicao e semelhante para aresta.
S sempre sera a raiz da dominator Tree e a relacao e transitiva.

- Relacionado: **Strong Bridges** e **Strong Cut Points**
Vertice ou Aresta que, se removido, aumenta a quantidade de SCCs.

Seja G um SCC e GR seu grafo reverso, podemos calcular as Strong Bridges e Strong Cut Points usando Dominators e EdgeDominators.
Seja S a raiz escolhida, calcule os dominators partindo de S em G. Depois troque G e Gr e recalcule.
build(n, 0); getDom(); for(int u=0; u<n; u++) swap(g[u], gr[u]); build(n, 0); getDom();
Os strong dominators serao a uniao dos dois conjuntos.
A unica excessao e S, que sempre sera um dominator por ser a raiz, ele deve ser considerado a parte.
Se G nao for um SCC, use tarjan pra encontrar os SCCs e isole cada chamada do Build e da DFS a um SCC.

```
const int maxn = 1e5;
int anc[maxn], par[maxn], pmin[maxn];
int sdom[maxn], idom[maxn];
int tin[maxn], tout[maxn];
vector<int> g[maxn];
vector<int> gr[maxn];
vector<int> tree[maxn];
vector<int> bucket[maxn];
vector<int> order;

int tt=0;
void dfs(int u){
  tin[u] = tt++;
  sdom[u] = pmin[u] = u;
  order.push_back(u);

  for(auto v : g[u])
    if(tin[v] == -1)
      dfs(v), par[v] = u;
}

int sfind(int v){
  if(anc[v] == -1) return v;
  if(anc[anc[v]] != -1){
    int u = sfind(anc[v]);
    if(tin[sdom[u]] < tin[sdom[pmin[v]]]) pmin[v] = u;
    anc[v] = anc[u];
  }
  return pmin[v];
}

void treefs(int u){
  tin[u] = ++tt;
  for(auto v : tree[u])
    treefs(v);
  tout[u] = tt;
}

bool dominates(int u, int v){
  return tin[u] <= tin[v] && tin[v] <= tout[u];
}

void build(int n, int s){
  memset(anc, -1, sizeof anc);
  memset(tin, -1, sizeof tin);

  dfs(s);

  reverse(begin(order), end(order)); order.pop_back();

  auto smin = [&](int u, int v){
    return tin[sdom[u]] < tin[sdom[v]] ? sdom[u] : sdom[v];
  };

  for(auto w : order){
    for(auto v : gr[w]) sdom[w] = smin(w, sfind(v));
    anc[w] = par[w];
    bucket[sdom[w]].push_back(w);

    for(auto v : bucket[par[w]]){
      int u = sfind(v);
      idom[v] = (u == v) ? sdom[v] : u;
    }
    bucket[anc[w]].clear();
  }

  reverse(begin(order), end(order));

  for(auto u : order)
    if(idom[u] != sdom[u])
      idom[u] = idom[idom[u]];

  idom[s] = -1;
  for(auto i : order) tree[idom[i]].push_back(i);
  treefs(s);
  // edgeDominator(order);
}

auto edgeDominator(vector<int>&nodes){
  vector<pair<int, int>> edges;
  for(auto w : nodes){
    int cnt = 0, okok = true;
    for(auto x : gr[w]) if(x == par[w]) cnt++; else okok &=
        dominates(w, x);
    if(okok && cnt == 1) edges.push_back({par[w], w});
  }
  return edges;
}
```

4.12 Euler Path

Euler Path - Algoritmo de Hierholzer para caminho Euleriano - O(V + E)
IMPORTANTE! O algoritmo esta 0-indexado

* Informacoes
  addEdge(u, v) -> Adiciona uma aresta de U para V
  EulerPath(n) -> Retorna o Euler Path, ou um vetor vazio se impossivel
  vi path -> vertices do Euler Path na ordem
  vi pathId -> id das Arestas do Euler Path na ordem

Euler em Undirected graph:
  - Cada vertice tem um numero par de arestas (circuito) ; OU
  - Exatamente dois vertices tem um numero impar de arestas (caminho);
Euler em Directed graph:
  - Cada vertice tem quantidade de arestas |entrada| == |saida| (circuito); OU
  - Exatamente 1 tem |entrada|+1 == |saida| && exatamente 1 tem |entrada| == |saida|+1 (caminho) ;
* Circuito -> U e o primeiro e ultimo
* Caminho -> U e o primeiro e V o ultimo

```cpp
#define vi vector<int>

const bool BIDIRECIONAL = true;
vector<pii> g [MAXN];
vector<bool> used;

void addEdge(int u, int v){
  g[u].emplace_back(v, used.size()); if(BIDIRECIONAL && u != v)
  g[v].emplace_back(u, used.size());
  used.emplace_back(false);
}

pair<vi, vi> EulerPath(int n, int src=0){
  int s=-1, t=-1;
  vector<int> selfLoop(n*BIDIRECIONAL, 0);

  if(BIDIRECIONAL)
  {
    for(int u=0; u<n; u++) for(auto&[v, id] : g[u]) if(u==v) selfLoop[u]++;
    for(int u=0; u<n; u++)
      if((g[u].size() - selfLoop[u])%2)
        if(t != -1) return {vi(), vi()};   // mais que 2 com grau impar
        else t = s, s = u;

    if(t == -1 && t != s) return {vi(), vi()}; // so 1 com grau impar
    if(s == -1 || t == src) s = src;           // se possivel, seta start como src
  }
  else
  {
    vector<int> in(n, 0), out(n, 0);

    for(int u=0; u<n; u++)
      for(auto [v, edg] : g[u])
        in[v]++, out[u]++;

    for(int u=0; u<n; u++)
      if(in[u] - out[u] == -1 && s == -1) s = u;  else
      if(in[u] - out[u] ==  1 && t == -1) t = u;  else
      if(in[u] !=out[u]) return {vi(), vi()};

    if(s == -1 && t == -1) s = t = src;          // se possivel, seta s como src
    if(s == -1 && t != -1) return {vi(), vi()}; // Existe S mas nao T
    if(s != -1 && t == -1) return {vi(), vi()}; // Existe T mas nao S
  }

  for(int i=0; g[s].empty() && i<n; i++) s =(s+1)%n; //evita ser vertice isolado

  /////// DFS ///////
  vector<int> path, pathId, idx(n, 0);
  stack<pii> st; // {Vertex, EdgeId}
  st.push({s, -1});

  while(!st.empty()){
    auto [u, edg] = st.top();
    while(idx[u] < g[u].size() && used[g[u][idx[u]].second]) idx[u]++;

    if(idx[u] < g[u].size()){
      auto [v, id] = g[u][idx[u]];
      used[id] = true;
      st.push({v, id});
      continue;
    }

    path.push_back(u);
    pathId.push_back(edg);
    st.pop();
  }

  pathId.pop_back();
  reverse(begin(path),   end(path));
  reverse(begin(pathId), end(pathId));

  /// Grafo conexo ? ///
  int edgesTotal = 0;
  for(int u=0; u<n; u++) edgesTotal += g[u].size() + (BIDIRECIONAL ? selfLoop[u] : 0);
  if(BIDIRECIONAL) edgesTotal /= 2;
  if(pathId.size() != edgesTotal) return {vi(), vi()};

  return {path, pathId};
}
```

4.13 Euler Tour

```cpp
ll vals[N];
int tin[N], tout[N];
vector<vector<int>> adj(N);
int timer = 0;

void euler(int node, int prev) {
    tin[node] = timer++;
    for(int v : adj[node]) {
        if(v != prev) {
            euler(v, node);
        }
    }
    tout[node] = timer;
}

bool ancestor(int u, int v) {
    return tin[u] <= tin[v] && tout[u] >= tout[v];
}

// Pre-processamento de LCA usando Seg Tree
// queries em O(log N) com pre-processamento em O(N)

const int N = 100000;

int tin[N], euler_list[2*N];
vector<vector<int>> adj(N);
int timer = 0;

int seg_lca[8*N];

void euler(int node, int prev) {
    tin[node] = timer;
    euler_list[timer++] = node;
    for(int v : adj[node]) {
        if(v != prev) {
            euler(v, node);
            euler_list[timer++] = node;
        }
    }
}

int join(int x, int y) {
    if(x == -1) {
        return y;
    }
    if(y == -1) {
        return x;
    }
    return (tin[x] < tin[y] ? x : y);
}

void build(int l = 0, int r = timer-1, int idx = 0) {
    if(l == r) {
        seg_lca[idx] = euler_list[l];
        return;
    }
    int mid = (l+r)/2;
    build(l, mid, 2*idx+1);
    build(mid+1, r, 2*idx+2);
    seg_lca[idx] = join(seg_lca[2*idx+1], seg_lca[2*idx+2]);
}

int query(int L, int R, int l = 0, int r = timer-1, int idx = 0) {
    if(R < l || L > r) {
        return -1;
    }
    if(L <= l && r <= R) {
        return seg_lca[idx];
    }
    int mid = (l+r)/2;
    return join(query(L, R, l, mid, 2*idx+1), query(L, R, mid+1, r, 2*idx+2));
}

int lca(int u, int v) {
    if(tin[u] > tin[v]) {
        swap(u, v);
    }
    return query(tin[u], tin[v]);
}
```

4.14 Floyd-Warshall

```cpp
const int MAXN = 5e2;
const int INF = 1e18;

int n;
ll g[MAXN][MAXN];

void floyd()
{
  for (int k = 0; k < n; ++k)
    for (int i = 0; i < n; ++i)
      for (int j = 0; j < n; ++j)
          if (g[i][k] < INF && g[k][j] < INF)
              g[i][j] = min(g[i][j], g[i][k] + g[k][j]);
}
```

4.15 HLD

Heavy-Light Decomposition

Complexity: O(LogN * (qry || updt))

Change qry(l, r) and updt(l, r) to call a query and update structure of your will

HLD hld(n); //call init
hld.add_edges(u, v); //add all edges
hld.build(); //Build everthing for HLD

tin[u] -> Pos in the structure (Seg, Bit, ...)
nxt[u] -> Head/Endpoint
IMPORTANTE! o grafo deve estar 0-indexado!

```cpp
const bool EDGE = false;
struct HLD {
public:
  vector<vector<int>> g;
  vector<int> sz, parent, tin, nxt;
  SegTree<int> seg;
  HLD() : seg(1) {}
  HLD(int n) : seg(n) { init(n); }
  void init(int n){
    t = 0; seg = SegTree<int>(n);
    g.resize(n); tin.resize(n);
    sz.resize(n);nxt.resize(n);
    parent.resize(n);
  }
  void addEdge(int u, int v){
    g[u].emplace_back(v);
    g[v].emplace_back(u);
  }
  void build(int root=0){
    nxt[root]=root;
    dfs(root, root);
    hld(root, root);
  }
  ll query_path(int u, int v){
    if(tin[u] < tin[v]) swap(u, v);
    if(nxt[u] == nxt[v]) return qry(tin[v]+EDGE, tin[u]);
    return qry(tin[nxt[u]], tin[u]) + query_path(parent[nxt[
        u]], v);
  }
  void update_path(int u, int v, ll x){
    if(tin[u] < tin[v]) swap(u, v);
    if(nxt[u] == nxt[v]) return updt(tin[v]+EDGE, tin[u], x)
        ;
    updt(tin[nxt[u]], tin[u], x); update_path(parent[nxt[u
        ]], v, x);
  }

private:
  ll qry(int l, int r){ if(EDGE && l>r) return seg.NEUTRO;
      return seg.query(l, r); }
  void updt(int l, int r, ll x){ if(EDGE && l>r) return; seg
      .update(l, r, x); }

  void dfs(int u, int p){
    sz[u] = 1, parent[u] = p;
    for(auto &v : g[u]) if(v != p) {
      dfs(v, u); sz[u] += sz[v];

      if(sz[v] > sz[g[u][0]] || g[u][0] == p)
        swap(v, g[u][0]);
    }
  }

  int t=0;
  void hld(int u, int p){
    tin[u] = t++;
    for(auto &v : g[u]) if(v != p)
      nxt[v] = (v == g[u][0] ? nxt[u] : v),
      hld(v, u);
  }

  /// OPTIONAL ///
  int lca(int u, int v){
    while(!inSubtree(nxt[u], v)) u = parent[nxt[u]];
    while(!inSubtree(nxt[v], u)) v = parent[nxt[v]];
    return tin[u] < tin[v] ? u : v;
  }
  bool inSubtree(int u, int v){ return tin[u] <= tin[v] &&
      tin[v] < tin[u] + sz[u]; }
  //query/update_subtree[tin[u]+EDGE, tin[u]+sz[u]-1];
  vector<pair<int, int>> pathToAncestor(int u, int a){
        vector<pair<int, int>> ans;
        while(nxt[u] != nxt[a])
            ans.emplace_back(tin[nxt[u]], tin[u]),
            u = parent[nxt[u]];
        ans.emplace_back(tin[a], tin[u]);
        return ans;
    }
};
```

4.16 Kahn's

```cpp
int n;
vector<vector<int>>& adj;



vector<int> toposort() {
    vector<int> deg(n, 0);
    for (int u = 0; u < n; ++u) {
        for (int v : adj[u]) {
            deg[v]++;
        }
    }

    queue<int> q;
    for (int i = 0; i < n; ++i) {
        if (deg[i] == 0) {
            q.push(i);
        }
    }

    vector<int> ord;
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        ord.push_back(u);

        for (int v : adj[u]) {
            deg[v]--;
            if (deg[v] == 0) {
                q.push(v);
            }
        }
    }

    return ord;
}
```

4.17 Kruskal

```cpp
/*Create a DSU*/
void join(int u, int v); int find(int u);

const int MAXN = 1e6 + 5;
struct Aresta{ int u, v, c; };
bool compAresta(Aresta a, Aresta b){ return a.c < b.c; }

vector<Aresta> arestas;        //Lista de Arestas

int kruskal(){
  sort(begin(arestas), end(arestas), compAresta); //Ordena
      pelo custo
  int resp = 0;          //Custo total da MST

  for(auto a : arestas)
    if( find(a.u) != find(a.v) )
    {
      resp += a.c;
      join(a.u, a.v);
    }
  return resp;
}
```

4.18 LCA

LCA - Lowest Common Ancestor - **Binary Lifting** - O(Log N) - Build O(N Log N)
Encontrar o menor ancestral comum entre dois vertices em uma arvore enraizada
IMPORTANTE! O algoritmo esta 0-indexado
-> chame dfs(root, root) para calcular o pai e a altura de cada vertice
-> chame buildBL() para criar a matriz do Binary Lifting
-> chame lca(u, v) para encontrar o menor ancestral comum
bl[i][u] -> Binary Lifting com o (2^i)-esimo pai de u
lvl[u] -> Altura ou level de U na arvore

```cpp
const int MAXN = 5e5 + 5;
const int MAXLG = 20;

vector<int> g[MAXN];
int bl[MAXLG][MAXN], lvl[MAXN];

void dfs(int u, int p, int l=0){
  lvl[u] = l;
  bl[0][u] = p;

  for(auto v : g[u]) if(v != p)
    dfs(v, u, l+1);
}

void buildBL(int N){
  for(int i=1; i<MAXLG; i++)
    for(int u=0; u<N; u++)
      bl[i][u] = bl[i-1][bl[i-1][u]];
}

int lca(int u, int v){
  if(lvl[u] < lvl[v]) swap(u, v);

  for(int i=MAXLG-1; i>=0; i--)
    if(lvl[u] - (1<<i) >= lvl[v])
      u = bl[i][u];
```

```
  if(u == v) return u;

  for(int i=MAXLG-1; i>=0; i--)
    if(bl[i][u] != bl[i][v])
      u = bl[i][u],
      v = bl[i][v];

  return bl[0][u];
}

// K steps
int n;
const int LOG = 31;
const int MAXN = 2e5 + 5;

int up[LOG][MAXN];
void bin_lift() {
    for(int i = 1; i < LOG; i++) {
        for(int j = 0; j < n; j++) {
            up[i][j] = up[i-1][up[i-1][j]];
        }
    }
}

int kSteps(int v, int k) {
    for(int i = 0; i < LOG; i++) {
        if(k & (1LL << i)) {
            v = up[i][v];
        }
    }
    return v;
}
```

4.19 MinCostMaxFlow

```
struct Edge {
  int u, v; ll cap, cost;
  Edge(int u, int v, ll cap, ll cost) : u(u), v(v), cap(cap)
      , cost(cost) {}
};

struct MCMF {
  const ll INF = numeric_limits<ll>::max();
  int n, src, snk;
  vector<vector<int>> adj;
  vector<Edge> edges;
  vector<ll> dist, pot;
  vector<int> from;

  MCMF(int n, int src, int snk) : n(n), src(src), snk(snk) {
       adj.resize(n); pot.resize(n); }

  void addEdge(int u, int v, ll cap, ll cost){
    adj[u].push_back(edges.size());
    edges.emplace_back(u, v, cap, cost);

    adj[v].push_back(edges.size());
    edges.emplace_back(v, u, 0, -cost);
  }

  queue<int> q;
  vector<bool> vis;
  bool SPFA(){
    dist.assign(n, INF);
    from.assign(n, -1);
    vis.assign(n, false);

    q.push(src);
    dist[src] = 0;

    while(!q.empty()){
      int u = q.front();
      q.pop();

      vis[u] = false;

      for(auto i : adj[u]){
        if(edges[i].cap == 0) continue;
        int v = edges[i].v;
        ll cost = edges[i].cost;

        if(dist[v] > dist[u] + cost + pot[u] - pot[v]){
          dist[v] = dist[u] + cost + pot[u] - pot[v];
          from[v] = i;
          if(!vis[v]) q.push(v), vis[v] = true;
        }
      }
    }

    for(int u=0; u<n; u++)  //fix pot
      if(dist[u] < INF)
        pot[u] += dist[u];

    return dist[snk] < INF;
  }

  pair<ll, ll> augment(){
    ll flow = edges[from[snk]].cap, cost = 0; //fixed flow:
         flow = min(flow, remainder)

    for(int v=snk; v != src; v = edges[from[v]].u)
      flow = min(flow, edges[from[v]].cap),
      cost += edges[from[v]].cost;

    for(int v=snk; v != src; v = edges[from[v]].u)
      edges[from[v]].cap   -= flow,
      edges[from[v]^1].cap += flow;

    return {flow, cost};
  }

  bool inCut(int u){ return dist[u] < INF; }

  pair<ll, ll> maxFlow(){
    ll flow = 0, cost = 0;

    while( SPFA() ){
      auto [f, c] = augment();
      flow += f;
      cost += f*c;
    }
    return {flow, cost};
  }
};
```

4.20 SCC - Kosaraju

```
Kosaraju - Strongly Connected Component
Algoritmo de Kosaraju para encontrar Componentes
     Fortemente Conexas

Complexity: O(V + E)
IMPORTANTE! O algoritmo esta 0-indexado

* Variaveis e explicacoes *
int C   -> C e a quantidade de Componetes Conexas. As
     componetes estao numeradas de 0 a C-1
dag     -> Apos rodar o Kosaraju, o grafo das
     componentes conexas sera criado aqui
comp[u] -> Diz a qual componente conexa U faz parte
g   -> grafo direcionado
gr  -> grafo reverso (que deve ser construido junto ao
     grafo normal) !!!

NOTA: A ordem que o Kosaraju descobre as componentes e
     uma Ordenacao Topologica do SCC
em que o dag[0] nao possui grau de entrada e o dag[C-1]
     nao possui grau de saida
```

```
const int MAXN = 1e6 + 5;

vector<int> g[MAXN],  gr[MAXN], dag[MAXN];
vector<int> comp, order;
vector<bool> vis;
int C = 0;

void dfs(int u){
  vis[u] = true;
  for(auto v : g[u]) if(!vis[v])
    dfs(v);
  order.push_back(u);
}

void dfsr(int u){
  comp[u] = C;
  for(auto v : gr[u]) if(comp[v] == -1)
    dfsr(v);
}

void kosaraju(int n){
  order.clear();
  comp.assign(n, -1);
  vis.assign(n, false);
  C = 0;

  for(int v=0; v<n; v++) if(!vis[v])
    dfs(v);

  reverse(begin(order), end(order));

  for(auto v : order) if(comp[v] == -1)
    dfsr(v), C++;

  //// Montar DAG  ////
  vector<bool> marc(C, false);

  for(int u=0; u<n; u++){
    for(auto v : g[u]){
      if(comp[v] == comp[u] || marc[comp[v]]) continue;

      marc[comp[v]] = true;
      dag[comp[u]].emplace_back(comp[v]);
    }
    for(auto v : g[u]) marc[comp[v]] = false;
  }
}
```

4.21 Tarjan

```
Tarjan - Pontes e Pontos de Articulacao - O(V + E)
Algoritmo para encontrar pontes e pontos de articulacao.

pre[u]= "Altura", ou, x-esimo elemento visitado na DFS.
     Usado para saber a posicao de um vertice na arvore
     de DFS
low[u]= Low Link de U, ou a menor aresta de retorno (
     mais proxima da raiz) que U alcanca entre seus
     filhos
chd   = Children. Quantidade de componentes filhos de U.
      Usado para saber se a Raiz e Ponto de Articulacao.
any   = Marca se alguma aresta de retorno em qualquer
     dos componentes filhos de U nao ultrapassa U. Se
     isso for verdade, U e Ponto de Articulacao.

if(low[v] >  pre[u]) pontes.emplace_back(u, v); -> se a
     mais alta aresta de retorno de V (ou o menor low)
     estiver abaixo de U, entao U-V e ponte
if(low[v] >= pre[u]) any = true;        -> se a mais
     alta aresta de retorno de V (ou o menor low)
     estiver abaixo de U ou igual a U, entao U e Ponto
     de Articulacao
```

```cpp
const int MAXN = 1e6 + 5;
int pre[MAXN], low[MAXN], clk=0;
vector<int> g[MAXN];

vector<pair<int, int>> pontes;
vector<int> cut;

void tarjan(int u, int p = -1){
  if(p == -1) memset(pre, -1, sizeof pre); //so chama na
       root
  pre[u] = low[u] = clk++;
  int any = false, chd = 0;

  for(auto v : g[u]) if(v != p){
    if(pre[v] == -1){
      tarjan(v, u);

      low[u] = min(low[v], low[u]);

      if(low[v] >  pre[u]) pontes.emplace_back(u, v);
      if(low[v] >= pre[u]) any = true;
      chd++;
    }
    else low[u] = min(low[u], pre[v]);
  }

  if(p == -1 && chd >= 2) cut.push_back(u);
  if(p != -1 && any)      cut.push_back(u);
}
```

4.22 bridges-cutpoints

```cpp
#define ff first
#define ss second



int dx[] = {0,0,1,-1};
int dy[] = {1,-1,0,0};



int n, m;
vector<vector<int>> adj;
int timer;

// for the normal graph
vector<bool> visited;
vector<int> tin, low, points;
vector<bool> is_cutpoint;


// for the grid
vector<vector<bool>> visGrid;
vector<vector<pair<int,int>>> times;
vector<pair<int,int>> pointsGrid;
vector<vector<bool>> is_cutpointGrid;

//find bridges
vector<pair<int, int>> bridges;

//////////////// - setup - /////////////////



void dfs_bridges(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++;

    for (int to : adj[v]) {
        if (to == p) continue;
        if (visited[to]) {
            low[v] = min(low[v], tin[to]);
        } else {
            dfs_bridges(to, v);
            low[v] = min(low[v], low[to]);

            if (low[to] > tin[v]) {
                bridges.push_back({min(v, to), max(v, to)});
            }
        }
    }
}




void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++;
    int children=0;
    for (int to : adj[v]) {
        if (to == p) continue;
        if (visited[to]) {
            low[v] = min(low[v], tin[to]);
        } else {
            dfs(to, v);
            low[v] = min(low[v], low[to]);
            if (low[to] >= tin[v] && p!=-1 && !is_cutpoint[v
                 ]) {
                points.push_back(v); //v is a cutpoint
                is_cutpoint[v] = true;
            }
            ++children;
        }
    }
    if(p == -1 && children > 1 ) {
        points.push_back(v); //v (the root of the dfs) is a
             cutpoint
        is_cutpoint[v] = true;
    }
}
```

```
bool check(int x, int y) { return x >= 0 && x < n && y >= 0
     && y < m && adj[x][y]; }

void dfsG(pair<int,int> v, pair<int,int> p={-1,-1}) {
    int& vf = v.ff;
    int& vs = v.ss;

    visGrid[vf][vs] = true;
    times[vf][vs] = {timer, timer}, timer++;
    int children=0;
    for (int i = 0; i < 4; i++) {
        int ax = dx[i] + vf;
        int ay = dy[i] + vs;

        if (!check(ax, ay) || (ax == p.ff && ay == p.ss))
             continue;

        if (visGrid[ax][ay]) times[vf][vs].ss = min(times[vf
             ][vs].ss, times[ax][ay].ff);
        else {
            dfsG({ax, ay}, v);
            times[vf][vs].ss = min(times[vf][vs].ss, times[
                 ax][ay].ss);

            if (!is_cutpointGrid[vf][vs] && times[ax][ay].ss
                  >= times[vf][vs].ff && (p.ff != -1)) {
                pointsGrid.push_back(v);
                is_cutpointGrid[vf][vs] = true;
            }
            ++children;
        }
    }
    if (!is_cutpointGrid[vf][vs] && p.ff == -1 && children >
          1) {
        pointsGrid.push_back(v);
        is_cutpointGrid[vf][vs] = true;
    }
}

void find_cutpoints() {
    timer = 0;
    visited.assign(n, false);
    is_cutpoint.assign(n, false);
    tin.assign(n, -1);
    low.assign(n, -1);
    points.clear();

    /*
    visGrid.assign(n, vector<bool>(m, false));
    times.assign(n, vector<pair<int,int>>(m, {-1,-1}));
    is_cutpointGrid.assign(n, vector<bool>(m, false));
    */

    for (int i = 0; i < n; ++i) {
        if (!visited[i]) {
            dfs (i);
        }
    }
}
```

4.23 gridBFS

```
int dx[] = {0,0,1,-1};
int dy[] = {1,-1,0,0};

int isvalid(int x, int y, int n, int m) {
    return x >= 0 && y >= 0 && x < n && y < m;
}

void bfs (vector<vector<int>>& graph, vector<vector<bool>>&
     visited, pair<int,int>& v) {
    int n =graph.size();
    int m = graph[0].size();
    queue<pair<int,int>> q;
    visited[v.first][v.second] = true;
    q.push({v.first, v.second});

    while (!q.empty()) {
        auto [x, y] = q.front(); q.pop();
        FOR (i, 4) {
            int ax = x +dx[i];
            int ay = y +dy[i];

            if (isvalid(ax,ay,n,m) && !visited[ax][ay]) {
                visited[ax][ay] = true;
                q.push({ax, ay});
            }
        }
    }
}
```

5 Math

5.1 CRT

```
#define ld long double

ll modinv(ll a, ll b, ll s0=1, ll s1=0){ return b == 0 ? s0
     : modinv(b, a%b, s1, s0 - s1 * (a/b)); }
ll mul(ll a, ll b, ll m){
  ll q = (ld)a*(ld)b / (ld)m;
  ll r = a*b - q*m;
  return (r + m) % m;
}

struct Equation {
  ll mod, ans;
  bool valid;
  Equation() { valid = false; }
  Equation(ll a, ll m) { mod = m, ans = (a % m + m) % m,
       valid = true; }
  Equation(Equation a, Equation b){
    if(!a.valid || !b.valid){ valid = false; return; }
    ll g = gcd(a.mod, b.mod);
    if((a.ans - b.ans) % g != 0){ valid = false; return; }

    valid = true;
    mod = a.mod * (b.mod / g);
    ans = a.ans;
    ans += mul( mul(a.mod,  modinv(a.mod, b.mod), mod),    (b
         .ans - a.ans) / g, mod);

    ans = (ans % mod + mod) % mod;
  }
  Equation operator+(const Equation& b) const { return
       Equation(*this, b); }
};
// Equation eq1(2, 3); // x = 2 mod 3
// Equation eq2(3, 5); // x = 3 mod 5
// Equation ans = eq1 + eq2;
```

5.2 Catalan

```
int main() {
    ios_base::sync_with_stdio(0); cin.tie(nullptr);
    vector<ll> cat_nums(11);
    cat_nums[0] = 1;
    for(int i = 0; i < 10; i++) {
        cat_nums[i+1] = cat_nums[i] * ((2*i+2)*(2*i+1))/((i
             +2)*(i+1));
    }
    return 0;
}

/* Alguns exemplos de como usar os numeros de Catalan

1. Cat(n) counts the number of distinct binary trees with n
     vertices, e.g. for n = 3:

          *    *   *   *   *
         /    /   / \   \   \
        *    *   *   *   *   *
       /      \         /     \
      *        *       *       *

2. Cat(n) counts the number of expressions containing n
     pairs of parentheses which are
correctly matched, e.g. for n = 3, we have: ()()(), ()(()),
     (())(), ((())), and (()()).

3. Cat(n) counts the number of different ways n + 1 factors
     can be completely parenthesized, e.g. for n = 3 and 3 +
      1 = 4 factors: {a, b, c, d}, we have: (ab)(cd), a(b(cd
     )),
((ab)c)d, (a(bc))(d), and a((bc)d).

4. Cat(n) counts the number of ways a convex polygon (see
     Section 7.3) of n + 2 sides
can be triangulated. See Figure 5.1, left.

5. Cat(n) counts the number of monotonic paths along the
     edges of an n x n grid, which
do not pass above the diagonal. A monotonic path is one
     which starts in the lower
left corner, finishes in the upper right corner, and
     consists entirely of edges pointing
rightwards or upwards. See Figure 5.1, right and also see
     Section 4.7.1.

*/
```

5.3 Combinatorics

```
vector<ll> fat, finv;

void Combin(int n){  // precalc
  fat.assign(n+1, 1);
  for(int i=2; i<=n; i++) fat[i] = fat[i-1]*i % mod;
  finv.assign(n+1, fexp(fat.back(), mod-2));
  for(int i=n; i>0; i--) finv[i-1] = finv[i]*i % mod;
}

ll choose(ll n, ll k){ if(k>n||k<0) return 0;
  return fat[n] * finv[k] % mod * finv[n-k] % mod;
}
```

```cpp
ll chooseLinear(ll n, ll k){ //O(k) || min(k, n-k);
  k = max(k, n-k);
  ll ans = 1, inv=1;
  for(int i=n; i>k; i--) ans = ans*i % mod;
  for(int i=1; i<=n-k; i++) inv = inv*i % mod;
  return ans * fexp(inv, mod-2) % mod;
}

ll permRepetition(const vector<int> &cnt){
  ll n = accumulate(begin(cnt), end(cnt), 0ll), ans = fat[n
      ];
  for(int x : cnt) ans = ans * finv[x] % mod;
  return ans;
}

//#Colorir N posicoes com exatamente K cores (equival
    stirling_2 * k!)
ll RhymingK(ll n, ll k){ //O(K log N)
    ll ans = 0;
    for(int j=0; j<=k; j++)
        if(j&1) ans = (ans + mod - choose(k, k-j) * fexp(k-j
            , n) % mod) % mod;
        else    ans = (ans + mod + choose(k, k-j) * fexp(k-j
            , n) % mod) % mod;
  return ans;
}

//#Colorir N posicoes de um colar com M cores (array
    circular)
ll burnsideLemma(ll n, ll m){
    ll ans = fexp(m, n);
    for(int i=1; i<n; i++)
        ans += fexp(m, gcd(i, n)), ans %= mod;
    return ans * fexp(n, mod-2);
}


ll pascal[5001][5001]; // pascal[n][k] = choose(n, k);
void Pascal(int N){
  pascal[0][0] = 1;
  for(int n=1; n<=N; n++){
    pascal[n][0] = pascal[n][n] = 1;
    for(int k=1; k<n; k++)
      pascal[n][k] = (pascal[n-1][k-1] + pascal[n-1][k]) %
          mod;
  }
}


////////////////////////////
////// Stars and Bars //////
////////////////////////////

ll starsBars(ll n, ll k){ //O(choose)
  return choose(n+k-1, n);
}
ll starsLowerBound(ll n, const vector<ll> &lw){ //O(k)
  for(auto x : lw) n -= x;
  return starsBars(n, lw.size());
}
ll starsUpperBound(ll n, ll k, ll up){ //O(k)
  ll ans = 0;
  for(int i=0; i<=k; i++)
    ans += choose(k, i) * choose(n+k-1-(up+1)*i, k-1) % mod
        * (i&1? -1:+1);
  return ans;
}
ll starsUpperBound(ll M, const vector<ll> &up){ //O(N*M)
  int N = up.size();
  vector dp(up.size()+1, vector<ll>(N+1));
  for(int m=0; m<=M; m++) dp[0][m] = choose(N+m-1, m);
  for(int n=1; n<=N; n++)
    for(int m=0; m<=M; m++)
      dp[n][m] = dp[n-1][m] - (m-up[n-1]-1 < 0 ? 0 : dp[n
          -1][m-up[n-1]-1]);
  return dp[N][M];
}
ll starsLowerUpperBound(ll n, const vector<ll> &lw, const
    vector<ll> &up){ //O(N*M)
  for(auto x : lw) n -= x;
  return starsUpperBound(n, up);
}

///// Propriedades /////
ll sumNci (ll n){ return fexp(2, n); }              //SUM(0<=
    i<=n) choose(n,i);
ll sumicK (ll n, ll k){ return choose(n+1, k+1); } //SUM(0<=
    i<=n) choose(i,k);
ll sumNKcK(ll n, ll k){ return choose(n+k+1, k); } //SUM(0<=
    i<=k) choose(n+i,i);
ll sumNsqr(ll n){ return choose(n+n, n); }          //SUM(0<=
    i<=n) choose(n,i)^2;
ll catalan(ll n){ return choose(2*n, n) * fexp(n+1, mod-2) %
     mod; }
```

5.4 FFT

Fast Fourier Transform for polynomials multiplication

conv(a, b) = c, where $c[x] = \sum a[i]b[x-i]$.

fft(a) computes $\hat{f}(k) = \sum_x a[x]\exp(2\pi i \cdot kx/N)$ for all k. N must be a power of 2.

Rounding is safe if $(\sum a_i^2 + \sum b_i^2)\log_2 N < 9 \cdot 10^{14}$ (in practice 10^{16}; higher for random inputs).

O(N log N) // N=|A|+|B| (1s N <= 2^22)

```
|  Four Sum i<j<k<l   ||    Tree Sum i<j<k      |
|iiii = vx4;          || iii = vx3;             |
|iiij = conv(vx3, v)  || iij = conv(vx2, v)     |
|iijj = conv(vx2, vx2)|| ijk = conv(v,v,v)      |
|iijk = conv(vx2, v,v)|| ans=(ijk-3*iij+2*iii)/6|
|ijkl = conv(v,v,v,v) ||  // vx3[i*3] = v[i] //
|ans = (ijkl -6*iijk +3*iijj +8*iiij -6*iiii) / 24
* similar pra FWHT, mas vx3 vira V^V^V ou V|V|V e etc...
```

```cpp
#define ld double
typedef complex<ld> CD;
const ld PI = acos(-1);

void fft(vector<CD> &a, bool inverse=false){
  int n = a.size(), L = 31 - __builtin_clz(n);
  vector<int> rev(n);
  for(int i=0; i<n; i++) rev[i] = (rev[i/2] | (i&1)<<L)/2;
  for(int i=0; i<n; i++) if(i<rev[i]) swap(a[i], a[rev[i]]);

  for(int k=1; k<n; k<<=1){
    ld ang= PI / k * (inverse ? -1 : +1);
    CD wlen(cos(ang), sin(ang));
    for(int i=0; i<n; i+=k+k){  CD w(1);
      for(int j=0; j<k; j++, w *= wlen){
        CD u = a[i+j];
        CD v = a[i+j+k] * w;
        a[i+j] = u+v;
        a[i+j+k] = u-v;
      }
    }
  }
  if(inverse) for (CD &x : a) x /= n;
}

vector<ld> conv(const vector<ld>& a, const vector<ld>& b){
  if(a.empty() || b.empty()) return {}; //or(32-builtin_clz(
      m))
  int m = a.size()+b.size()-1, n=1<<__bit_width(m-1);//
      ifcomp err

  vector<CD> fa(begin(a), end(a)); fa.resize(n);
  vector<CD> fb(begin(b), end(b)); fb.resize(n);

  fft(fa); fft(fb);
  for(int i=0; i<n; i++) fa[i] *= fb[i];
  fft(fa, true);

  vector<ld> ans(m);
  for(int i=0; i<m; i++) ans[i] = fa[i].real();
  return ans;
}
```

5.5 FFT2in1

Fast Fourier Transform for polynomials multiplication

conv(a, b) = c, where $c[x] = \sum a[i]b[x-i]$.

fft(a) computes $\hat{f}(k) = \sum_x a[x]\exp(2\pi i \cdot kx/N)$ for all k. N must be a power of 2.

Rounding is safe if $(\sum a_i^2 + \sum b_i^2)\log_2 N < 9 \cdot 10^{14}$ (in practice 10^{16}; higher for random inputs).

O(N log N) // N=|A|+|B| (1s N <= 2^22)

```
|  Four Sum i<j<k<l   ||    Tree Sum i<j<k      |
|iiii = vx4;          || iii = vx3;             |
|iiij = conv(vx3, v)  || iij = conv(vx2, v)     |
|iijj = conv(vx2, vx2)|| ijk = conv(v,v,v)      |
|iijk = conv(vx2, v,v)|| ans=(ijk-3*iij+2*iii)/6|
|ijkl = conv(v,v,v,v) ||  // vx3[i*3] = v[i] //
|ans = (ijkl -6*iijk +3*iijj +8*iiij -6*iiii) / 24
* similar pra FWHT, mas vx3 vira V^V^V ou V|V|V e etc...
```

```cpp
#define ld double //(10% slower if long double)
typedef complex<ld> CD;

void fft(vector<CD>& a){
  int n = a.size(), L = 31 - __builtin_clz(n);

  static vector<complex<long double>> R(2, 1);
  static vector<CD> rt(2, 1);

  for(static int k = 2; k < n; k *= 2){
    auto x = polar(1.0L, acos(-1.0L)/k);
    R.resize(n); rt.resize(n);

    for(int i=k; i<2*k; i++)
      rt[i] = R[i] = i&1 ? R[i/2] * x : R[i/2];
  }
```

```
  vector<int> rev(n);
  for(int i=0; i<n; i++) rev[i] = (rev[i/2] | (i&1)<<L)/2;
  for(int i=0; i<n; i++) if(i<rev[i]) swap(a[i], a[rev[i]]);

  for(int k=1; k<n; k*=2)
    for(int i=0; i<n; i+=2*k)
      for(int j=0; j<k; j++){
        auto x=(ld*)&rt[j+k], y=(ld*)&a[i+j+k];
        CD z (x[0]*y[0] - x[1]*y[1], x[0]*y[1] + x[1]*y[0]);
      //  CD z = rt[j+k] * a[i+j+k]; //(~25% slower,but less
            code. Delete 2lines above)
        a[i+j+k] = a[i+j] - z;
        a[i+j] += z;
      }
}

vector<ld> conv(const vector<ld>& a, const vector<ld>& b){
  if(a.empty() || b.empty()) return {};
  vector<ld> res(a.size() + b.size() - 1);
  int n = 1<<(32 - __builtin_clz(res.size()));

  vector<CD> in(n), out(n);
  copy(begin(a), end(a), begin(in));
  for(int i=0; i<b.size(); i++) in[i].imag(b[i]);

  fft(in);
  for(auto& x : in) x *= x;
  for(int i=0; i<n; i++) out[i] = in[-i&(n-1)] - conj(in[i])
      ;
  fft(out);

  for(int i=0; i<res.size(); i++) res[i] = imag(out[i]) /
       (4*n);
  return res;
}
```

5.6 FFTmod

Fast Fourier Transform for polynomials multiplication with **MOD**
FFT com **ALTA PRECISAO** (nao precisa do mod, so coloque cut=1<<15)
Can be used for convolutions modulo arbitrary integers.
as long as $N \log_2 N \cdot \text{mod} < 8.6 \cdot 10^{14}$ (in practice 10^{16} or higher).

!!! Inputs must be in [0, mod). !!!
Get the fft function from fft section.
O(N log N) // (2x slower than NTT or FFT)

```
#include "FFT.cpp"

template<const int mod> vector<ll> convMod(const vector<ll>
     &a, const vector<ll> &b){
  if (a.empty() || b.empty()) return {};
  vector<ll> res(a.size() + b.size() - 1);
  int B=32-__builtin_clz(res.size()), n=1<<B, cut=int(sqrt(
       mod));
  vector<CD> L(n), R(n), outs(n), outl(n);

    for(int i=0; i<a.size(); i++) L[i] = CD((int)a[i] / cut,
         (int)a[i] % cut);
  for(int i=0; i<b.size(); i++) R[i] = CD((int)b[i] / cut, (
       int)b[i] % cut);
    fft(L), fft(R);
  for(int i=0; i<n; i++){
    int j = -i&(n-1);
    outl[j] = (L[i] + conj(L[j])) * R[i] / (2.0 * n);
    outs[j] = (L[i] - conj(L[j])) * R[i] / (2.0 * n) / 1i;
  }
    fft(outl), fft(outs);

    for(int i=0; i<res.size(); i++){
    ll av = (ll)(real(outl[i])+.5) % mod;
    ll bv = (ll)(imag(outl[i])+.5) + (ll)(real(outs[i])+.5);
        ll cv = (ll)(imag(outs[i])+.5);
    res[i] = ((av * cut + bv) % mod * cut + cv) % mod;
  }
  return res;
}
```

5.7 FWHT

Fast Walsh Hadamard Transform - **Convolucao de XOR, OR e AND**
O(N log N)

```
template<const char op>
vector<ll> FWHT(vector<ll> a, const bool inv = false){
  int n = a.size();
  for(int len=1; len<n; len+=len)
    for(int i=0; i<n; i += 2*len)
      for(int j=0; j<len; j++){
        ll u = a[i+j], v = a[i+j+len];
        if(op == '^') a[i+j]   = u+v, a[i+j+len] = u-v;
        if(op == '|') a[i+j+len] = v + (inv ? -u : +u);
        if(op == '&') a[i+j]     = u + (inv ? -v : +v);
      }

  if(op=='^'&&inv) for(auto &x : a) x /= n;
  return a;
}

template<const char op>
vector<ll> multiply(vector<ll> a, vector<ll> b){
  int n=1; while(n < max(a.size(), b.size())) n*=2;
  a.resize(n, 0);  b.resize(n, 0);
  a = FWHT<op>(a); b = FWHT<op>(b);

  vector<ll> ans(n);
  for(int i=0; i<n; i++) ans[i] = a[i]*b[i] % mod;
  ans = FWHT<op>(ans, true);
  return ans;
}


const int mxlog = 17;
vector<ll> subset_multiply(vector<ll> a, vector<ll> b){ //
     OPTIONAL
  int n = 1; while(n < max(a.size(), b.size())) n <<= 1;
  a.resize(n, 0);  b.resize(n, 0);
  vector<ll> ans(n, 0LL); vector A(mxlog+1, vector<ll>(n)),
       B = A;
  for(int i=0; i<n; i++) A[__builtin_popcount(i)][i]=a[i], B
       [__builtin_popcount(i)][i]=b[i];
  for(int i=0; i<=mxlog; i++) A[i] = FWHT<'|'>(A[i]), B[i] =
        FWHT<'|'>(B[i]);
  for(int i=0; i<=mxlog; i++){
    vector<ll> C(n);
    for(int x=0; x<=i; x++)
      for(int j=0; j<n; j++)
        C[j] += A[x][j] * B[i-x][j];

    C = FWHT<'|'>(C, true);
    for(int j=0; j < n; j++)
      if(__builtin_popcount(j) == i)
        ans[j] += C[j];
  }
  return ans;
}
```

5.8 GaussElimin

Eliminacao Gaussiana - O(N*N*M)

Resolve um sistema de equacoes lineares, escalonando a matriz;

```
Entrada       || Saida         || interpretacao
1  4  6   80  || 1  0  0  76   || X0 = 76
2  0  6  140  || 0  1  0  4    || X1 = 4
1  2  12  60  || 0  0  1  -2   || X2 = -2

Entrada       || Saida         || interpretacao
1  4  6  18   || 1  4  0  -6   || X0 = -4 * X1  - 6
0  0  6  24   || 0  0  1   4   || X2 =  4
1  4  3   6   || 0  0  0   0   || 0  =  0

* Se 0 != 0, o sistema nao tem solucao
* Variaveis do lado direito da equacao tem valor livre
```

```
template<typename T> struct Gauss {
  int n, m = 0;
  vector<vector<T>> mat;
  Gauss(int n) : n(n){}

  void addLine(vector<T> l = vector<T>()){
    l.resize(n, T(0));
    mat.push_back(l); m++;
  }
  void solve(){
    for(int c=0, i=0, g; c<n && i < m; c++, i+=!g){
      for(int j=i; j<m && mat[i][c] == T(0); j++)
        swap(mat[j], mat[i]);

      if(g = mat[i][c] == T(0)) continue;
      for(int j=n-1; j>=0; j--) mat[i][j] /= mat[i][c];

      for(int j=0; j<m; j++) if(i != j && mat[j][c] != T(0))
           {
        T alpha = mat[j][c];
        for(int k=i; k<n; k++)
          mat[j][k] -= mat[i][k] * alpha;
      }
    }
  }
  int isValid(){
        int pivos = 0;
        for(int i=0, c; i<m; i++){
      for(c=0; c<n-2; c++) if(mat[i][c] != T(0)) break;
            if(mat[i][c] != T(0)){ pivos++;
                for(int j=c+1; j<n-1; j++)
                    if(mat[i][j] != T(0))
                        pivos = -2*n;
```

```
            }
        else if(mat[i][n-1] != T(0)) return 0; // 0 = 1
    }
        return pivos == n-1 ? +1 : -1; // 1 - Solucao unica
            // 0 - Sem solucao // -1 - Infinitas solucoes
    }
  void print(){ for(auto v : mat){ for(auto x:v) cout<<x<<"\
      t"; cout<<"\n"; } }
  void printSolution(){ // OPTIONAL / FOR DEBUG
    for(int i=0, c; i<m; i++){
      for(c=0; c<n-2; c++) if(mat[i][c] != T(0)) break;
      if(mat[i][c] != T(0)) cout << "X" << c << " = ";
      else cout << "0  = ";
      for(int j=c+1; j<n-1; j++) if(mat[i][j] != T(0))
        cout << mat[i][j]*T(-1) << " * X" << j << "   +   ";
      cout << mat[i][n-1] << endl;
    }
  }
};
```

5.9 Matrix

```
template<typename T> struct Matrix {
  vector<vector<T>> mat;
  int n, m;

  Matrix(int N, int M=0) : n(N), m(M?M:N){ mat.assign(n,
      vector<T>(m, 0)); }

  friend Matrix operator* (const Matrix &a, const Matrix &b)
      {
    assert(a.m == b.n);
    Matrix ans(a.n, b.m);
    for(int i=0; i<a.n; i++)
      for(int j=0; j<b.m; j++)
        for(int k=0; k<a.m; k++)
          ans.mat[i][j] += a.mat[i][k] * b.mat[k][j];
    return ans;
  }
};

template<typename T> Matrix<T> inverse(Matrix<T> mat){
  int n = mat.n; assert(n == mat.m);
  Gauss<T> g(n+n);
  for(int i=0; i<n; i++){
    vector<T> line = mat.mat[i];
    line.resize(n+n, 0); line[n+i] = 1;
    g.addLine(line);
  }
  g.solve();
  for(int i=0; i<n; i++)
    mat.mat[i] = {begin(g.mat[i])+n, end(g.mat[i])};
  return mat;
}
```

5.10 NTT

Number Theoretic Transform for polynomials multiplication MOD

conv(a, b) = c, where $c[x] = \sum a[i]b[x-i]$.

!!! Inputs must be in [0, mod). !!!

For manual convolution: NTT the inputs, multiply pointwise, divide by n, reverse(start+1, end), NTT back.
Consider using template<const ll mod, const ll root> in conv and ntt if you need more than one mod.
Mod primes must be of the form $2^a b + 1$,
Consider using CRT (Chinese Remainder Theorem) or FFTmod if you need a different MOD.
ntt(a) computes $\hat{f}(k) = \sum_x a[x] g^{xk}$ for all k, where $g = \text{root}^{(mod-1)/N}$.

O(N log N)

```
const ll mod = 998244353, root = 62; //// 9e8 < mod1 < 1e9

void ntt(vector<ll> &a){
  int n = a.size(), L = 31 - __builtin_clz(n);

  static vector<ll> rt(2, 1);
  for(static int k=2, s=2; k<n; k*=2, s++){
    rt.resize(n);
    ll z[] = {1, fexp(root, mod >> s)};
    for(int i=k; i<2*k; i++) rt[i] = rt[i/2] * z[i&1] % mod;
  }

  vector<int> rev(n);
  for(int i=0; i<n; i++) rev[i] = (rev[i / 2] | (i & 1) << L
      ) / 2;
  for(int i=0; i<n; i++) if (i < rev[i]) swap(a[i], a[rev[i
      ]]);

  for(int k=1; k<n; k*=2)
    for(int i=0; i<n; i+=2*k)
      for(int j=0; j<k; j++){
        ll z = rt[j+k] * a[i+j+k] % mod, &ai =a[i+j];
        a[i+j+k] = ai - z + (z>ai? mod:0);
        ai += z - (ai+z>=mod? mod:0);
      }
}

vector<ll> conv(const vector<ll> &a, const vector<ll> &b) {
  if (a.empty() || b.empty()) return {};
  int s = a.size()+b.size()-1, B = 32 - __builtin_clz(s), n
      = 1<<B;

  vector<ll> L(a), R(b), out(n);
  L.resize(n), R.resize(n);
  ntt(L), ntt(R);

  int inv = fexp(n, mod - 2);
  for(int i=0; i<n; i++) out[-i&(n-1)] = L[i]*R[i] % mod *
      inv % mod;

  ntt(out);
  return {out.begin(), out.begin() + s};
}

const ll mod2 = 918552577, root2 = 63; // 9e8 < mod2 < 1e9
    //also valid mods
const ll mod3 = 7340033,   root3 = 25; // 7e6 < mod3 < 1e7
```

Computes the first LIM terms of $P(x)^K$ in O(n*limit)

```
vector<ll> power(vector<ll> &p, int k, int limit=-1){ //O(n*
    limit)
  while(p.back() == 0) p.pop_back();
  if(p.empty() || limit == 0) return {};
  if(limit == -1) limit = (p.size()-1) * k;

  vector<ll> ans(limit+1, 0);
  ans[0] = fexp(p[0], k);

  for(int i=1; i<=limit; i++){
    for(int j=1; j <= i && j < p.size(); j++){
      ans[i] += p[j] * ans[i-j] % mod * (k*j - (i-j)) % mod;
      ans[i] %= mod;
    }
    ans[i] = ans[i] * fexp(i * p[0] % mod, mod-2) % mod;
  }
  return ans;
}
```

5.11 Sieve

```
const int LIM = 1e6 + 5;
bool isPrime[LIM];

vector<int> sieve(){
    memset(isPrime, 1, sizeof isPrime);
    isPrime[0] = isPrime[1] = false;

    vector<int> primes;
    FOR(i, 2, LIM) {
        if (isPrime[i]) {
            primes.push_back(i);
            for(int j = i + i; j < LIM; j += i) isPrime[j] =
                false;
        }
    }

    return primes;
}
// O(n*log(log n))


vector<int> calc_prime(int n){ // O(n log n)
  vector<int> prime(n+1, 1);
  for(int i=2; i<=n; i++) if(prime[i] == i)
    for(int j=i+i; j<=n; j+=i)
      prime[j] = false;
  return prime;
}

vector<int> calc_phi(int n){ // O(n log n)
  vector<int> phi(n+1);
  for(int i=0; i<=n; i++) phi[i] = i;
  for(int i=2; i<=n; i++) if(phi[i] == i)
    for(int j=i; j<=n; j+=i)
      phi[j] -= phi[j] / i;
  return phi;
}

vector<int> calc_mobius(int n){ // O(n log n)
  vector<int> mobius(n+1, 1), prime(n+1, 1);
  for(int i=2, j; i<=n; i++) if(prime[i])
    for(mobius[i]=-1, j=i+i; j<=n; j+=i){
      if((j/i)%i) mobius[j] *= -1;
      else mobius[j] = 0;
      prime[j] = false;
    }
  return mobius;
}
```

5.12 StirlingNumbers

```cpp
#include "Combinatorics.cpp"

ll st1[MAXN][MAXN];
void StirlingFirst_nn(){ // O(n^2)
  st1[0][0] = 1;

  for(int n=1; n<MAXN; n++)
    for(int k=1; k<=n; k++)
      st1[n][k] = (n-1) * st1[n-1][k] + st1[n-1][k-1];
}

ll st2[MAXN][MAXN];
void StirlingSecond_nn(){ // O(n^2)
  st2[0][0] = 1;

  for(int n=1; n<MAXN; n++)
    for(int k=1; k<=n; k++)
      st2[n][k] = k * st2[n-1][k] + st2[n-1][k-1];
}

//// Fixed N ////
#include "NTT.cpp"

vector<ll> shift(vector<ll> pol, ll n){ //para StirlingFirst
    ll s = pol.size(), k = 1;
    vector<ll> a(s), b(s);
    for(int i=0; i<s; i++, k=k*n%mod){
      a[i] = pol[i] * fat[i] % mod;
      b[s-i-1] = k * finv[i] % mod;
    }
    a = conv(a, b);
    for(int i=0; i<s; i++) pol[i] = a[s-1+i] * finv[i] % mod
        ;
    return pol;
}

vector<ll> StirlingFirst(ll n, bool sign = false){  //O(n
    log^2 n) //lembrar do fat/finv
  if(n == 0) return {1};
  vector<ll> pol(n+1), q = n&1 ? StirlingFirst(n-1) :
       StirlingFirst(n/2);

  if(n&1){
    for(int i=0; i<n-1; i++)
      pol[i+1] = (q[i] + q[i+1] * (n-1)) % mod;
    pol[n] = 1;
  }
  else pol = conv(q, shift(q, n/2));

  if(sign&&n&1) for(int i=0; i<n/2+1; i++) pol[i*2]    = (mod
       - pol[i*2])    % mod;
  if(sign>(n&1))for(int i=0; i<n/2;   i++) pol[i*2+1] = (mod
       - pol[i*2+1]) % mod;

  return pol;
}

vector<ll> StirlingSecond(int n){ //O(n log^2 n) //lembrar
    do fat/finv
  vector<ll> a(n+1), b(n+1);
  for(int i=0; i<n+1; i++){
    a[i] = fexp(i, n) * finv[i] % mod;
    b[i] = i&1 ? mod - finv[i] : finv[i];
  }
  a = conv(a, b);
  return {begin(a), begin(a)+n+1};
}
```

5.13 fexp

```cpp
ll mod = 1e9 + 7;

ll fexp(ll b, ll p){
  ll ans = 1;
  while(p){
    if(p&1) ans = ans * b % mod;
    b = b * b % mod;
    p >>= 1;
  }
  return ans;
}
// O(Log P) // b - Base // p - Potencia

Matrix<ll> Matfexp(Matrix<ll>& b, ll p){
    Matrix<ll> ans(b.n);
    FOR (i,0, b.n) ans.mat[i][i] = 1;

    while(p) {
        if(p&1) ans = ans * b;
        b = b * b;
        p >>= 1;
    }
    return ans;
}
```

5.14 frac

```cpp
struct frac{
  ll num, den;
  frac(){}
  frac(ll num, ll den):num(num), den(den){
    if(!num) den = 1;
    if(num > 0 && den < 0) num = -num,  den = -den;
    simplify();
  }
  void simplify(){
    ll g = __gcd(abs(num), abs(den));
    if(g) num /= g,  den /= g;
  }
  frac operator+(const frac& b){ return {num*b.den + b.num*
      den, den*b.den};}
  frac operator-(const frac& b){ return {num*b.den - b.num*
      den, den*b.den};}
  frac operator*(const frac& b){ return {num*b.num, den*b.
      den};}
  frac operator/(const frac& b){ return {num*b.den, den*b.
      num};}
  bool operator<(const frac& b)const{ return num*b.den < den
      *b.num; }
};
```

5.15 getprimes

```cpp
#define FOR(i,j,n) for(int i = j; i < n; i++)
#define br '\n'


int cntprimes(int n) {
    int ans = 1;
    for (int i = 2; i*i <= n; i++) {
        int cnt = 0;

        while (n % i == 0) {
            n /= i;
            cnt++;
        }

        if (cnt > 0) ans *= cnt+1;
    }

    if (n > 1) ans *= 2;
    return ans;
}
// O(sqrt(n))


 vector<pair<int,int>> getprimes(int n) {
    vector<pair<int,int>> primes;
    for (int i = 2; i*i <= n; i++) {
        int cnt = 0;

        while (n % i == 0) {
            n /= i;
            cnt++;
        }

        if (cnt > 0) primes.push_back({i, cnt});
    }

    if (n > 1) primes.push_back({n, 1});
    return primes;
}
// O(sqrt(n))
```

5.16 mint

```cpp
struct mint {
  ll v = 0;
  mint(ll x=0) : v((x%mod+mod)%mod){}
  mint operator+ (const mint &b) const { ll a = v+b.v;
      return a < mod ? a : a-mod; }
  mint operator- (const mint &b) const { ll a = v-b.v;
      return a <  0  ? a+mod : a; }
  mint operator* (const mint &b) const { return v * b.v %
      mod; }
  mint operator/ (const mint &b) const { return v * fexp(b.v
      , mod-2) % mod;  }
};
// Extra Operators - Bool Operators
bool operator==(const mint&a, const mint &b){ return a.v ==
    b.v; }
bool operator!=(const mint&a, const mint &b){ return a.v !=
    b.v; }
bool operator< (const mint&a, const mint &b){ return a.v <
    b.v; }
// Assignment Operators
mint operator+= (mint&a, const mint &b){ return a = a + b; }
mint operator-= (mint&a, const mint &b){ return a = a - b; }
mint operator*= (mint&a, const mint &b){ return a = a * b; }
mint operator/= (mint&a, const mint &b){ return a = a / b; }
```

5.17 random

```cpp
mt19937 rng(chrono::steady_clock::now().time_since_epoch().
    count());
//int x = rng();

int uniform(int l, int r){
  uniform_int_distribution<int> uid(l, r);
  return uid(rng);
}
```

5.18 utils

```cpp
#define i128 __int128_t




ll gcd_ll(ll a, ll b) {
    while (b != 0) {
        ll t = a % b;
        a = b;
        b = t;
    }
    return a;
}

ll lcm_ll(ll a, ll b) {
    return a / gcd_ll(a, b) * b;
}

// Bit manipulation utilities
ll count_set_bits(ll n) {
    return __popcount<ll>(n); // __builtin_popcount(n);
}

inline ll safe_mod(ll x) {
    return (x % MOD + MOD) % MOD;
}


#define i128 __int128_t

// Usando i128 pra evitar overflow no lcm
i128 lcmgcd(ll a, ll b) {
    return (i128)a/gcd(a, b) * b;
}
// Dps checa se o lcm e maior que 1e18 e decide o que fazer
     (ignora ou outra coisa)

// ------------------------------------------------

// Cuidado com re-hashing do map e unordered_map
// Usar .reserve

// ------------------------------------------------

// Estrategia de random shuffle para algoritmos randomizados

void random_shuffle(vector<int>& vec, int n) {
    for(int i = 0; i < n; i++) {
        int j = rand() % (i+1);
        swap(vec[i], vec[j]);
    }
}
```

6 Others

6.1 BuscaBinariaParalela

Busca Binaria Paralela - O(Q+K log K)

Dado **K** updates ordenados pelo tempo, e **Q** queries da forma:
- Qual o primeiro momento entre [0, K-1] em que * e verdade?

De forma que:
- A resposta e monotonica (se e verdadeiro para t=x, e verdade para t'=x+1)
- E possivel responder em O(N^2) iterando pelos updates e verificando a cada passo todas as queries nao satisfeitas ainda.

* No caso do range de busca ser [0, 1e9]: recursivo OU veja se e possivel olhar apenas os valores que aparecem no input(?)

```cpp
void reset(){};
void update(int mid){};
bool check(int i){}

//q = #queries | [0, k-1] = intervalo de busca
vector<int> pbs(int q, int k){
  bool LACK = true;
  vector<int> L(q, 0), R(q, k-1), ans(q, -1);
  vector<vector<int>> atMid(k);

  while(LACK){
    reset(); LACK = false;

    for(int i=0; i<q; i++)
      if(L[i] <= R[i])
        atMid[(L[i]+R[i])/2].push_back(i), LACK = true;

    for(int mid=0; mid<k; mid++){
      update(mid);
      for(auto i : atMid[mid])
        if(check(i)) R[i] = mid-1, ans[i] = mid;
        else L[i] = mid+1;
      atMid[mid].clear();
    }
  }
  return ans;
}

vector<int> ans; // recursive version
void pbsR(vector<int> &qidx, int l, int r){
  if(l > r) return;
  int mid = (l+r)/2; update(mid);
  vector<int> L, R;
  for(auto i : qidx){
    if(check(i)) L.push_back(i), ans[i] = mid;
    else R.push_back(i);
  }
  if(!L.empty()) pbsR(L, l, mid-1);
  if(!R.empty()) pbsR(R, mid+1, r);
}
```

6.2 BuscaTernaria

Ternary search and Golden section search

if(f1 < f2) to find minimum
if(f1 > f2) to find maximum

Golden Search faz menos chamadas a funcao;
Para busca em inteiros, cuidado, mantenha uma margem do L e R para evitar looping infinito.

```cpp
#define ld long double

ld func(ld x){ return 2*x*x - 4*x + 2; }

ld ternary_search(ld l, ld r) {
  for(int it=250; it--;){
    ld m1 = l + (r-l)/3, m2 = r - (r-l)/3;
    ld f1 = func(m1), f2 = func(m2);
    if(f1 < f2) r = m2;
    else l = m1;
  }
  return l;
}

ld golden_search(ld l, ld r) {
  const ld iphi = (sqrt(5)-1)/2;

  ld m1 = r - iphi*(r-l);
  ld m2 = l + iphi*(r-l);
  ld f1 = func(m1), f2 = func(m2);

  for(int it=250; it--;){
    if(f1 < f2){
      r = m2;
      tie(m2, f2) = tie(m1, f1);
      f1 = func( m1 = r - iphi*(r-l) );
    } else {
      l = m1;
      tie(m1, f1) = tie(m2, f2);
      f2 = func( m2 = l + iphi*(r-l) );
    }
  }
  return l;
}

int iternary_search(int l, int r) {
  while(r-l > 5){  //margem de seguranca
    int m1 = l + (r-l)/3, m2 = r - (r-l)/3;
    int f1 = func(m1), f2 = func(m2);
    if(f1 < f2) r = m2;
    else l = m1;
  }
  int ans = r, fa = func(ans);
  for(int f1; l<r; l++){
    f1 = func(l);
    if(f1 < fa)
      ans = l, fa = f1;
  }
  return ans;
}
```

6.3 Date

```cpp
//
```

converts Gregorian date to integer (Julian day number)

```cpp
int dateToInt (int m, int d, int y){ return
```

```
  + 1461 * (y + 4800 + (m - 14) / 12) / 4
  + 367  * (m - 2    - (m - 14) / 12  * 12)  / 12
  - 3    *((y + 4900 + (m - 14) / 12) / 100) / 4
  + d - 32075;
}
```

converts integer (Julian day number) to Gregorian date: day/month/year

```
tuple<int, int, int> intToDate(int jd){
  int x, n, i, j, d, m, y;
  x = jd + 68569;
  n = 4 * x / 146097;
  x -= (146097 * n + 3) / 4;
  i = (4000 * (x + 1)) / 1461001;
  x -= 1461 * i / 4 - 31;
  j = 80 * x / 2447;
  d = x - 2447 * j / 80;
  x = j / 11;
  m = j + 2 - 12 * x;
  y = 100 * (n - 49) + i + x;
  return {d, m, y};
}
```

converts integer (Julian day number) to day of week

```
string dayOfWeek[] = {"Mon", "Tue", "Wed", "Thu", "Fri", "
    Sat", "Sun"};
string intToWeek (int jd){ return dayOfWeek[jd % 7]; }
```

6.4 Dice

```
   Dadinho de 6 lados      |   |4|       ^ norte
24 estados possiveis       | |2|6|5| w < + > leste
Assume que a soma de lados |   |3|       v
opostos e 7.               |   |1|      sul
Se precisar de labels diferentes, crie um
 array<int, 7> face = {0, 1, 2, 3, 4, 5, 6}
e acesse a posicao da face retornada.
```

```
struct Dice {
  int topo=6, norte=4, leste=5;

  int top()   { return topo; }
  int north() { return norte; }
  int east()  { return leste; }
  int bottom(){ return 7 - topo; }
  int south() { return 7 - norte; }
  int west()  { return 7 - leste; }

  void roll_north(){ tie(topo, norte) = pair(south(), top())
      ;  }
  void roll_east( ){ tie(topo, leste) = pair(west(),  top())
      ;  }
  void rotate_ccw(){ tie(norte,leste) = pair(east(),south())
      ;  }
  void roll_south(){ roll_north(); roll_north(); roll_north
      (); }
  void roll_west( ){ roll_east();  roll_east();  roll_east()
      ;  }
  void rotate_cw (){ rotate_ccw(); rotate_ccw(); rotate_ccw
      (); }

  int get_id(){ //[0, 23]
    int id = (topo-1)^(norte-1);
    id %= topo==3||topo==4 ? 6 : 4;
    return topo*4 + id - 4;
  }
};
```

6.5 Hungarian

```
Hungarian Algorithm - Assignment Problem
Algoritmo para o problema de atribuicao minima.

Complexity: O(N^2 * M)

hungarian(int n, int m);  ->  Retorna o valor do custo
     minimo
getAssignment(int m)      ->  Retorna a lista de pares <
     linha, Coluna> do Minimum Assignment

n -> Numero de Linhas // m -> Numero de Colunas

IMPORTANTE! O algoritmo e 1-indexado
IMPORTANTE! O tipo padrao esta como int, para mudar para
      outro tipo altere | typedef <TIPO> TP; |
Extra: Para o problema da atribuicao maxima, apenas
     multiplique os elementos da matriz por -1
```

```
typedef int TP;

const int MAXN = 1e3 + 5;
const TP INF = 0x3f3f3f3f;

TP matrix[MAXN][MAXN];
TP row[MAXN], col[MAXN];
int match[MAXN], way[MAXN];

TP hungarian(int n, int m){
  memset(row, 0, sizeof row);
  memset(col, 0, sizeof col);
  memset(match, 0, sizeof match);

  for(int i=1; i<=n; i++){
    match[0] = i;
    int j0 = 0, j1, i0;
    TP delta;

    vector<TP> minv (m+1, INF);
    vector<bool> used (m+1, false);

    do {
      used[j0] = true;
      i0 = match[j0];
      j1 = -1;
      delta = INF;

      for(int j=1; j<=m; j++)
        if(!used[j]){
          TP cur = matrix[i0][j] - row[i0] - col[j];

          if( cur < minv[j] ) minv[j] = cur, way[j] = j0;
          if(minv[j] < delta) delta = minv[j], j1 = j;
        }

      for(int j=0; j<=m; j++)
        if(used[j]){
          row[match[j]] += delta,
          col[j] -= delta;
        }
        else minv[j] -= delta;

      j0 = j1;
    } while(match[j0]);
    do {
      j1 = way[j0];
      match[j0] = match[j1];
      j0 = j1;
    } while(j0);
  }

  return -col[0];
}

vector<pair<int, int>> getAssignment(int m){
  vector<pair<int, int>> ans;
  for(int i=1; i<=m; i++)
    ans.push_back(make_pair(match[i], i));
  return ans;
}
```

6.6 MO

```
Algoritmo de MO para query em range

Complexity: O( (N + Q) * SQRT(N) * F ) | F e a
     complexidade do Add e Remove

IMPORTANTE! Queries devem ter seus indices (Idx) 0-
     indexados!

Modifique as operacoes de Add, Remove e GetAnswer de
     acordo com o problema.
BLOCK_SZ pode ser alterado para aproximadamente SQRT(
     MAX_N)
```

```
const int BLOCK_SZ = 700;

struct Query{
  int l, r, idx;
  Query(int l, int r, int idx) : l(l), r(r), idx(idx) {}
  bool operator < (Query q) const {
    if(l / BLOCK_SZ != q.l / BLOCK_SZ) return l < q.l;
    return (l / BLOCK_SZ &1) ? ( r < q.r ) : (r > q.r );
  }
};

void add(int idx);
void remove(int idx);
int getAnswer();

vector<int> MO(vector<Query> &queries){
  vector<int> ans(queries.size());

  sort(queries.begin(), queries.end()); // to use hilbert
       curves, call sortQueries instead

  int L = 0, R = 0;
  add(0);

  for(auto [l, r, idx] : queries){
    while(l < L) add(--L);
    while(r > R) add(++R);
    while(l > L) remove(L++);
    while(r < R) remove(R--);

    ans[idx] = getAnswer();
  }
```

```
  return ans;
}

//OPTIONAL
void sortQueries(vector<Query> &qr){
  vector<ll> h(qr.size());
  for(int i=0; i<qr.size(); i++) h[i] = hilbert(qr[i].l, qr[
      i].r);
  sort(qr.begin(), qr.end(), [&](Query&a, Query&b) { return
      h[a.idx] < h[b.idx]; });
}

inline ll hilbert(int x, int y){ //OPTIONAL
  static int N = 1 << (__builtin_clz(0) - __builtin_clz(MAXN
      ));
  int rx, ry, s; ll d = 0;
  for(s = N/2; s > 0; s /= 2){
    rx = (x & s) > 0, ry = (y & s) > 0;
    d += s * (ll)(s) * ((3 * rx) ^ ry);
    if(ry == 0) { if(rx == 1) x = N-1 - x, y = N-1 - y; swap
        (x, y); }
  }
  return d;
}
```

6.7 MOTree

Algoritmo de MO para query de **caminho em arvore**
Complexity: O((N + Q) * SQRT(N) * F) | F e a complexidade do Add e Remove
IMPORTANTE! 0-indexado!

```
const int MAXN = 1e5+5;
const int BLOCK_SZ = 500;
struct Query{int l, r, idx;}; //same of MO. Copy operator <

vector<int> g[MAXN];
int tin[MAXN], tout[MAXN];
int pai[MAXN], order[MAXN];

void remove(int u);
void add(int u);
int getAnswer();

void go_to(int ti, int tp, int otp){
  int u = order[ti], v, to;
  to = tout[u];
  while(!(ti <= tp && tp <= to)){ //subo com U (ti) ate ser
      ancestral de W
    v = pai[u];

    if(ti <= otp && otp <= to) add(v);
    else remove(u);

    u = v;
    ti = tin[u];
    to = tout[u];
  }

  int w = order[tp];
  to = tout[w];
  while(ti < tp){ //subo com W (tp) ate U
    v = pai[w];

    if(tp <= otp && otp <= to) remove(v);
    else add(w);

    w = v;
    tp = tin[w];
    to = tout[w];
  }
}

int TIME = 0;
void dfs(int u, int p){
  pai[u] = p;
  tin[u] = TIME++;
  order[tin[u]] = u;

  for(auto v : g[u])
    if(v != p)
      dfs(v, u);
  tout[u] = TIME-1;
}

vector<int> MO(vector<Query> &queries){
  vector<int> ans(queries.size());
  dfs(0, 0);

  for(auto &[u, v, i] : queries)
    tie(u, v) = minmax(tin[u], tin[v]);
  sort(queries.begin(), queries.end());

  add(0);
  int Lm = 0, Rm = 0;
  for(auto [l, r, idx] : queries){
    if(l < Lm) go_to(Lm, l, Rm), Lm = l;
    if(r > Rm) go_to(Rm, r, Lm), Rm = r;
    if(l > Lm) go_to(Lm, l, Rm), Lm = l;
    if(r < Rm) go_to(Rm, r, Lm), Rm = r;
    ans[idx] = getAnswer();
  }

  return ans;
}
```

6.8 stressTest

```
P=code   #mude pro filename do codigo
Q=brute  #mude pro filename do brute [correto]
g++ ${P}.cpp -o sol -O2 || exit 1
g++ ${Q}.cpp -o brt -O2 || exit 1
g++ gen.cpp  -o gen -O2 || exit 1
for ((i = 1; ; i++)) do
  echo $i
  ./gen $i > in
  ./sol < in > out
  ./brt < in > out2
  if (! cmp -s out out2) then
    echo "--> entrada:"
    cat in
    echo "--> saida code:"
    cat out
    echo "--> saida brute:"
    cat out2
    break;
  fi
done
```

7 String

7.1 KMP

```
vector<int> Pi(string &t){
  vector<int> p(t.size(), 0);

  for(int i=1, j=0; i<t.size(); i++){
    while(j > 0  && t[j] != t[i]) j = p[j-1];
    if(t[j] == t[i]) j++;
    p[i] = j;
  }
  return p;
}

vector<int> kmp(string &s, string &t){
  vector<int> p = Pi(t), occ;

  for(int i=0, j=0; i<s.size(); i++){
    while( j > 0 && s[i] != t[j]) j = p[j-1];
    if(s[i]==t[j]) j++;
    if(j == t.size()) occ.push_back(i-j+1), j = p[j-1];
  }
  return occ;
}
```

Optional: KMP Automato. j = state atual [root=j=0]

```
struct Automato {
  vector<int> p;
    string t;
  Automato(string &t) : t(t), p(Pi(t)){}
    int next(int j, char c){ //return nxt state
        if(final(j)) j = p[j-1];
        while(j && c != t[j]) j = p[j-1];
        return j + (c == t[j]);
    }
    bool final(int j){ return j == t.size(); }
};
```

KMP - Knuth-Morris-Pratt Pattern Searching
Complexity: O(|S|+|T|)
kmp(s, t) -> returns all occurences of t in s
p = Pi(t) -> p[i] = biggest prefix that is a sufix of t[0,i]

7.2 Manacher

Manacher Algorithm - O(N)
Find every palindrome in string

```
vector<int> manacher(string &st){
  string s = "$_";
  for(char c : st){ s += c; s += "_"; }
  s += "#";

  int n = s.size()-2, l=1, r=1;
  vector<int> p(n+2, 0);

  for(int i=1, j; i<=n; i++){
    p[i] = max(0, min(r-i, p[l+r-i]) ); //atualizo o valor
        atual para o valor do palindromo espelho na string
        ou para o total que esta contido
    while( s[i-p[i]] == s[i+p[i]] ) p[i]++;
    if( i+p[i] > r ) l = i-p[i], r = i+p[i];
  }
```

```
  for(auto &x : p) x--; //o valor de p[i] era o tamanho do
       palindromo + 1
  return p; //agora e o tamanho real
}
```

7.3 SuffixArray

```
sf = suffixArray(s) -> O(N log N)
LCP(s, sf) -> O(N)

SuffixArray -> index of suffix in lexicographic order
LCP[i] -> LargestCommonPrefix of sufix at sf[i] and sf[i
    -1]
LCP(i,j) = min(lcp[i+1...j])

To better understand, print: lcp[i] sf[i] s.substr(sf[i
    ])
```

```
vector<int> suffixArray(string s){
  int n = (s += "!").size();//if vector, s.push_back(-INF);
  vector<int> sf(n), ord(n), aux(n), cnt(n);
  iota(begin(sf), end(sf), 0);
  sort(begin(sf), end(sf), [&](int i, int j){ return s[i] <
       s[j]; });

  int cur = ord[sf[0]] = 0;
  for(int i=1; i<n; i++)
    ord[sf[i]] = s[sf[i]] == s[sf[i-1]] ? cur : ++cur;

  for(int k=1; cur+1 < n && k < n; k<<=1){
    cnt.assign(n, 0);
    for(auto &i : sf)          i = (i-k+n)%n, cnt[ord[i]]++;
    for(int i=1; i<n; i++)     cnt[i] += cnt[i-1];
    for(int i=n-1; i>=0; i--) aux[--cnt[ord[sf[i]]]] = sf[i
        ];
    sf.swap(aux);

    aux[sf[0]] = cur = 0;
    for(int i=1; i<n; i++)
      aux[sf[i]] = ord[sf[i]] == ord[sf[i-1]] &&
      ord[(sf[i]+k)%n] == ord[(sf[i-1]+k)%n] ? cur : ++cur;
    ord.swap(aux);
  }
  return vector<int>(begin(sf)+1, end(sf));
}

vector<int> LCP(string &s, vector<int> &sf){
  int n = s.size();
  vector<int> lcp(n), pof(n);
  for(int i=0; i<n; i++) pof[sf[i]] = i;

  for(int i=0, j, k=0; i<n; k?--k:k, i++){
    if(!pof[i]) continue;
    j = sf[pof[i]-1];
    while(i+k<n && j+k<n && s[i+k]==s[j+k]) k++;
    lcp[pof[i]] = k;
  }
  return lcp;
}
```

7.4 Z-Function

```
vector<int> Zfunction(string &s){ // O(N)
  int n = s.size();
  vector<int> z (n, 0);

  for(int i=1, l=0, r=0; i<n;  i++){
    if(i <= r) z[i] = min(z[i-l], r-i+1);

    while(z[i] + i < n && s[z[i]] == s[i+z[i]]) z[i]++;

    if(r < i+z[i]-1) l = i, r = i+z[i]-1;
  }
  return z;
}
```

7.5 ahoCorasick

```
Aho-Corasick: Trie automaton to search multiple patterns
     in a text
O(SUM|P| + |S|) * ALPHA

for(auto p: patterns) aho.add(p);
aho.buildSufixLink();
auto ans = aho.findPattern(s);

parent(p), sufixLink(sl), outputLink(ol), patternID(idw)
outputLink -> edge to other pattern end (when p is a
    sufix of it)
ALPHA -> Size of the alphabet. If big, consider changing
     nxt to map

To find ALL occurrences of all patterns, don't delete ol
     in findPattern. But it can be slow (at number of
    occ), so consider using DP on the automaton.
If you need a nextState function, create it using the
    while in findPattern.
if you need to store node indexes add int i to Node, and
     in Aho add this and change the new Node() to it:
vector<trie> nodes;
trie new_Node(trie p, char c){
  nodes.push_back(new Node(p, c));
  nodes.back()->i = nodes.size()-1;
  return nodes.back();
}
```

```
const int ALPHA = 26, off = 'a';
struct Node {
  Node* p  = NULL;
  Node* sl = NULL;
  Node* ol = NULL;
  array<Node*, ALPHA> nxt;

  int idw = -1, i; char c;

  Node(){ nxt.fill(NULL); }
  Node(Node* p, char c) : p(p), c(c) { nxt.fill(NULL); }
};
typedef Node* trie;
struct Corasick {
  trie root;
  int nwords = 0;
  vector<trie> nodes;
  Corasick(){ root = new_Node(NULL, 0); }

  void add(string &s){
    trie t = root;
    for(auto c : s){ c -= off;
      if(!t->nxt[c])
        t->nxt[c] = new Node(t, c);
      t = t->nxt[c];
    }
    t->idw = nwords++; //cuidado com strings iguais! use
         vector
  }
  void buildSufixLink(){
    deque<trie> q(1, root);

    while(!q.empty()){
      trie t = q.front();
      q.pop_front();

      if(trie w = t->p){
        do w = w->sl; while(w && !w->nxt[t->c]);
        t->sl = w ? w->nxt[t->c] : root;
        t->ol = t->sl->idw == -1 ? t->sl->ol : t->sl;
      }

      for(int c=0; c<ALPHA; c++)
        if(t->nxt[c])
          q.push_back(t->nxt[c]);
    }
  }
  vector<bool> findPattern(string &s){
    vector<bool> ans(nwords, 0);
    trie w = root;
    for(auto c : s){ c -= off;
      while(w && !w->nxt[c]) w = w->sl;  // trie next(w, c)
      w = w ? w->nxt[c] : root;

      for(trie z=w, nl; z; nl=z->ol, z->ol=NULL, z=nl)
        if(z->idw != -1) //get ALL occ: dont delete ol (may
             slow)
          ans[z->idw] = true;
    }
    return ans;
  }
  trie new_Node(trie p, char c){
    nodes.push_back(new Node(p, c));
    nodes.back()->i = nodes.size()-1;
    return nodes.back();
  }
};
```

7.6 hash

```
const int MAXN = 1e6 + 5;

const ll MOD = 1e9 + 7; //WA? Muda o MOD e a base
const ll base = 153;
ll expb[MAXN];

void precalc(){
  expb[0] = 1;
  for(int i=1; i<MAXN; i++)
    expb[i] = (expb[i-1]*base)%MOD;
}

struct StringHash{
  vector<ll> hsh;

  StringHash(string &s){
    hsh.assign(s.size()+1, 0);
    for(int i=0; i<s.size(); i++)
      hsh[i+1] = (hsh[i] * base % MOD + s[i]) % MOD;
```

```
  }

  ll gethash(int l, int r){
    return (MOD + hsh[r+1] - hsh[l]*expb[r-l+1] % MOD ) %
        MOD;
  }
};
String Hash
precalc()     -> O(N)
StringHash() -> O(|S|)
gethash()     -> O(1)

StringHash hash(s); -> Cria uma struct de StringHash para a
     string s
hash.gethash(l, r); -> Retorna o hash do intervalo L R da
     string (0-Indexado)

IMPORTANTE! Chamar precalc() no inicio do codigo

const ll MOD  = 131'807'699; -> Big Prime Number
const ll base = 127;         -> Random number larger than
     the Alphabet
```

7.7 hash2

```
String Hash - Double Hash
precalc()     -> O(N)
StringHash() -> O(|S|)
gethash()     -> O(1)

StringHash hash(s); -> Cria o Hash da string s
hash.gethash(l, r); -> Hash [L,R] (0-Indexado)
```

```
const int MAXN = 1e6 + 5;

const ll MOD1 = 131'807'699;
const ll MOD2 = 1e9 + 9;
const ll base = 157;

ll expb1[MAXN], expb2[MAXN];

#warning "Call precalc() before use StringHash"
void precalc(){
  expb1[0] = expb2[0] = 1;

  for(int i=1;i<MAXN;i++)
    expb1[i] = expb1[i-1]*base % MOD1,
    expb2[i] = expb2[i-1]*base % MOD2;
}

struct StringHash{
  vector<pair<ll,ll>> hsh;
  string s; // comment S if you dont need it

  StringHash(string& s) : s(s){
    hsh.assign(s.size()+1, {0,0});

    for (int i=0;i<s.size();i++)
      hsh[i+1].first  = ( hsh[i].first *base % MOD1 + s[i] )
          % MOD1,
      hsh[i+1].second = ( hsh[i].second*base % MOD2 + s[i] )
          % MOD2;
  }

  ll gethash(int a,int b){
    ll h1 = (MOD1+ hsh[b+1].first  - hsh[a].first *expb1[b-a
        +1] % MOD1) % MOD1;
    ll h2 = (MOD2+ hsh[b+1].second - hsh[a].second*expb2[b-a
        +1] % MOD2) % MOD2;
    return (h1<<32) | h2;
  }
};

// OPTIONAL
int firstDiff(StringHash& a, int la, int ra, StringHash& b,
     int lb, int rb){
  int l=0, r=min(ra-la, rb-lb), diff=r+1;
  while(l <= r){
    int m = (l+r)/2;
    if(a.gethash(la, la+m) == b.gethash(lb, lb+m)) l = m+1;
    else r = m-1, diff = m;
  }
  return diff;
}

int hshComp(StringHash& a, int la, int ra, StringHash& b,
     int lb, int rb){
  int diff = firstDiff(a, la, ra, b, lb, rb);
  if(diff > ra-la && ra-la == rb-lb) return 0; //equal
  if(diff > ra-la || diff  >  rb-lb) return ra-la < rb-lb ?
      -2 : +2; //prefix of the other
  return a.s[la+diff] < b.s[lb+diff] ? -1 : +1;
}

ll merge_hashes(ll hash_A, ll hash_B, int len_B) {
    ll h1_A = (hash_A >> 32) & 0xFFFFFFFF;
    ll h2_A = hash_A & 0xFFFFFFFF;

    ll h1_B = (hash_B >> 32) & 0xFFFFFFFF;
    ll h2_B = hash_B & 0xFFFFFFFF;

    ll h1_C = (h1_A * expb1[len_B] + h1_B) % MOD1;
    ll h2_C = (h2_A * expb2[len_B] + h2_B) % MOD2;

    return (h1_C << 32) | h2_C;
}
```

7.8 hash2A

```
String Hash - Double Hash
precalc()     -> O(N)
StringHash() -> O(|S|)
gethash()     -> O(1)
merge_hashes() -> O(1)

StringHash hash(s); -> Cria o Hash da string s
hash.gethash(l, r); -> Hash [L,R] (0-Indexado)
merge_hashes(hsA, hsB, szB) -> hash da concatenacao a+b
```

```
const int MAXN = 1e6 + 5;

const array<ll, 2> mod = {131'807'699, 1e9 + 9};
const ll base = 157;

array<ll, 2> expb[MAXN];

#warning "Call precalc() before use StringHash"
void precalc(){
  expb[0][0] = expb[0][1] = 1;

  for(int i=1, j;i<MAXN;i++) for(j=1; ~j; j--)
    expb[i][j] = expb[i-1][j]*base % mod[j];
}

struct StringHash{
  vector<array<ll,2>> hsh;
  string s; // comment S if you dont need it

  StringHash(string& s) : s(s){
    hsh.assign(s.size()+1, {0,0});

    for(int i=0, j;i<s.size();i++) for(j=1; ~j; j--)
      hsh[i+1][j] = ( hsh[i][j]*base + s[i] ) % mod[j];
  }
  ll gethash(int a,int b){
    ll h1 = (mod[0]+ hsh[b+1][0] - hsh[a][0]*expb[b-a+1][0]
        % mod[0]) % mod[0];
    ll h2 = (mod[1]+ hsh[b+1][1] - hsh[a][1]*expb[b-a+1][1]
        % mod[1]) % mod[1];
    return (h1<<32) | h2;
  }
};

// OPTIONAL
int firstDiff(StringHash& a, int la, int ra, StringHash& b,
     int lb, int rb){
  int l=0, r=min(ra-la, rb-lb), diff=r+1;
  while(l <= r){
    int m = (l+r)/2;
    if(a.gethash(la, la+m) == b.gethash(lb, lb+m)) l = m+1;
    else r = m-1, diff = m;
  }
  return diff;
}

int hshComp(StringHash& a, int la, int ra, StringHash& b,
     int lb, int rb){
  int diff = firstDiff(a, la, ra, b, lb, rb);
  if(diff > ra-la && ra-la == rb-lb) return 0; //equal
  if(diff > ra-la || diff  >  rb-lb) return ra-la < rb-lb ?
      -2 : +2; //prefix of the other
  return a.s[la+diff] < b.s[lb+diff] ? -1 : +1;
}

ll merge_hashes(ll hash_A, ll hash_B, int len_B) {
    ll h1_A = (hash_A >> 32) & 0xFFFFFFFF;
    ll h2_A = hash_A & 0xFFFFFFFF;

    ll h1_B = (hash_B >> 32) & 0xFFFFFFFF;
    ll h2_B = hash_B & 0xFFFFFFFF;

    ll h1_C = (h1_A * expb[len_B][0] + h1_B) % mod[0];
    ll h2_C = (h2_A * expb[len_B][1] + h2_B) % mod[1];

    return (h1_C << 32) | h2_C;
}
```

7.9 trie

```
Trie - Arvore de Prefixos
insert(P) - O(|P|)
count(P)  - O(|P|)
sigma - Tamanho do alfabeto
off   - primeiro simbolo do alfabeto (offset)
```

```
const int ALPHA = 26, off = 'a';
struct Node {
```

```
    array<Node*, ALPHA> nxt;
    int terminal = 0;
    Node() { nxt.fill(NULL); }
};

struct Trie {
  Node* root;
  Trie(){ root = new Node(); }

  void add(string &s){
    Node* t = root;
    for(auto c : s){ c -= off;
      if(!t->nxt[c])
        t->nxt[c] = new Node();
      t = t->nxt[c];
    }
    t->terminal++;
  }

  int count(string &s){
    Node* t = root;
    for(auto c : s){ c -= off;
      if(!t->nxt[c]) return 0;
      t = t->nxt[c];
    }
    return t->terminal;
  }
};
```

8 Theorems

8.1 Combinatorics

- **Sums:**

$\sum_{k=0}^{n} k = \frac{n(n+1)}{2}$
$\sum_{k=a}^{b} k = \frac{(a+b)(b-a+1)}{2}$
$\sum_{k=0}^{n} k^2 = \frac{n(n+1)(2n+1)}{6}$
$\sum_{k=0}^{n} k^3 = \frac{n^2(n+1)^2}{4}$
$\sum_{k=0}^{n} k^4 = \frac{6n^5+15n^4+10n^3-n}{30}$
$\sum_{k=0}^{n} k^5 = \frac{2n^6+6n^5+5n^4-n^2}{12}$
$\sum_{k=0}^{n} x^k = \frac{x^{n+1}-1}{x-1}$
$\sum_{k=0}^{n} kx^k = \frac{x-(n+1)x^{n+1}+nx^{n+2}}{(x-1)^2}$
$1+x+x^2+\cdots = \frac{1}{1-x}$

$\binom{n}{k} = \frac{n!}{(n-k)!k!}$
$\binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}$
$\binom{n+1}{k} = \frac{n+1}{n-k+1}\binom{n}{k}$
$\binom{n}{k+1} = \frac{n-k}{k+1}\binom{n}{k}$
$\binom{n}{k} = \frac{n}{n-k}\binom{n-1}{k}$
$\binom{n}{k} = \frac{n-k+1}{k}\binom{n}{k-1}$
$12! \approx 2^{28.8}$
$20! \approx 2^{61.1}$

  - **Hockey-stick identity:** $\sum_{i=r}^{n} \binom{i}{r} = \binom{n+1}{r+1}$
  - **Number of ways to color n-objects with r-colors** if all colors must be used at least once:

$$\sum_{k=0}^{r} \binom{r}{k} (-1)^{r-k} k^n \quad \text{or} \quad \sum_{k=0}^{r} \binom{r}{r-k} (-1)^k (r-k)^n$$

- **Binomial coefficients:**
  - Number of ways to pick a multiset of size k from n elements: $\binom{n+k-1}{k}$
  - Number of n-tuples of non-negative integers with sum s: $\binom{s+n-1}{n-1}$, at most s: $\binom{s+n}{n}$
  - Number of n-tuples of positive integers with sum s: $\binom{s-1}{n-1}$
  - Number of lattice paths from $(0,0)$ to (a,b), restricted to east and north steps: $\binom{a+b}{a}$
- **Multinomial theorem:** $(a_1 + \cdots + a_k)^n = \sum \binom{n}{n_1,\ldots,n_k} a_1^{n_1} \ldots a_k^{n_k}$, where $n_i \geq 0$ and $\sum n_i = n$.

$$\binom{n}{n_1,\ldots,n_k} = M(n_1,\ldots,n_k) = \frac{n!}{n_1!\ldots n_k!}$$

$$M(a,\ldots,b,c,\ldots) = M(a+\cdots+b,c,\ldots)M(a,\ldots,b)$$

- **Catalan numbers:**
  - $C_n = \frac{1}{n+1}\binom{2n}{n} = \frac{(2n)!}{(n+1)!n!}$ with $n \geq 0$, $C_0 = 1$ and $C_{n+1} = \frac{2(2n+1)}{n+2} C_n$. Also, $C_n = \sum_{i=0}^{n-1} C_i C_{n-1-i}$
  - Sequence: 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, 208012, 742900, 2674440, 9694845, 35357670
  - C_n is the number of: properly nested sequences of n pairs of parentheses; rooted ordered binary trees with $n+1$ leaves; triangulations of a convex $(n+2)$-gon.
- **Derangements:** Number of permutations of $n = 0, 1, 2, \ldots$ elements without fixed points is $1, 0, 1, 2, 9, 44, 265, 1854, 14833, \ldots$
  Recurrence: $D_n = (n-1)(D_{n-1} + D_{n-2}) = nD_{n-1} + (-1)^n$.
  Corollary: number of permutations with exactly k fixed points is $\binom{n}{k} D_{n-k}$.
- **Stirling numbers of 1^{st} kind:** $s_{n,k}$ is $(-1)^{n-k}$ times the number of permutations of n elements with exactly k permutation cycles.
  $|s_{n,k}| = |s_{n-1,k-1}| + (n-1)|s_{n-1,k}|$. $\sum_{k=0}^{n} s_{n,k} x^k = x^{\underline{n}}$
- **Stirling numbers of 2^{nd} kind:** $S_{n,k}$ is the number of ways to partition a set of n elements into exactly k non-empty subsets.
  $S_{n,k} = S_{n-1,k-1} + kS_{n-1,k}$. $S_{n,1} = S_{n,n} = 1$.
  $x^n = \sum_{k=0}^{n} S_{n,k} x^{\underline{k}}$
- **Bell numbers:** B_n is the number of partitions of n elements.
  $B_0, \ldots = 1, 1, 2, 5, 15, 52, 203, \ldots$
  $B_{n+1} = \sum_{k=0}^{n} \binom{n}{k} B_k = \sum_{k=1}^{n} S_{n,k}$.
  Bell triangle: $B_r = a_{r,1} = a_{r-1,r-1}$, $a_{r,c} = a_{r-1,c-1} + a_{r,c-1}$.
- **Bernoulli numbers:** $\sum_{k=0}^{m-1} k^n = \frac{1}{n+1} \sum_{k=0}^{n} \binom{n+1}{k} B_k m^{n+1-k}$.
  $\sum_{j=0}^{m} \binom{m+1}{j} B_j = 0$. $B_0 = 1$, $B_1 = -\frac{1}{2}$. $B_n = 0$, for all odd $n \neq 1$.
- **Eulerian numbers:** $E(n,k)$ is the number of permutations with exactly k descents ($i : \pi_i < \pi_{i+1}$) / ascents ($\pi_i > \pi_{i+1}$) / excedances ($\pi_i > i$) / $k+1$ weak excedances ($\pi_i \geq i$).
  Formula: $E(n,k) = (k+1)E(n-1,k) + (n-k)E(n-1,k-1)$. $x^n = \sum_{k=0}^{n-1} E(n,k)\binom{x+k}{n}$.
- **Burnside's lemma:** The number of orbits under group G's action on set X:

$$|X/G| = \frac{1}{|G|} \sum_{g \in G} |X_g|$$

  where $X_g = \{x \in X : g(x) = x\}$ ("Average number of fixed points.")
  Let $w(x)$ be weight of x's orbit. Sum of all orbits' weights:

$$\sum_{o \in X/G} w(o) = \frac{1}{|G|} \sum_{g \in G} \sum_{x \in X_g} w(x)$$

8.2 Games

- **Grundy numbers:** For a two-player, normal-play (last to move wins) game on a graph (V,E): $G(x) = \text{mex}(\{G(y) : (x,y) \in E\})$, where $\text{mex}(S) = \min\{n \geq 0 : n \notin S\}$. x is losing iff $G(x) = 0$.
- **Sums of games:**
  - *Player chooses a game and makes a move in it.* Grundy number of a position is xor of grundy numbers of positions in summed games.
  - *Player chooses a non-empty subset of games (possibly, all) and makes moves in all of them.* A position is losing iff each game is in a losing position.
  - *Player chooses a proper subset of games (not empty and not all), and makes moves in all chosen ones.* A position is losing iff grundy numbers of all games are equal.
  - *Player must move in all games, and loses if can't move in some game.* A position is losing if any of the games is in a losing position.
- **Misère Nim:** A position with pile sizes $a_1, a_2, \ldots, a_n \geq 1$, not all equal to 1, is losing iff $a_1 \oplus a_2 \oplus \cdots \oplus a_n = 0$ (like in normal nim.) A position with n piles of size 1 is losing iff n is *odd*.

8.3 Geometria

- **Fórmula de Euler:** Em um grafo planar ou poliedro convexo, temos: $V - E + F = 2$ onde V é o número de vértices, E o número de arestas e F o número de faces.
- **Teorema de Pick:** Para polígonos com vértices em coordenadas inteiras:

$$\text{Área} = i + \frac{b}{2} - 1$$

onde i é o número de pontos interiores e b o número de pontos sobre o perímetro.

- **Teorema das Duas Orelhas (Two Ears Theorem):** Todo polígono simples com mais de três vértices possui pelo menos duas "orelhas"— vértices que podem ser removidos sem gerar interseções. A remoção repetida das orelhas resulta em uma triangulação do polígono.
- **Incentro de um Triângulo:** É o ponto de interseção das bissetrizes internas e centro da circunferência inscrita. Se a, b e c são os comprimentos dos lados opostos aos vértices $A(X_a, Y_a)$, $B(X_b, Y_b)$ e $C(X_c, Y_c)$, então o incentro (X, Y) é dado por:

$$X = \frac{aX_a + bX_b + cX_c}{a+b+c}, \quad Y = \frac{aY_a + bY_b + cY_c}{a+b+c}$$

- **Triangulação de Delaunay:** Uma triangulação de um conjunto de pontos no plano tal que nenhum ponto está dentro do círculo circunscrito de qualquer triângulo. Essa triangulação:
  - Maximiza o menor ângulo entre todos os triângulos.
  - Contém a árvore geradora mínima (MST) euclidiana como subconjunto.
- **Fórmula de Brahmagupta:** Para calcular a área de um quadrilátero cíclico (todos os vértices sobre uma circunferência), com lados a, b, c e d:

$$s = \frac{a+b+c+d}{2}, \quad \text{Área} = \sqrt{(s-a)(s-b)(s-c)(s-d)}$$

Se $d = 0$ (ou seja, um triângulo), ela se reduz à fórmula de Heron:

$$\text{Área} = \sqrt{(s-a)(s-b)(s-c)s}$$

8.4 Point in Polygon e Truques Geométricos

- **Ray Casting (Curva de Jordan):** Resolve ponto em polígono geral em $O(N)$. Traça-se um raio do ponto P até o infinito. Se cruzar as arestas um número ímpar de vezes, P está dentro. **Truque dos Coprimos:** Para evitar *edge cases* (raio bater num vértice ou aresta colinear), use um vetor de direção com primos altos:

$$P_\infty = (P.x + 10^9 + 7, P.y + 10^9 + 9)$$

Isso inclina a reta o suficiente para garantir que ela nunca acerte uma coordenada inteira exata, reduzindo o problema a uma simples checagem de interseção de segmentos.

- **Winding Number (Soma dos Ângulos):** Alternativa em $O(N)$ baseada na soma dos ângulos entre P e vértices adjacentes:

$$\sum \text{Ângulos} = \begin{cases} \pm 2\pi, & \text{dentro} \\ 0, & \text{fora} \end{cases}$$

Dica: Implemente as checagens usando apenas **Produto Vetorial** (verificando semiplanos) para manter a aritmética inteira e fugir da imprecisão do `atan2`.

- **Prefix Sum de Áreas (Shoelace):** Usado para responder *queries* de área de sub-polígonos (cortes) em $O(1)$. Mantém-se o prefixo do produto vetorial:

$$S[k] = S[k-1] + (V_{k-1} \times V_k)$$

A área da fatia conectando V_i a V_j é:

$$\text{Area}_{i,j} = \frac{1}{2}\Big|S[j] - S[i] + (V_j \times V_i)\Big|$$

- **Busca Binária em Polígono Convexo:** Resolve ponto em polígono estritamente convexo em $O(\log N)$.
  - Fixe o vértice V_0.
  - Use busca binária baseada no `ccw` (Cross Product) para achar o triângulo angular (V_0, V_k, V_{k+1}) que contém P.
  - Verifique se P está no semiplano interno da aresta $V_k \to V_{k+1}$.

8.5 Grafos

- **Fórmula de Euler (para grafos planares):**

$$V - E + F = 2$$

onde V é o número de vértices, E o número de arestas e F o número de faces.

- **Handshaking Lemma:** O número de vértices com grau ímpar em um grafo é par.
- **Teorema de Kirchhoff (contagem de árvores geradoras):** Monte a matriz M tal que:

$$M_{i,i} = \deg(i), \quad M_{i,j} = \begin{cases} -1 & \text{se existe aresta } i-j \\ 0 & \text{caso contrário} \end{cases}$$

O número de árvores geradoras (spanning trees) é o determinante de qualquer co-fator de M (remova uma linha e uma coluna).

- **Condições para Caminho Hamiltoniano:**
  - **Teorema de Dirac:** Se todos os vértices têm grau $\geq n/2$, o grafo contém um caminho Hamiltoniano.
  - **Teorema de Ore:** Se para todo par de vértices não adjacentes u e v, temos $\deg(u) + \deg(v) \geq n$, então o grafo possui caminho Hamiltoniano.
- **Algoritmo de Borůvka:** Enquanto o grafo não estiver conexo, para cada componente conexa escolha a aresta de menor custo que sai dela. Essa técnica constrói a árvore geradora mínima (MST).
- **Árvores:**
  - Existem C_n árvores binárias com n vértices (C_n é o n-ésimo número de Catalan).
  - Existem C_{n-1} árvores enraizadas com n vértices.
  - **Fórmula de Cayley:** Existem n^{n-2} árvores com vértices rotulados de 1 a n.
  - **Código de Prüfer:** Remova iterativamente a folha com menor rótulo e adicione o rótulo do vizinho ao código até restarem dois vértices.
- **Fluxo em Redes:**
  - **Corte Mínimo:** Após execução do algoritmo de fluxo máximo, um vértice u está do lado da fonte se level$[u] \neq -1$.
  - **Máximo de Caminhos Disjuntos:**
    - **Arestas disjuntas:** Use fluxo máximo com capacidades iguais a 1 em todas as arestas.
    - **Vértices disjuntos:** Divida cada vértice v em v_{in} e v_{out}, conectados por aresta de capacidade 1. As arestas que entram vão para v_{in} e as que saem saem de v_{out}.
  - **Teorema de König:** Em um grafo bipartido:

    Cobertura mínima de vértices = Matching máximo

    O complemento da cobertura mínima de vértices é o conjunto independente máximo.

- **Coberturas:**
  - **Vertex Cover mínimo:** Os vértices da partição X que **não** estão do lado da fonte no corte mínimo, e os vértices da partição Y que **estão** do lado da fonte.
  - **Independent Set máximo:** Complementar da cobertura mínima de vértices.
  - **Edge Cover mínimo:** É $N -$ matching, pegando as arestas do matching e mais quaisquer arestas restantes para cobrir os vértices descobertos.
- **Path Cover:**
  - **Node-disjoint path cover mínimo:** Duplicar vértices em tipo A e tipo B e criar grafo bipartido com arestas de $A \to B$. O path cover é $N -$ matching.
  - **General path cover mínimo:** Criar arestas de $A \to B$ sempre que houver caminho de A para B no grafo. O resultado também é $N -$ matching.
- **Teorema de Dilworth:** O path cover mínimo em um grafo dirigido acíclico é igual à **antichain máxima** (conjunto de vértices sem caminhos entre eles).
- **Teorema do Casamento de Hall:** Um grafo bipartido possui um matching completo do lado X se:

$$\forall W \subseteq X, \quad |W| \leq |\text{vizinhos}(W)|$$

- **Fluxo Viável com Capacidades Inferiores e Superiores:** Para rede sem fonte e sumidouro:
  - Substituir a capacidade de cada aresta por $c_{\text{upper}} - c_{\text{lower}}$
  - Criar nova fonte S e sumidouro T
  - Para cada vértice v, compute:

$$M[v] = \sum_{\text{arestas entrando}} c_{\text{lower}} - \sum_{\text{arestas saindo}} c_{\text{lower}}$$

  - Se $M[v] > 0$, adicione aresta (S, v) com capacidade $M[v]$; se $M[v] < 0$, adicione (v, T) com capacidade $-M[v]$.
  - Se todas as arestas de S estão saturadas no fluxo máximo, então um fluxo viável existe. O fluxo viável final é o fluxo computado mais os valores de c_{lower}.

8.6 Propriedades Matemáticas

- **Lagrange:** Todo número inteiro pode ser representado como soma de 4 quadrados.
- **Zeckendorf:** Todo número pode ser representado como soma de números de Fibonacci diferentes e não consecutivos.
- **Tripla de Pitágoras (Euclides):** Toda tripla pitagórica primitiva pode ser gerada por $(n^2-m^2, 2nm, n^2+m^2)$ onde n e m são coprimos e um deles é par.
- **Progressão Geométrica:** $S_n = a_1 \cdot \frac{q^n-1}{q-1}$
- **Soma de quadrados e cubos:** $\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$, $\sum_{i=1}^{n} i^3 = \frac{n^2(n+1)^2}{4}$
- **Chicken McNugget:** Para dois coprimos x e y, o número de inteiros que não podem ser expressos como $ax + by$ é $(x-1)(y-1)/2$. O maior inteiro não representável é $xy - x - y$.
- **Primos até 200:** 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97, 101, 103, 107, 109, 113, 127, 131, 137, 139, 149, 151, 157, 163, 167, 173, 179, 181, 191, 193, 197, 199
- **Fibonacci** $[0, 20]$: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987, 1597, 2584, 4181, 6765.
- **Fatorial** $[0, 15]$: 1, 1, 2, 6, 24, 120, 720, 5040, 40320, 362880, 3628800, 39916800, 479001600, 6227020800, 87178291200, 1307674368000.
- **Potencias**
  2^n $[1, 16]$: 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048, 4096, 8192, 16384, 32768, 65536
  3^n $[1, 13]$: 3, 9, 27, 81, 243, 729, 2187, 6561, 19683, 59049, 177147, 531441, 1594323
  5^n $[1, 12]$: 5, 25, 125, 625, 3125, 15625, 78125, 390625, 1953125, 9765625, 48828125, 244140625
  7^n $[1, 9]$: 7, 49, 343, 2401, 16807, 117649, 823543, 5764801, 40353607, 282475249
- **Totiente de Euler:** $\varphi(n)$ conta quantos números entre 1 e n são coprimos com n.
  Seq.$[1, 32]$: 1, 1, 2, 2, 4, 2, 6, 4, 6, 4, 10, 4, 12, 6, 8, 8, 16, 6, 18, 8, 12, 10, 22, 8, 20, 12, 18, 12, 28, 8, 30

Primos

- **Goldbach:** Todo número par $n > 2$ pode ser representado como $n = a + b$, onde a e b são primos.
- **Primos Gêmeos:** Existem infinitos pares de primos p, $p + 2$.
- **Legendre:** Sempre existe um primo entre n^2 e $(n+1)^2$.
- **Wilson:** n é primo se e somente se $(n-1)! \bmod n = n - 1$.

Combinatória

- **Propriedades de Coeficientes Binomiais:**

$$\binom{n}{k} = \binom{n}{n-k} = \frac{n}{k}\binom{n-1}{k-1} \quad \sum_{i=0}^{k} \binom{m}{i}\binom{n}{k-i} = \binom{m+n}{k}$$

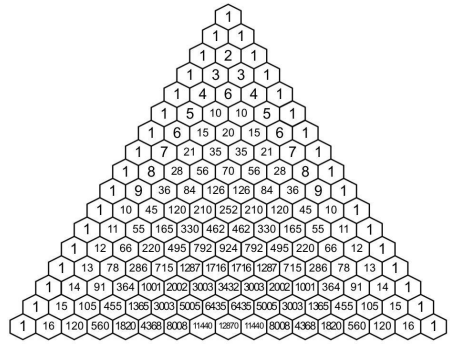
$$\sum_{i=0}^{m} \binom{n}{i}(-1)^i = \binom{n-1}{m}(-1)^m \sum_{i=0}^{n} \binom{n}{i} = 2^n$$

$$\sum_{i=0}^{n} \binom{n}{i} i = n2^{n-1} \qquad \sum_{i=0}^{n} \binom{i}{k} = \binom{n+1}{k+1}$$

$$\sum_{i=0}^{n} \binom{n-i}{i} = F_{n+1} \qquad \sum_{i=0}^{m} \binom{n+i}{i} = \binom{n+m+1}{m}$$

$$\sum_{i=0}^{n} \binom{n}{i}^2 = \binom{2n}{n}$$

- **Triângulo de Pascal**



- **Números de Catalan:** expressões de parênteses bem formadas, #binary trees com n+1 nodes, etc. $C_0 = 1$, e:

$$C_n = \frac{1}{n+1}\binom{2n}{n} = \sum_{i=0}^{n-1} C_i C_{n-1-i}$$

  Seq.$[0, 20]$: 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, 208012, 742900, 2674440, 9694845, 35357670, 129644790, 477638700, 1767263190, 6564120420.

- **Burnside Lemma:** Número de colares diferentes (sem contar rotações), com m cores e comprimento n:

$$\frac{1}{n}\left(m^n + \sum_{i=1}^{n-1} m^{\gcd(i,n)}\right)$$

- **Inversão de Möbius:** Útil para inclusão e exclusão nos primos. Seq.[1,30]: 1, -1, -1, 0, -1, 1, -1, 0, 0, 1, -1, 0, 1, 1, 0, -1, 1, 0, 0, 1, -1, 1, 0, -1, -1, 0, 1, 0, 0, 1. $\sum_{d|n}\mu(d) = 0$ if $n > 1$ else 1
- **Lindström-Gessel-Viennot:** A quantidade de caminhos disjuntos em um grid pode ser computada como o determinante da matriz do número de caminhos.
- **Numeros de Stirling:**
  - **First Type:** $|s(n,m)|$ é definido como a quantidade de permutações de n elementos que contém exatamente m ciclos de permutação. Tem aplicações em inclusão e exclusão.

$$\begin{bmatrix} n \\ k \end{bmatrix} = (n-1)\begin{bmatrix} n-1 \\ k \end{bmatrix} + \begin{bmatrix} n-1 \\ k-1 \end{bmatrix} = (-1)^{n-k} s(n,k)$$

$$\begin{bmatrix} n \\ 1 \end{bmatrix} = (n-1)! \quad , \quad \begin{bmatrix} n \\ n \end{bmatrix} = 1 \quad , \quad \begin{bmatrix} n+1 \\ m+1 \end{bmatrix} = \sum_i \begin{bmatrix} n \\ i \end{bmatrix} \binom{i}{m}$$

$n\|k$	0	1	2	3	4	5	6
0	1						
1		1					
2		−1	1				
3		2	−3	1			
4		−6	11	−6	1		
5		24	−50	35	−10	1	
6		−120	274	−225	85	−15	1

  - **Second Type:** O Stirling set $\left\{ n \atop k \right\}$ conta maneiras de particionar um conjunto de n elementos em k sub-conjuntos não vazios. Equivalente a Rhyming Schemes de n posições e k símbolos. (exemplo n=3: k=1 {aaa}; k=2 {aab, aba, abb}; k=3 {abc})

$$\left\{ n \atop k \right\} = k\left\{ n-1 \atop k \right\} + \left\{ n-1 \atop k-1 \right\}$$

$$\left\{ n \atop 1 \right\} = \left\{ n \atop n \right\} = 1 \quad , \quad \left\{ n+1 \atop k+1 \right\} = \sum_i \left\{ i \atop k \right\} \binom{n}{i}$$

$n\|k$	0	1	2	3	4	5	6
0	1						
1		1					
2		1	1				
3		1	3	1			
4		1	7	6	1		
5		1	15	25	10	1	
6		1	31	90	65	15	1

- **Números de Bell** B_n**:** numero de partições de um conjunto de n elementos. $B_n = \sum_i \left\{ n \atop i \right\} = \sum_i \binom{n-1}{i} B_i$. [0,12]: 1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, 115975, 678570, 4213597.
- **Eulerian numbers (first order):**
  O Eulerian number $\left\langle n \atop k \right\rangle$ ou $A(n,k)$ conta o número de permutações de tamanho n com exatamente k *ascents* ($p_i < p_{i+1}$).

$$\left\langle n \atop k \right\rangle = (k+1)\left\langle n-1 \atop k \right\rangle + (n-k)\left\langle n-1 \atop k-1 \right\rangle$$

$$= \sum_{i=0}^{k} (-1)^i \binom{n+1}{i} (k+1-i)^n$$

$$\left\langle 1 \atop 0 \right\rangle = 1, \qquad \left\langle n \atop 1 \right\rangle = 2^n - n - 1, \qquad \sum_{k=0}^{n-1} \left\langle n \atop k \right\rangle = n!$$

$n\|k$	0	1	2	3	4	5	6
1	1						
2	1	1					
3	1	4	1				
4	1	11	11	1			
5	1	26	66	26	1		
6	1	57	302	302	57	1	
7	1	120	1191	2416	1191	120	1

Modular

- **Fermat:** Se p é primo, então $a^{p-1} \equiv 1 \mod p$. Se x e m são coprimos e m primo, então $x^k \equiv x^{k \bmod (m-1)} \mod m$. *Euler:* $x^{\varphi(m)} \equiv 1 \mod m$. $\varphi(m)$ é o totiente de Euler.
- **Teorema Chinês do Resto:** Dado um sistema de congruências: $x \equiv a_1 \mod m_1, \quad \ldots, \quad x \equiv a_n \mod m_n$ com m_i coprimos dois a dois. E seja $M_i = \frac{m_1 m_2 \cdots m_n}{m_i}$ e $N_i = M_i^{-1} \mod m_i$. Então a solução é dada por $x = \sum_{i=1}^{n} a_i M_i N_i$ Outras soluções são obtidas somando $m_1 m_2 \cdots m_n$.

Probabilidade

- **Bertrand Ballot:** Com $p > q$ votos, a probabilidade de sempre haver mais votos do tipo A do que B até o fim é: $\frac{p-q}{p+q}$ Permitindo empates: $\frac{p+1-q}{p+1}$. Multiplicando pela combinação total $\binom{p+q}{q}$, obtém-se o número de possibilidades.
- **Linearidade da Esperança:** $E[aX + bY] = aE[X] + bE[Y]$
- **Variância:** $\mathrm{Var}(X) = E[(X-\mu)^2] = E[X^2] - E[X]^2$
- **Uniforme:** $X \in \{a, a+1, \ldots, b\}$, $E[X] = \frac{a+b}{2}$
- **Binomial:** n tentativas com probabilidade p de sucesso: $P(X = x) = \binom{n}{x} p^x (1-p)^{n-x}$, $E[X] = np$
- **Geométrica:** Número de tentativas até o primeiro sucesso: $P(X = x) = (1-p)^{x-1}p$, $E[X] = \frac{1}{p}$

9 Number Theory

Linear diophantine equation: . $ax + by = c$. Let $d = \gcd(a,b)$. A solution exists iff $d|c$. If (x_0, y_0) is any solution, then all solutions are given by $(x, y) = (x_0 + \frac{b}{d}t, y_0 - \frac{a}{d}t)$, $t \in \mathbb{Z}$. To find some solution (x_0, y_0), use extended GCD to solve $ax_0 + by_0 = d = \gcd(a,b)$, and multiply its solutions by $\frac{c}{d}$.

Linear diophantine equation in n variables: $a_1x_1 + \cdots + a_nx_n = c$ has solutions iff $\gcd(a_1, \ldots, a_n)|c$. To find some solution, let $b = \gcd(a_2, \ldots, a_n)$, solve $a_1x_1 + by = c$, and iterate with $a_2x_2 + \cdots = y$.

Extended GCD:

```
// Finds g = gcd(a,b) and x, y such that ax+by=g.
// Bounds: |x|<=b+1, |y|<=a+1.
void gcdext(int &g, int &x, int &y, int a, int b)
{ if (b == 0) { g = a; x = 1; y = 0; }
  else        { gcdext(g, y, x, b, a % b); y = y -
```

Multiplicative inverse of a modulo m: x in $ax+my = 1$, or $a^{\phi(m)-1} \pmod m$.

Chinese Remainder Theorem: . System $x \equiv a_i \pmod{m_i}$ for $i = 1, \ldots, n$, with pairwise relatively-prime m_i has a unique solution modulo $M = m_1 m_2 \ldots m_n$: $x = a_1 b_1 \frac{M}{m_1} + \cdots + a_n b_n \frac{M}{m_n} \pmod M$, where b_i is modular inverse of $\frac{M}{m_i}$ modulo m_i.

System $x \equiv a \pmod m$, $x \equiv b \pmod n$ has solutions iff $a \equiv b \pmod g$, where $g = \gcd(m,n)$. The solution is unique modulo $L = \frac{mn}{g}$, and equals: $x \equiv a + T(b-a)m/g \equiv b + S(a-b)n/g \pmod L$, where S and T are integer solutions of $mT + nS = \gcd(m,n)$.

Prime-counting function: . $\pi(n) = |\{p \le n : p \text{ is prime}\}|$. $n/\ln(n) < \pi(n) < 1.3n/\ln(n)$. $\pi(1000) = 168$, $\pi(10^6) = 78498$, $\pi(10^9) = 50\ 847\ 534$. n-th prime $\approx n \ln n$.

Miller-Rabin's primality test: . Given $n = 2^r s + 1$ with odd s, and a random integer $1 < a < n$.
If $a^s \equiv 1 \pmod n$ or $a^{2^j s} \equiv -1 \pmod n$ for some $0 \le j \le r-1$, then n is a probable prime. With bases 2, 7 and 61, the test indentifies all composites below 2^{32}. Probability of failure for a random a is at most $1/4$.

Pollard-ρ: . Choose random x_1, and let $x_{i+1} = x_i^2 - 1 \pmod n$. Test $\gcd(n, x_{2^k+i} - x_{2^k})$ as possible n's factors for $k = 0, 1, \ldots$ Expected time to find a factor: $O(\sqrt{m})$, where m is smallest prime power in n's factorization. That's $O(n^{1/4})$ if you check $n = p^k$ as a special case before factorization.

Fermat primes: . A Fermat prime is a prime of form $2^{2^n} + 1$. The only known Fermat primes are 3, 5, 17, 257, 65537. A number of form $2^n + 1$ is prime only if it is a Fermat prime.

Fermat's Theorem: . Let m be a prime and x and m coprimes, then:

- $x^{m-1} \equiv 1 \mod m$
- $x^k \mod m = x^{k \mod (m-1)} \mod m$
- $x^{\phi(m)} \equiv 1 \mod m$

Perfect numbers: . $n > 1$ is called perfect if it equals sum of its proper divisors and 1. Even n is perfect iff $n = 2^{p-1}(2^p - 1)$ and $2^p - 1$ is prime (Mersenne's). No odd perfect numbers are yet found.

Carmichael numbers: . A positive composite n is a Carmichael number ($a^{n-1} \equiv 1 \pmod n$ for all a: $\gcd(a, n) = 1$), iff n is square-free, and for all prime divisors p of n, $p - 1$ divides $n - 1$.

Number/sum of divisors: . $\tau(p_1^{a_1} \ldots p_k^{a_k}) = \prod_{j=1}^{k}(a_j + 1)$. $\sigma(p_1^{a_1} \ldots p_k^{a_k}) = \prod_{j=1}^{k} \frac{p_j^{a_j+1}-1}{p_j-1}$.

Product of divisors: . $\mu(n) = n^{\frac{\tau(n)}{2}}$

- if p is a prime, then: $\mu(p^k) = p^{\frac{k(k+1)}{2}}$
- if a and b are coprimes, then: $\mu(ab) = \mu(a)^{\tau(b)}\mu(b)^{\tau(a)}$

Euler's phi function: . $\phi(n) = |\{m \in \mathbb{N}, m \leq n, \gcd(m, n) = 1\}|$.

- $\phi(mn) = \frac{\phi(m)\phi(n)\gcd(m,n)}{\phi(\gcd(m,n))}$.
- $\phi(p) = p - 1$ si p es primo
- $\phi(p^a) = p^a(1 - \frac{1}{p}) = p^{a-1}(p - 1)$
- $\phi(n) = n(1-\frac{1}{p_1})(1-\frac{1}{p_2})...(1-\frac{1}{p_k})$ donde p_i es primo y divide a n

Euler's theorem: . $a^{\phi(n)} \equiv 1 \pmod n$, if $\gcd(a, n) = 1$.

Wilson's theorem: . p is prime iff $(p - 1)! \equiv -1 \pmod p$.

Mobius function: . $\mu(1) = 1$. $\mu(n) = 0$, if n is not squarefree. $\mu(n) = (-1)^s$, if n is the product of s distinct primes. Let f, F be functions on positive integers. If for all $n \in N$, $F(n) = \sum_{d|n} f(d)$, then $f(n) = \sum_{d|n} \mu(d)F(\frac{n}{d})$, and vice versa. $\phi(n) = \sum_{d|n} \mu(d)\frac{n}{d}$. $\sum_{d|n} \mu(d) = 1$.
If f is multiplicative, then $\sum_{d|n} \mu(d)f(d) = \prod_{p|n}(1-f(p))$, $\sum_{d|n} \mu(d)^2 f(d) = \prod_{p|n}(1 + f(p))$.
$\sum_{d|n} \mu(d) = e(n) = [n == 1]$.
$S_f(n) = \prod_{p=1}(1+f(p_i)+f(p_i^2)+...+f(p_i^{e_i}))$, p - primes(n).

Legendre symbol: . If p is an odd prime, $a \in \mathbb{Z}$, then $\left(\frac{a}{p}\right)$ equals 0, if $p|a$; 1 if a is a quadratic residue modulo p; and -1 otherwise. Euler's criterion: $\left(\frac{a}{p}\right) = a^{(\frac{p-1}{2})} \pmod p$.

Jacobi symbol: . If $n = p_1^{a_1} \cdots p_k^{a_k}$ is odd, then $\left(\frac{a}{n}\right) = \prod_{i=1}^{k} \left(\frac{a}{p_i}\right)^{k_i}$.

Primitive roots: . If the order of g modulo m (min $n > 0$: $g^n \equiv 1 \pmod m$) is $\phi(m)$, then g is called a primitive root. If Z_m has a primitive root, then it has $\phi(\phi(m))$ distinct primitive roots. Z_m has a primitive root iff m is one of 2, 4, p^k, $2p^k$, where p is an odd prime. If Z_m has a primitive root g, then for all a coprime to m, there exists unique integer $i = \text{ind}_g(a)$ modulo $\phi(m)$, such that $g^i \equiv a \pmod m$. $\text{ind}_g(a)$ has logarithm-like properties: $\text{ind}(1) = 0$, $\text{ind}(ab) = \text{ind}(a) + \text{ind}(b)$.

If p is prime and a is not divisible by p, then congruence $x^n \equiv a \pmod p$ has $\gcd(n, p - 1)$ solutions if $a^{(p-1)/\gcd(n,p-1)} \equiv 1 \pmod p$, and no solutions otherwise. (Proof sketch: let g be a primitive root, and $g^i \equiv a \pmod p$, $g^u \equiv x \pmod p$. $x^n \equiv a \pmod p$ iff $g^{nu} \equiv g^i \pmod p$ iff $nu \equiv i \pmod p$.)

Discrete logarithm problem: . Find x from $a^x \equiv b \pmod m$. Can be solved in $O(\sqrt{m})$ time and space with a meet-in-the-middle trick. Let $n = \lceil\sqrt{m}\rceil$, and $x = ny - z$. Equation becomes $a^{ny} \equiv ba^z \pmod m$. Precompute all values that the RHS can take for $z = 0, 1, \ldots, n-1$, and brute force y on the LHS, each time checking whether there's a corresponding value for RHS.

Pythagorean triples: . Integer solutions of $x^2 + y^2 = z^2$ All relatively prime triples are given by: $x = 2mn, y = m^2 - n^2, z = m^2 + n^2$ where $m > n, \gcd(m, n) = 1$ and $m \not\equiv n \pmod 2$. All other triples are multiples of these. Equation $x^2 + y^2 = 2z^2$ is equivalent to $(\frac{x+y}{2})^2 + (\frac{x-y}{2})^2 = z^2$.

- Given an arbitrary pair of integers m and n with $m > n > 0$:
  $a = m^2 - n^2$, $b = 2mn$, $c = m^2 + n^2$
- The triple generated by Euclid's formula is primitive if and only if m and n are coprime and not both odd.
- To generate all Pythagorean triples uniquely:
  $a = k(m^2 - n^2)$, $b = k(2mn)$, $c = k(m^2 + n^2)$
- If m and n are two odd integer such that $m > n$, then:
  $a = mn$, $b = \frac{m^2-n^2}{2}$, $c = \frac{m^2+n^2}{2}$
- If $n = 1$ or 2 there are no solutions. Otherwise
  n is even: $((\frac{n^2}{4} - 1)^2 + n^2 = (\frac{n^2}{4} + 1)^2)$
  n is odd: $((\frac{n^2-1}{2})^2 + n^2 = (\frac{n^2+1}{2})^2)$

Postage stamps/McNuggets problem: . Let a, b be relatively-prime integers. There are exactly $\frac{1}{2}(a-1)(b-1)$ numbers *not* of form $ax + by$ $(x, y \geq 0)$, and the largest is $(a - 1)(b - 1) - 1 = ab - a - b$.

Fermat's two-squares theorem: . Odd prime p can be represented as a sum of two squares iff $p \equiv 1 \pmod 4$. A product of two sums of two squares is a sum of two squares. Thus, n is a sum of two squares iff every prime of form $p = 4k + 3$ occurs an even number of times in n's factorization.

RSA: . Let p and q be random distinct large primes, $n = pq$. Choose a small odd integer e, relatively prime to $\phi(n) = (p - 1)(q - 1)$, and let $d = e^{-1} \pmod{\phi(n)}$. Pairs (e, n) and (d, n) are the public and secret keys, respectively. Encryption is done by raising a message $M \in Z_n$ to the power e or d, modulo n.

9.1 DP

9.1.1 General

- **Divide and Conquer Optimization:** Utilizada em problemas do tipo:

$$dp[i][j] = \min_{k<j}\{dp[i-1][k] + C[k][j]\}$$

onde o objetivo é dividir o subsegmento até j em i segmentos com algum custo. A otimização é válida se:

$$A[i][j] \leq A[i][j+1]$$

onde $A[i][j]$ é o valor de k que minimiza a transição.

- **Knuth Optimization:** Aplicável quando:

$$dp[i][j] = \min_{i<k<j}\{dp[i][k] + dp[k][j]\} + C[i][j]$$

e a condição de monotonicidade é satisfeita:

$$A[i][j-1] \leq A[i][j] \leq A[i+1][j]$$

com $A[i][j]$ sendo o índice k que minimiza a transição.

- **Slope Trick:** Técnica usada para lidar com funções lineares por partes e convexas. A função é representada por pontos onde a derivada muda, que podem ser manipulados com `multiset` ou `heap`. Útil para manter o mínimo de funções acumuladas em forma de envelopes convexos.
- **Outras Técnicas e Truques Importantes:**
  - **FFT (Fast Fourier Transform):** Convolução eficiente de vetores.
  - **CHT (Convex Hull Trick):** Otimização para DP com funções lineares e monotonicidade.
  - **Aliens Trick:** Técnica para binarizar o custo em problemas de otimização paramétrica (geralmente em problemas com limite no número de grupos/segmentos).
  - **Bitset:** Utilizado para otimizações de espaço e tempo em DP de subconjuntos ou somas parciais, especialmente em problemas de mochila.